A PROJECT REPORT

On

"FURNIFLEX – FURNITURE RENTAL WEB APPLICATION"

Submitted by

Ms. Rehana Mansuri

In partial fulfilment for the award of the degree

Of

BACHELOR OF SCIENCE

In

COMPUTER SCIENCE

Under the guidance of

Ms. Zeba

Department of Computer Science



B.N.N. College, Bhiwandi.

Sem V

A.Y. 2026-2027

Padmashri Annasaheb Jadhav Bhartiya Samaj Unnati Mandal's

B.N.N COLLEGE (ARTS, SCIENCE AND COMMERCE)

(Affiliated to University of Mumbai)

BHIWANDI, MAHARASHTRA-421302

(Self-Funded Courses)

Department of Computer Science

A.Y. 2026-2027

CERTIFICATE

This is to certify that **Ms. Rehana Mansuri**, of **T.Y. B.Sc. (Sem.-V)** class has satisfactorily completed the Project "**FurniFlex – Furniture Rental Web Application**" to be submitted in the partial fulfilment for the award of **Bachelor of Science** in **Computer Science** during the academic year **2026 – 2027**.

Date of Submission:

Project Guide
(Ms. Zeba)

Head / In-Charge,
Department Computer Science

College Seal

Signature of Examiner

DECLARATION BY THE STUDENT

I, **Ms. Rehana Mansuri**, hereby declare that the project entitled "**FurniFlex – Furniture Rental Web Application**" submitted in the partial fulfilment for the award of Bachelor of Science in Computer Science during the academic year 2026 – 2027 is my original work and the project has not formed the basis for the award of any degree, associateship, fellowship or any other similar titles.

Signature of the Student:

Place:

Date:

ACKNOWLEDGEMENT

I would like to extend my heartiest thanks with a deep sense of gratitude and respect to all those who provided me immense help and guidance during my project work. I would like to thank my project guide **Ms. Zeba** for providing a clear vision about the system and for her valuable suggestions at every stage of the development of FurniFlex.

I have greatly benefited from her regular critical reviews and inspiration throughout my work. I am grateful for her guidance, encouragement, understanding and insightful support in the development process. I would also like to thank my college for providing the required resources whenever I needed them and for giving me the opportunity to develop this project.

I would like to express my sincere thanks to our Principal **Dr. Shashikant Malunkar** and our Head of Department **Mr. Pramod L. Shewale** for having facilitated us with the essential infrastructure and resources without which this project would not have seen the light of the day. I am also thankful to the entire staff of the Department of Computer Science for their constant encouragement, suggestions and moral support throughout the duration of my project.

Last but not the least, I would like to mention here that I am greatly indebted to my family, my friends and everybody who has been associated with my project at any stage but whose name does not find a place in this acknowledgement.

With Sincere Regards,

Ms. Rehana Mansuri



“FURNIFLEX”

FURNITURE RENTAL WEB APPLICATION

TABLE OF CONTENTS

Chapter No.	Description	Page No.
1.	**INTRODUCTION**	**7**
1.1	Introduction	
1.2	Objectives	
1.3	Scope	
1.4	Methodology	
1.5	Hardware Required	
1.6	Software Required	
2.	**FUNCTIONAL REQUIREMENT**	**11**
2.1	User Management	
2.2	Rental & Order Management	
2.3	Database Management	
3.	**DIAGRAMS**	**14**
3.1	System Flowchart	
3.2	Use Case Diagram	
3.3	Activity Diagram	
3.4	Class Diagram	
3.5	Sequence Diagram	
3.6	ER Diagram	
3.7	Gantt Chart	
4.	**PROGRAM CODE**	**21**
4.1	Backend Code (FastAPI)	
4.2	Frontend Code (React)	
5.	**TESTING**	**123**
5.1	Testing Approach	
5.2	Test Cases	
6.	**DEPLOYMENT**	**126**
6.1	Deployment Overview	
6.2	Deployment Architecture	
6.3	Deployment Use Case Diagram	
6.4	Deployment Flow	
6.5	Step-by-Step Deployment on AWS EC2	
6.6	Request Flow (HTTPS)	
6.7	SSL Certificate Flow (Certbot)	
6.8	Maintenance & Updates	
7.	**ADVANTAGES & LIMITATIONS**	**139**
7.1	Advantages	
7.2	Limitations	
7.3	Plagiarism Chart	
8.	**CONCLUSION**	**142**
8.1	Future Scope	
8.2	Conclusion	
8.3	References	

INTRODUCTION

1.1 Introduction:-

Furniture is one of the largest one-time expenses for anybody setting up a home. Students, young professionals and families who relocate frequently for work or study often find that buying furniture outright is expensive, difficult to transport and hard to resell when they move again. Renting furniture on a monthly basis has therefore become a popular alternative in Indian cities, where customers pay a small monthly rent instead of a large up-front price.

"FurniFlex" is a full-stack furniture rental web application built to address this need. It allows customers to browse furniture across multiple categories such as Bedroom, Living Room, Dining, Study, Storage, Chairs and complete BHK combos, choose a rental plan (Monthly, Quarterly or Yearly), add items to a cart, check out with a simulated Card / UPI / GPay QR payment step and track their orders. All prices are shown in Indian Rupees (₹) and longer rental plans automatically receive a discount.

The application also provides a dedicated **Admin Panel** through which the store administrator can view dashboard statistics, add, edit and delete products and categories (with image upload), manage the status of customer orders and view the list of registered users. The frontend is developed in **React** with **Material UI**, the backend is a REST API written in **Python** using the **FastAPI** framework, and data is stored in a **PostgreSQL** relational database. The backend and database run inside **Docker** containers, and the complete application is deployed on an **AWS EC2** instance behind an **Nginx** reverse proxy with a free **DuckDNS** domain and an HTTPS certificate from **Let's Encrypt** (Certbot).

By moving the complete rental process online, FurniFlex reduces paperwork for the store, makes the catalogue available to customers at any time from any device and gives the administrator a single place to manage products and orders.

1.2 Objectives:-

The primary objectives of the "FurniFlex" furniture rental platform are outlined as follows:

- **Make Furniture Affordable and Flexible:** To provide a simple online platform where customers can rent quality furniture on flexible monthly, quarterly or yearly plans instead of buying it, with the rent and discount calculated automatically.
- **Improve Accessibility and Usability:** To design a clean, responsive and attractive user interface (with light and dark themes) that works equally well on mobile phones, tablets and desktops.
- **Streamline the Rental Process:** To automate the complete rental flow – browsing, plan selection, cart, checkout and order history – so that no manual paperwork is needed.

- **Simplify Store Management:** To give administrators an easy panel to manage products, categories, images, orders and users from one place.
- **Ensure Security and Data Integrity:** To protect user accounts with hashed passwords and JWT based authentication, to restrict all admin operations to authorised administrators only, and to serve the live site only over HTTPS.
- **Make the Application Live:** To deploy the application on the cloud (AWS EC2) so that it is publicly accessible through a domain name with a valid SSL certificate.

1.3 Scope:-

The scope of "FurniFlex" is focused on delivering a complete furniture rental management platform with the following core features:

- **User Management:** The system supports two roles – **Customer** and **Administrator**. Customers can register, log in, manage their profile, rent furniture and view their orders. Administrators have full control over the catalogue, orders and users.
- **Catalogue Browsing and Search:** Products are organised into categories, each with images, a description, a refundable deposit, stock quantity and three rental prices. Customers can search globally from the navigation bar, filter by category and price and sort by price or newest.
- **Rental Plans:** Every product can be rented on a Monthly, Quarterly (10% discount) or Yearly (25% discount) plan. The selected plan is stored with each cart item and order item.
- **Cart, Checkout and Orders:** Customers can add, update and remove cart items, enter a shipping address and choose a payment method (Card, UPI or GPay QR). Orders are saved with their status and can be tracked in Order History.
- **Admin Panel:** Administrators can view dashboard statistics, perform CRUD operations on products and categories, upload images, change order status (Placed, Confirmed, Delivered, Returned, Cancelled) and view users.
- **Cloud Deployment:** The application is hosted on an AWS EC2 instance using Docker, with Nginx as a reverse proxy, a free DuckDNS sub-domain and a Let's Encrypt SSL certificate issued by Certbot.

Out of scope: real payment gateway integration (the payment step is a demo only and no card data is stored), e-mail/SMS notifications, product reviews and multi-vendor support.

1.4 Methodology:-

The development of the "FurniFlex" platform follows an **Agile** development methodology, particularly an **Incremental** approach. The system was built in layers – database first, then the REST API, then the user interface – with new features added and tested in short iterations. The process of development can be divided into the following stages:

- **Requirement Gathering and Analysis:** The features of popular furniture rental services were studied and a feature list was prepared covering registration, browsing, rental plans, cart, checkout, order history and the admin panel. A reference mock-up in a warm amber/cream theme was used to define the visual style.
- **System Design and Architecture:** A three-tier architecture was designed: a React single-page application (presentation layer), a FastAPI REST API (business logic layer) and a PostgreSQL database (data layer). The relational schema was designed with seven tables – users, categories, products, rental_plans, cart_items, orders and order_items.
- **Iterative Development and Testing:** Each feature (authentication, catalogue, cart, checkout, admin CRUD, image upload, search, dark mode) was developed as a small increment and tested immediately through the Swagger UI and the browser before moving to the next one.
- **Deployment and Maintenance:** The backend and database were containerised with Docker and Docker Compose and deployed on an AWS EC2 Ubuntu server. Nginx serves the React build and forwards API requests to the backend container, DuckDNS provides the domain name and Certbot installs a free Let's Encrypt SSL certificate.

1.5 Hardware Requirements:-

- **Development Computer:** A laptop or desktop with an Intel i3 / i5 processor (or equivalent) or higher, at least 8 GB of RAM (recommended for running Docker, the Node.js development server and a browser together) and at least 20 GB of free storage.
- **Production Server:** An AWS EC2 instance (t2.micro / t3.micro free tier or t3.small) running Ubuntu Server 24.04 LTS with 1–2 GB RAM (plus swap), 1–2 vCPU and a 20 GB EBS volume.
- **Client Devices:** Any device with a modern web browser – desktop, laptop, tablet or smartphone – can be used to access the application, as the interface is fully responsive.
- **Internet Connection:** A stable internet connection is needed for installing packages, deploying to AWS and for users to access the hosted application.
- **Input Devices:** Standard input devices such as a keyboard, mouse or touchpad / touch screen.

1.6 Software Requirements:-

Component	Software / Technology	Purpose
Operating System	Windows / macOS (development), Ubuntu 24.04 LTS (server)	Development and hosting environment
Frontend	React 19, Vite, Material UI v7, React Router v7, Axios	User interface, routing and API calls
Backend	Python 3.12, FastAPI, Uvicorn	REST API and business logic
ORM / Validation	SQLAlchemy 2.0, Pydantic v2	Database models and request validation

Component	Software / Technology	Purpose
Security	python-jose (JWT), bcrypt	Token authentication and password hashing
Database	PostgreSQL 16	Relational data storage
Containers	Docker, Docker Compose	Containerised backend and database
Cloud	AWS EC2, Security Groups	Hosting the live application
Web Server	Nginx	Reverse proxy and serving the React build
Domain & SSL	DuckDNS, Certbot (Let's Encrypt)	Free domain name and HTTPS certificate
Tools	VS Code, Git & GitHub, Node.js & npm, Swagger UI, SSH	Coding, version control, API testing, server access
Browser	Google Chrome / Firefox / Edge	Running and testing the web application

FUNCTIONAL REQUIREMENTS

The development of an online furniture rental system necessitates clear functional requirements to ensure that the platform meets the needs of all its stakeholders – customers and administrators. These functional requirements outline the expected behaviour of the system and define how it should operate in various scenarios.

2.1 User Management :-

• Admin Functions:

- **Secure Admin Access:** The admin must be able to log in through the same login page; the system identifies administrators by the *is_admin* flag and shows the Admin Panel link only to them.
- **Dashboard:** The admin dashboard must show key statistics such as total products, total categories, total users, total orders and total revenue.
- **Product Management:** The admin must be able to add, edit and delete products, including name, category, description, monthly/quarterly/yearly prices, deposit, stock quantity, featured flag and multiple images.
- **Category Management:** The admin must be able to add, edit and delete categories with a name, slug, description and cover image.
- **Image Upload:** The admin must be able to upload images (JPG, PNG, WEBP, GIF up to 5 MB), which are stored in a per-category folder on the server.
- **Order Management:** The admin must be able to view all customer orders and change their status (Placed, Confirmed, Delivered, Returned, Cancelled).
- **User Monitoring:** The admin must be able to view the list of all registered users with their contact details and role.

• Customer Functions:

- **Account Access:** Customers must be able to register with name, e-mail, phone and password, log in securely and log out.
- **Profile Management:** Customers must be able to view and update their name, phone number and address.
- **Browsing:** Customers must be able to browse categories, search products, filter by price and category, sort results and view complete product details with an image gallery.
- **Renting:** Customers must be able to add products to their cart with a selected rental plan and quantity, place orders and view their complete order history with status.

2.2 Rental & Order Management :-

• Rental Plans:

- The system must support three rental plans – Monthly (1 month), Quarterly (3 months, 10% discount) and Yearly (12 months, 25% discount).
- The price shown in the cart and saved in the order must match the plan selected by the customer.

• Cart Management:

- The cart must be stored on the server against the logged-in user, so it is available on any device.
- Adding the same product with the same plan again must increase the quantity instead of creating a duplicate line. Customers must be able to change quantity/plan or remove items.

• Checkout and Payment:

- Checkout must require a shipping address and a payment method (Card, UPI or GPay QR).
- The payment step is a simulation only – no real payment is processed and only a masked reference (e.g. "Card ending 1234" or the UPI ID) is saved against the order.
- On successful checkout the system must create an order with its order items, store the price at the time of order, set the status to PLACED and clear the cart.

• Order Tracking:

- Customers must be able to see all their past orders with items, plan, amount, payment method and current status. Customers must not be able to see other users' orders.

2.3 Database Management :-

A robust relational database management system (PostgreSQL) is used for storing, retrieving and managing all data related to users, products, carts and orders. The database must ensure data integrity, security and efficient access.

• User Data Management:

- The database must securely store user information including personal details, login credentials (passwords hashed with bcrypt, never stored in plain text) and the role flag.
- E-mail addresses must be unique so that one e-mail cannot be registered twice.

• Catalogue Data Management:

- Categories and products are stored in separate tables linked by a foreign key; deleting a category removes its products (cascade).
- Product image paths are stored as a JSON list, while the image files themselves are stored on disk and served by the backend under */static/images*.

• Order Data Management:

- Orders and order items are stored in separate tables. Each order item keeps the product, rental plan, quantity and the price at the time of ordering, so later price changes do not

affect old orders.

- **Backup and Recovery:**
  - Database data is stored in a named Docker volume (*furniflex_pgdata*), so it persists across container restarts. Regular backups can be taken with the standard *pg_dump* utility.

DIAGRAMS

1. System Flow Chart

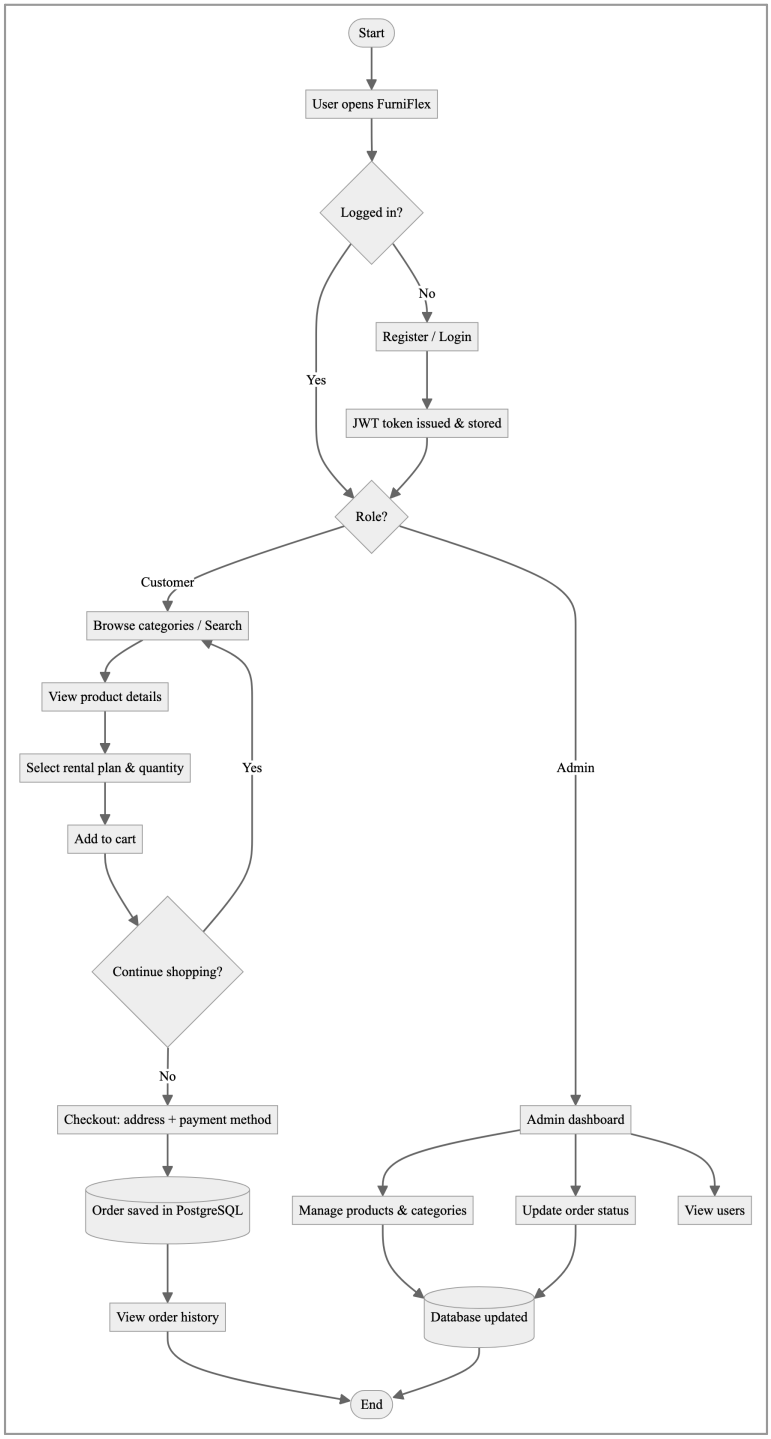


Fig 3.1: System flowchart of FurniFlex

2. Use Case Diagram

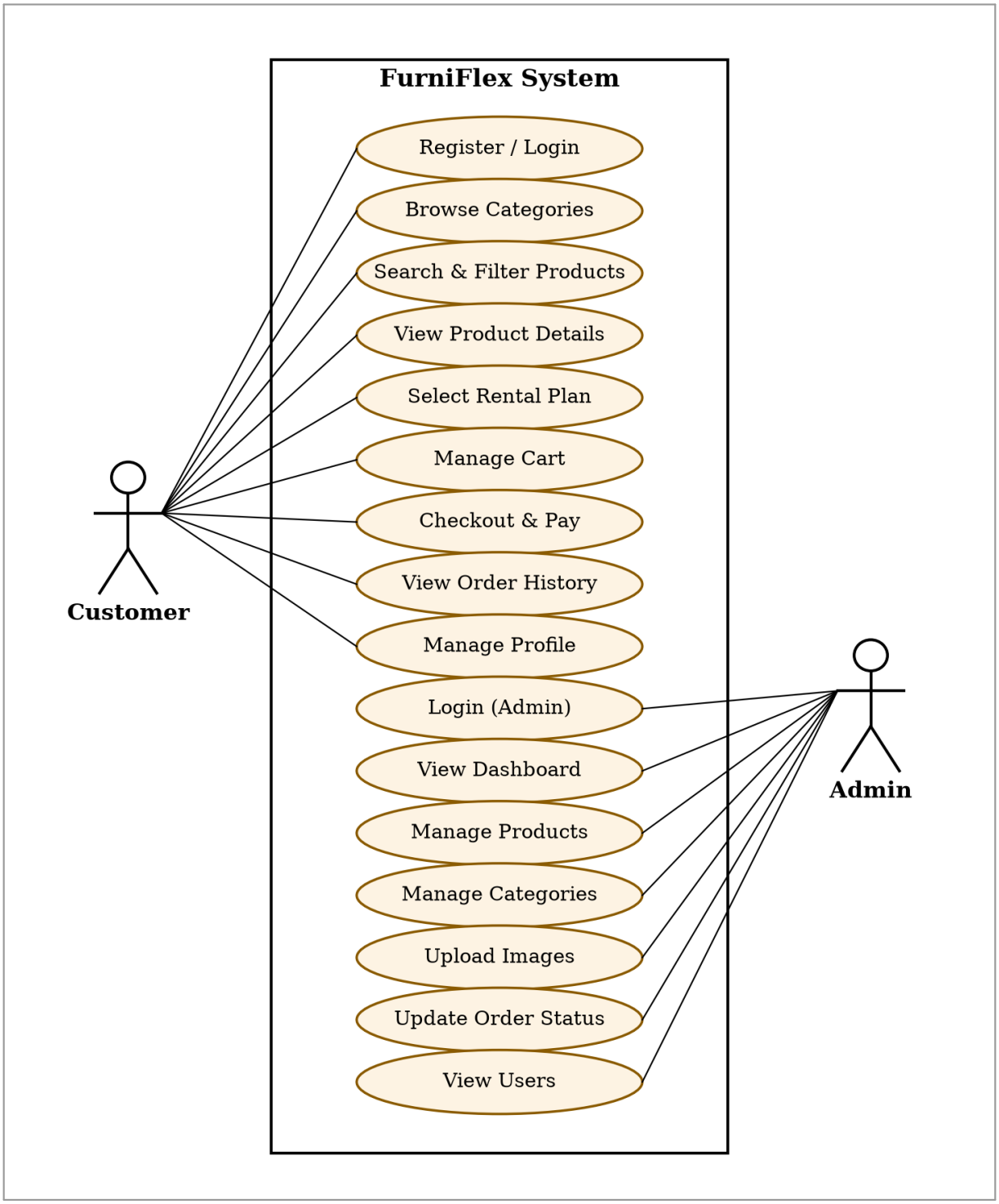


Fig 3.2: Use case diagram (Customer and Admin)

3. Activity Diagram

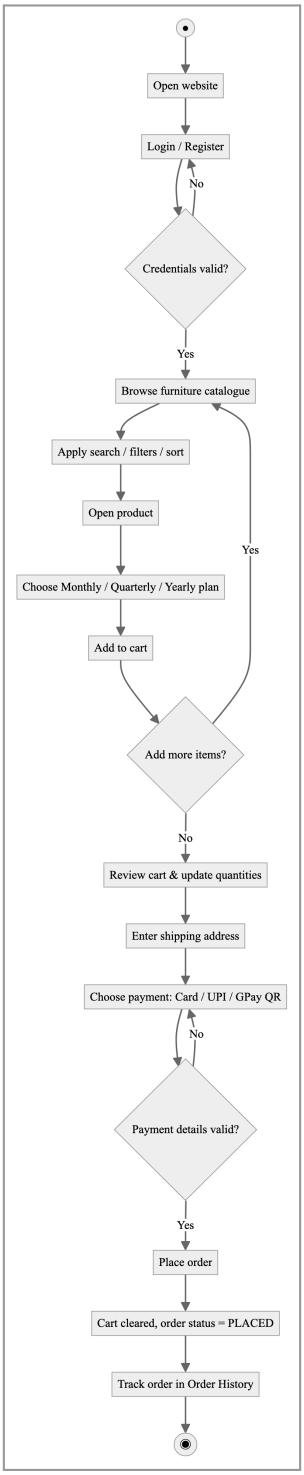


Fig 3.3: Activity diagram for renting furniture

4. Class Diagram

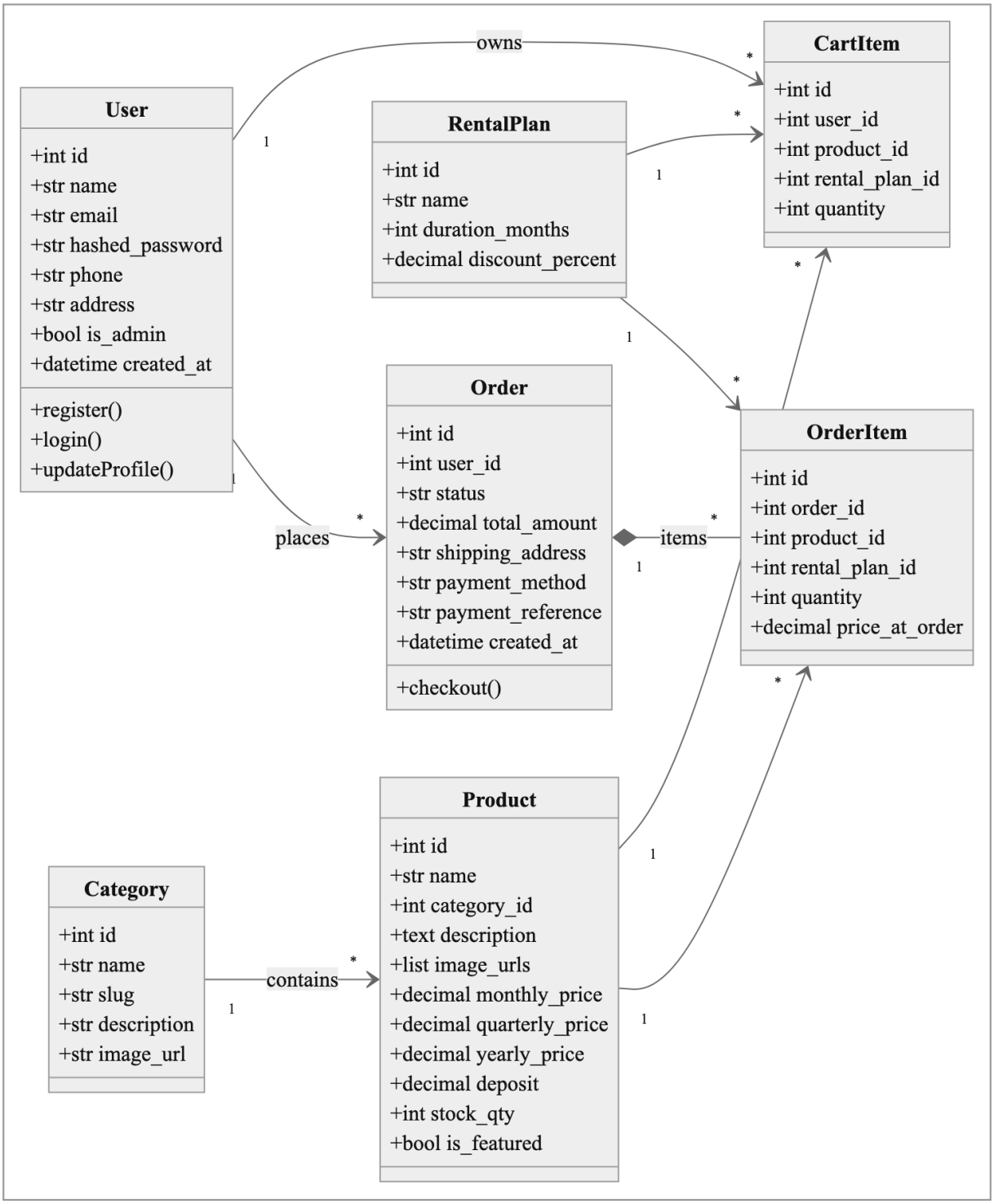


Fig 3.4: Class diagram (SQLAlchemy models)

5. Sequence Diagram

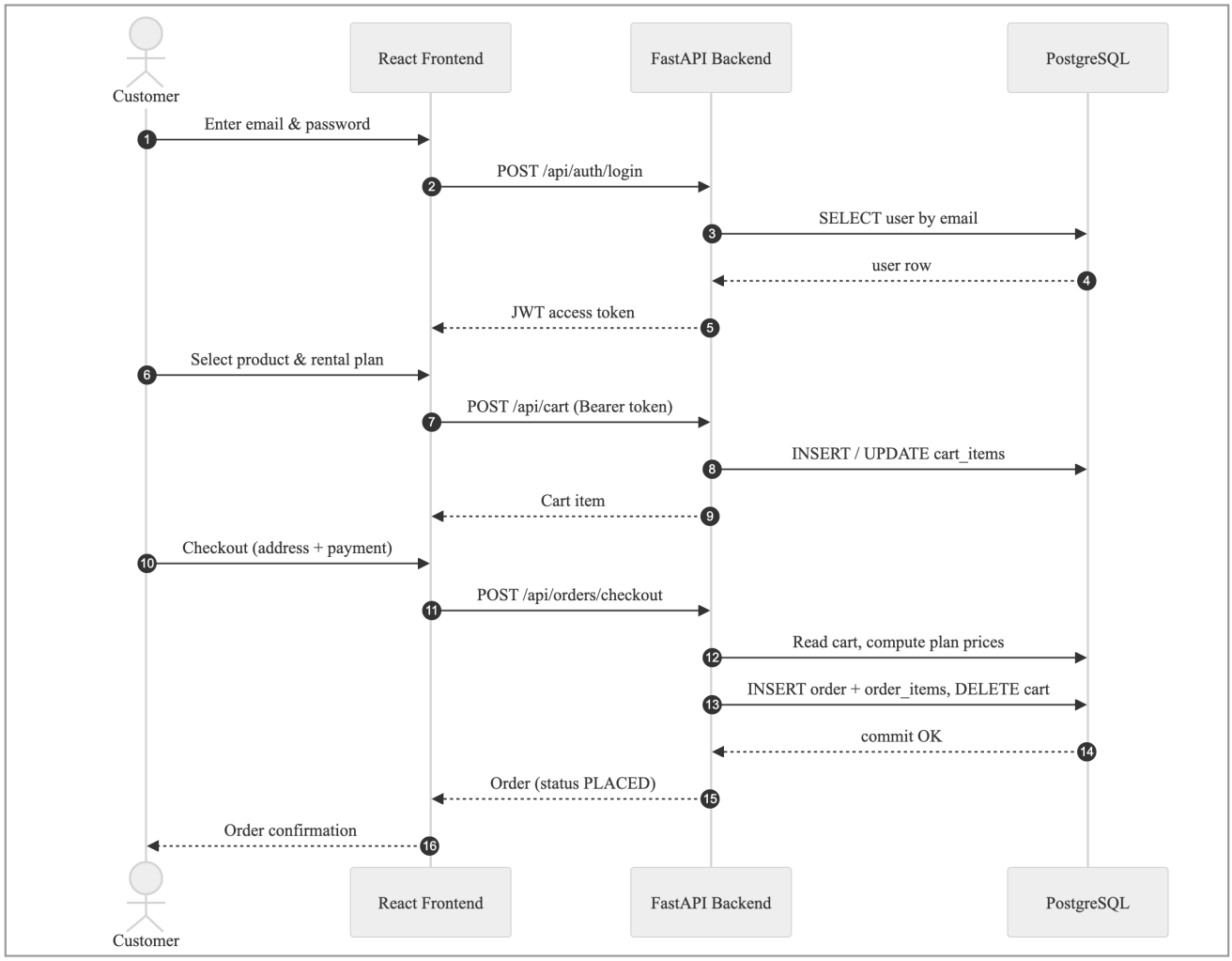


Fig 3.5: Sequence diagram for login, add to cart and checkout

6. ER Diagram

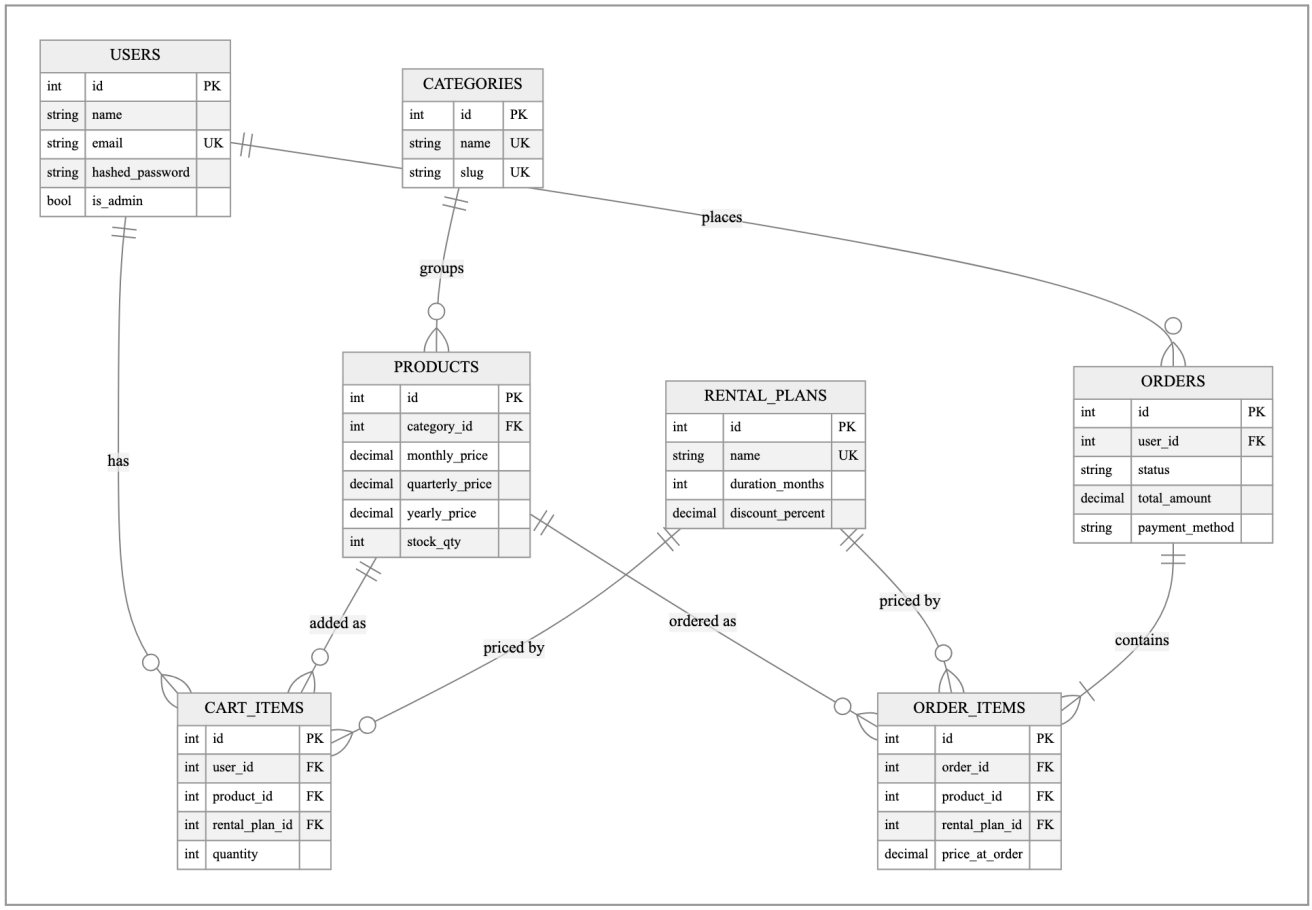


Fig 3.6: Entity relationship diagram of the PostgreSQL database

7. Gantt Chart

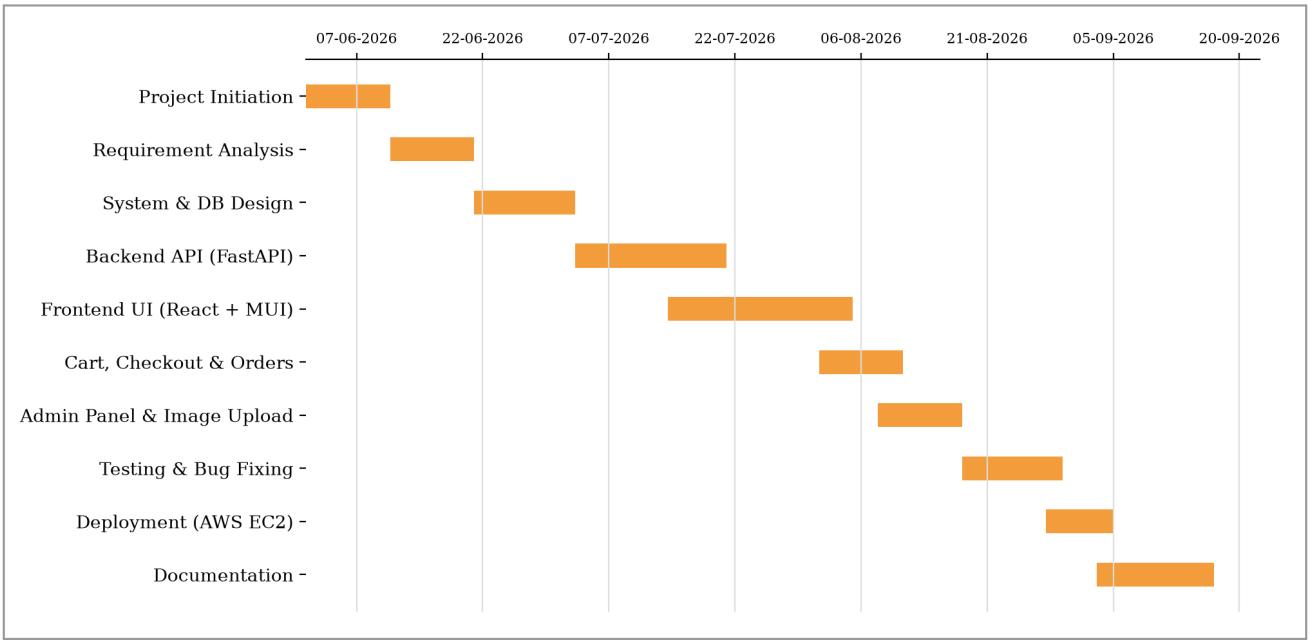


Fig 3.7: Project schedule (Gantt chart)

PROGRAM CODE

This chapter lists the complete source code of the FurniFlex application. The code is organised into a Python FastAPI backend (configuration, security, database models, schemas and API routers) and a React frontend (routing, theme, contexts, API services, reusable components and pages). Screenshots of the running application are included as **Output** after the corresponding page.

4.1 Backend Code (FastAPI):-

➢ requirements.txt (Python dependencies)

Path: backend/requirements.txt

Code:

```
fastapi==0.115.6
uvicorn[standard]==0.32.1
sqlalchemy==2.0.36
psycopg2-binary==2.9.10
pydantic==2.10.3
pydantic-settings==2.6.1
email-validator==2.2.0
python-jose[cryptography]==3.3.0
python-multipart==0.0.19
bcrypt==4.0.1
```

➢ Dockerfile (Backend Docker image)

Path: backend/Dockerfile

Code:

```
FROM python:3.12-slim

WORKDIR /app

RUN apt-get update && apt-get install -y --no-install-recommends gcc libpq-dev \
    && rm -rf /var/lib/apt/lists/*

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

➢ docker-compose.yml (PostgreSQL + Backend services)

Path: docker-compose.yml

Code:

```
services:
  db:
    image: postgres:16-alpine
    container_name: furniflex-db
    restart: unless-stopped
    environment:
      POSTGRES_USER: furniflex
      POSTGRES_PASSWORD: furniflex
      POSTGRES_DB: furniflex
    ports:
      - "5432:5432"
    volumes:
      - furniflex_pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U furniflex"]
      interval: 5s
      timeout: 5s
      retries: 10

  backend:
    build: ./backend
    container_name: furniflex-backend
    restart: unless-stopped
    environment:
      DATABASE_URL: postgresql+psycopg2://furniflex:furniflex@db:5432/furniflex
      JWT_SECRET_KEY: change-this-secret-in-production
      CORS_ORIGINS: http://localhost:5173
    ports:
      - "8000:8000"
    volumes:
      - ./backend/static:/app/static
    depends_on:
      db:
        condition: service_healthy
    command: >
      sh -c "python -m app.seed && uvicorn app.main:app --host 0.0.0.0 --port 8000"

volumes:
  furniflex_pgdata:
```

➢ config.py (Application settings)

Path: backend/app/config.py

Code:

```
from pathlib import Path

from pydantic_settings import BaseSettings, SettingsConfigDict

# backend/static — holds category-organized product/category images.
# Seed images live here at build time; admin uploads land here at runtime too.
STATIC_DIR = Path(__file__).resolve().parent.parent / "static"
IMAGES_DIR = STATIC_DIR / "images"


class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", extra="ignore")

    DATABASE_URL: str = "postgresql+psycopg2://furniflex:furniflex@localhost:5432/furniflex"
    JWT_SECRET_KEY: str = "change-this-secret-in-production"
    JWT_ALGORITHM: str = "HS256"
    ACCESS_TOKEN_EXPIRE_MINUTES: int = 1440
    CORS_ORIGINS: str = "http://localhost:5173"

    @property
    def cors_origins_list(self) -> list[str]:
        return [origin.strip() for origin in self.CORS_ORIGINS.split(",") if origin.strip()]


settings = Settings()
```

➢ database.py (Database engine and session)

Path: backend/app/database.py

Code:

```
from sqlalchemy import create_engine
from sqlalchemy.orm import DeclarativeBase, sessionmaker

from app.config import settings

engine = create_engine(settings.DATABASE_URL, pool_pre_ping=True)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)


class Base(DeclarativeBase):
    pass


def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

➢ main.py (FastAPI application entry point)

Path: backend/app/main.py

Code:

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from fastapi.staticfiles import StaticFiles

from app.config import IMAGES_DIR, STATIC_DIR, settings
from app.database import Base, engine
from app.routers import admin, auth, cart, categories, orders, products, rental_plans, users

Base.metadata.create_all(bind=engine)
IMAGES_DIR.mkdir(parents=True, exist_ok=True)

app = FastAPI(
    title="FurniFlex API",
    description="REST API for the FurniFlex furniture rental platform",
    version="1.0.0",
)

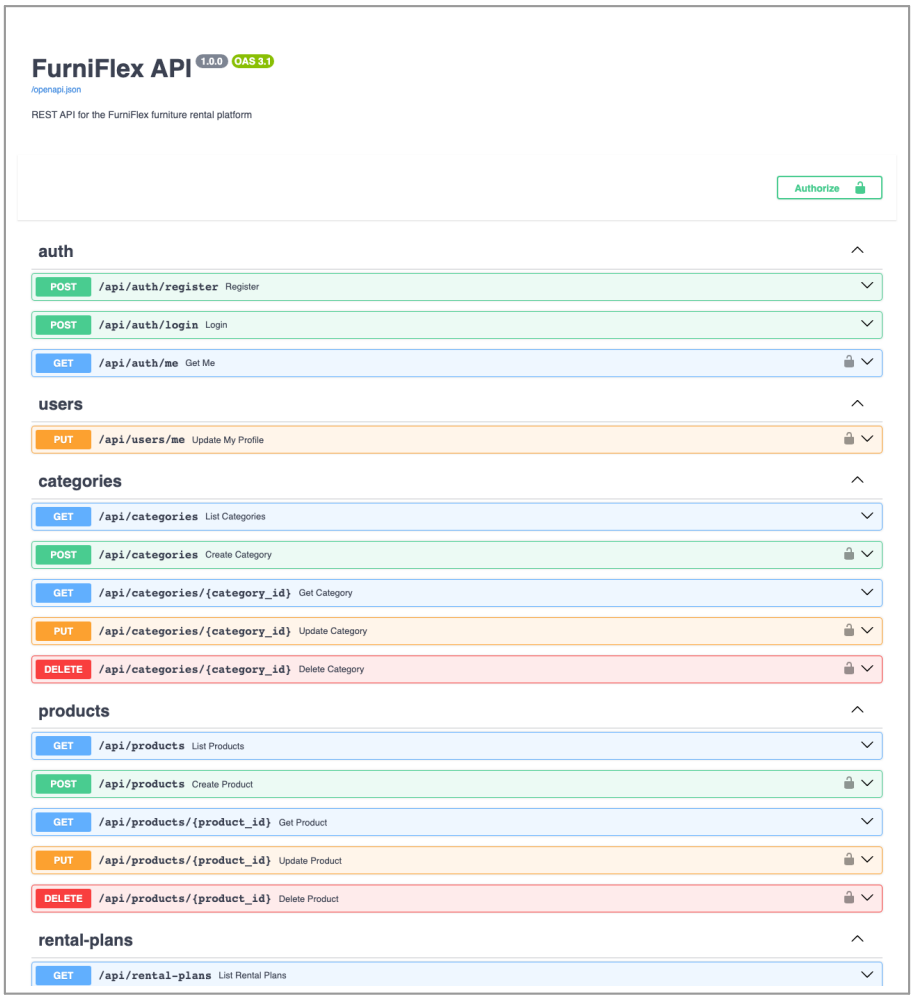
app.add_middleware(
    CORSMiddleware,
    allow_origins=settings.cors_origins_list,
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# Category/product images: seeded at build time, admin-uploaded at runtime.
app.mount("/static", StaticFiles(directory=STATIC_DIR), name="static")

app.include_router(auth.router)
app.include_router(users.router)
app.include_router(categories.router)
app.include_router(products.router)
app.include_router(rental_plans.router)
app.include_router(cart.router)
app.include_router(orders.router)
app.include_router(admin.router)


@app.get("/api/health", tags=["health"])
def health_check():
    return {"status": "ok"}
```

Output:


FurniFlex API
1.0.0
OAS 3.1
/openapi.json
REST API for the FurniFlex furniture rental platform
Authorize
auth
POST /api/auth/register Register
POST /api/auth/login Login
GET /api/auth/me Get Me
users
PUT /api/users/me Update My Profile
categories
GET /api/categories List Categories
POST /api/categories Create Category
GET /api/categories/{category_id} Get Category
PUT /api/categories/{category_id} Update Category
DELETE /api/categories/{category_id} Delete Category
products
GET /api/products List Products
POST /api/products Create Product
GET /api/products/{product_id} Get Product
PUT /api/products/{product_id} Update Product
DELETE /api/products/{product_id} Delete Product
rental-plans
GET /api/rental-plans List Rental Plans

➢ security.py (Password hashing and JWT)

Path: backend/app/core/security.py

Code:

```
from datetime import datetime, timedelta, timezone

import bcrypt
from jose import JWTError, jwt

from app.config import settings


def hash_password(password: str) -> str:
    return bcrypt.hashpw(password.encode("utf-8"), bcrypt.gensalt()).decode("utf-8")


def verify_password(plain_password: str, hashed_password: str) -> bool:
    return bcrypt.checkpw(plain_password.encode("utf-8"), hashed_password.encode("utf-8"))


def create_access_token(subject: str) -> str:
    expire = datetime.now(timezone.utc) +
        timedelta(minutes=settings.ACCESS_TOKEN_EXPIRE_MINUTES)
    payload = {"sub": subject, "exp": expire}
    return jwt.encode(payload, settings.JWT_SECRET_KEY, algorithm=settings.JWT_ALGORITHM)


def decode_access_token(token: str) -> str | None:
    try:
        payload = jwt.decode(token, settings.JWT_SECRET_KEY,
            algorithms=[settings.JWT_ALGORITHM])
        return payload.get("sub")
    except JWTError:
        return None
```

➢ deps.py (Authentication dependencies)

Path: backend/app/core/deps.py

Code:

```
from fastapi import Depends, HTTPException, status
from fastapi.security import OAuth2PasswordBearer
from sqlalchemy.orm import Session

from app.core.security import decode_access_token
from app.database import get_db
from app.models.user import User

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/api/auth/token")


def get_current_user(
    token: str = Depends(oauth2_scheme), db: Session = Depends(get_db)
) -> User:
    credentials_exception = HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Could not validate credentials",
        headers={"WWW-Authenticate": "Bearer"},
    )
```

```
    user_id = decode_access_token(token)
    if user_id is None:
        raise credentials_exception

    user = db.get(User, int(user_id))
    if user is None:
        raise credentials_exception

    return user


def get_current_admin(current_user: User = Depends(get_current_user)) -> User:
    if not current_user.is_admin:
        raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN,
            detail="Admin privileges required",
        )
    return current_user
```

➢ __init__.py (Models package)

Path: backend/app/models/__init__.py

Code:

```
from app.models.user import User
from app.models.category import Category
from app.models.rental_plan import RentalPlan
from app.models.product import Product
from app.models.cart import CartItem
from app.models.order import Order, OrderItem

__all__ = [
    "User",
    "Category",
    "RentalPlan",
    "Product",
    "CartItem",
    "Order",
    "OrderItem",
]
```

➢ user.py (User model)

Path: backend/app/models/user.py

Code:

```
from datetime import datetime, timezone

from sqlalchemy import Boolean, DateTime, String
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.database import Base


class User(Base):
    __tablename__ = "users"

    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(120), nullable=False)
    email: Mapped[str] = mapped_column(String(255), unique=True, index=True, nullable=False)
    hashed_password: Mapped[str] = mapped_column(String(255), nullable=False)
    phone: Mapped[str | None] = mapped_column(String(20), nullable=True)
    address: Mapped[str | None] = mapped_column(String(500), nullable=True)
    is_admin: Mapped[bool] = mapped_column(Boolean, default=False, nullable=False)
    created_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True), default=lambda: datetime.now(timezone.utc)
    )

    cart_items = relationship("CartItem", back_populates="user", cascade="all,
        delete-orphan")
    orders = relationship("Order", back_populates="user", cascade="all, delete-orphan")
```

➢ category.py (Category model)

Path: backend/app/models/category.py

Code:

```
from sqlalchemy import String
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.database import Base


class Category(Base):
    __tablename__ = "categories"

    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(100), unique=True, nullable=False)
    slug: Mapped[str] = mapped_column(String(100), unique=True, nullable=False)
    description: Mapped[str | None] = mapped_column(String(500), nullable=True)
    image_url: Mapped[str | None] = mapped_column(String(1000), nullable=True)

    products = relationship("Product", back_populates="category", cascade="all,
        delete-orphan")
```

➢ product.py (Product model)

Path: backend/app/models/product.py

Code:

```
from datetime import datetime, timezone

from sqlalchemy import Boolean, DateTime, ForeignKey, Integer, JSON, Numeric, String, Text
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.database import Base


class Product(Base):
    __tablename__ = "products"

    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(200), nullable=False)
    category_id: Mapped[int] = mapped_column(ForeignKey("categories.id"), nullable=False)
    description: Mapped[str] = mapped_column(Text, nullable=False)
    image_urls: Mapped[list[str]] = mapped_column(JSON, default=list)

    monthly_price: Mapped[float] = mapped_column(Numeric(10, 2), nullable=False)
    quarterly_price: Mapped[float] = mapped_column(Numeric(10, 2), nullable=False)
    yearly_price: Mapped[float] = mapped_column(Numeric(10, 2), nullable=False)
    deposit: Mapped[float] = mapped_column(Numeric(10, 2), default=0)

    stock_qty: Mapped[int] = mapped_column(Integer, default=10)
    is_featured: Mapped[bool] = mapped_column(Boolean, default=False)
    created_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True), default=lambda: datetime.now(timezone.utc)
    )

    category = relationship("Category", back_populates="products")
    cart_items = relationship("CartItem", back_populates="product", cascade="all,
        delete-orphan")
    order_items = relationship("OrderItem", back_populates="product")
```

➢ rental_plan.py (Rental plan model)

Path: backend/app/models/rental_plan.py

Code:

```
from sqlalchemy import Integer, Numeric, String
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.database import Base


class RentalPlan(Base):
    """A rental duration tier, e.g. Monthly / Quarterly / Yearly."""

    __tablename__ = "rental_plans"

    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(50), unique=True, nullable=False)
    duration_months: Mapped[int] = mapped_column(Integer, nullable=False)
    discount_percent: Mapped[float] = mapped_column(Numeric(5, 2), default=0)

    cart_items = relationship("CartItem", back_populates="rental_plan")
    order_items = relationship("OrderItem", back_populates="rental_plan")
```

➢ cart.py (Cart item model)

Path: backend/app/models/cart.py

Code:

```
from sqlalchemy import ForeignKey, Integer
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.database import Base


class CartItem(Base):
    __tablename__ = "cart_items"

    id: Mapped[int] = mapped_column(primary_key=True)
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id"), nullable=False)
    product_id: Mapped[int] = mapped_column(ForeignKey("products.id"), nullable=False)
    rental_plan_id: Mapped[int] = mapped_column(ForeignKey("rental_plans.id"),
        nullable=False)
    quantity: Mapped[int] = mapped_column(Integer, default=1)

    user = relationship("User", back_populates="cart_items")
    product = relationship("Product", back_populates="cart_items")
    rental_plan = relationship("RentalPlan", back_populates="cart_items")
```

➢ order.py (Order and order item models)

Path: backend/app/models/order.py

Code:

```
from datetime import datetime, timezone

from sqlalchemy import DateTime, ForeignKey, Integer, Numeric, String
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.database import Base


class Order(Base):
    __tablename__ = "orders"

    id: Mapped[int] = mapped_column(primary_key=True)
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id"), nullable=False)
    status: Mapped[str] = mapped_column(String(30), default="PLACED")
    total_amount: Mapped[float] = mapped_column(Numeric(10, 2), nullable=False)
    shipping_address: Mapped[str] = mapped_column(String(500), nullable=False)
    payment_method: Mapped[str] = mapped_column(String(20), default="CARD")
    payment_reference: Mapped[str | None] = mapped_column(String(100), nullable=True)
    created_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True), default=lambda: datetime.now(timezone.utc)
    )

    user = relationship("User", back_populates="orders")
    items = relationship("OrderItem", back_populates="order", cascade="all, delete-orphan")


class OrderItem(Base):
    __tablename__ = "order_items"

    id: Mapped[int] = mapped_column(primary_key=True)
    order_id: Mapped[int] = mapped_column(ForeignKey("orders.id"), nullable=False)
```

```
    product_id: Mapped[int] = mapped_column(ForeignKey("products.id"), nullable=False)
    rental_plan_id: Mapped[int] = mapped_column(ForeignKey("rental_plans.id"),
        nullable=False)
    quantity: Mapped[int] = mapped_column(Integer, default=1)
    price_at_order: Mapped[float] = mapped_column(Numeric(10, 2), nullable=False)

    order = relationship("Order", back_populates="items")
    product = relationship("Product", back_populates="order_items")
    rental_plan = relationship("RentalPlan", back_populates="order_items")
```

➢ user.py (User schemas)

Path: backend/app/schemas/user.py

Code:

```
from datetime import datetime

from pydantic import BaseModel, ConfigDict, EmailStr


class UserBase(BaseModel):
    name: str
    email: EmailStr
    phone: str | None = None
    address: str | None = None


class UserCreate(UserBase):
    password: str


class UserUpdate(BaseModel):
    name: str | None = None
    phone: str | None = None
    address: str | None = None
class UserOut(UserBase):
    model_config = ConfigDict(from_attributes=True)

    id: int
    is_admin: bool
    created_at: datetime


class UserLogin(BaseModel):
    email: EmailStr
    password: str


class Token(BaseModel):
    access_token: str
    token_type: str = "bearer"
    user: UserOut
```

➢ category.py (Category schemas)

Path: backend/app/schemas/category.py

Code:

```
from pydantic import BaseModel, ConfigDict


class CategoryBase(BaseModel):
    name: str
    slug: str
    description: str | None = None
    image_url: str | None = None


class CategoryCreate(CategoryBase):
    pass


class CategoryUpdate(BaseModel):
    name: str | None = None
    slug: str | None = None
    description: str | None = None
    image_url: str | None = None
class CategoryOut(CategoryBase):
    model_config = ConfigDict(from_attributes=True)

    id: int
```

➢ product.py (Product schemas)

Path: backend/app/schemas/product.py

Code:

```
from datetime import datetime

from pydantic import BaseModel, ConfigDict

from app.schemas.category import CategoryOut


class ProductBase(BaseModel):
    name: str
    category_id: int
    description: str
    image_urls: list[str] = []
    monthly_price: float
    quarterly_price: float
    yearly_price: float
    deposit: float = 0
    stock_qty: int = 10
    is_featured: bool = False
class ProductCreate(ProductBase):
    pass


class ProductUpdate(BaseModel):
    name: str | None = None
    category_id: int | None = None
    description: str | None = None
    image_urls: list[str] | None = None
```

```
    monthly_price: float | None = None
    quarterly_price: float | None = None
    yearly_price: float | None = None
    deposit: float | None = None
    stock_qty: int | None = None
    is_featured: bool | None = None


class ProductOut(ProductBase):
    model_config = ConfigDict(from_attributes=True)
    id: int
    created_at: datetime
    category: CategoryOut | None = None


class ProductListOut(BaseModel):
    """Lightweight product representation used for list/grid views."""

    model_config = ConfigDict(from_attributes=True)

    id: int
    name: str
    category_id: int
    image_urls: list[str] = []
    monthly_price: float
    quarterly_price: float
    yearly_price: float
    is_featured: bool
```

➢ rental_plan.py (Rental plan schemas)

Path: backend/app/schemas/rental_plan.py

Code:

```
from pydantic import BaseModel, ConfigDict


class RentalPlanOut(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    id: int
    name: str
    duration_months: int
    discount_percent: float
```

➢ cart.py (Cart schemas)

Path: backend/app/schemas/cart.py

Code:

```
from pydantic import BaseModel, ConfigDict

from app.schemas.product import ProductListOut
from app.schemas.rental_plan import RentalPlanOut


class CartItemCreate(BaseModel):
    product_id: int
    rental_plan_id: int
    quantity: int = 1


class CartItemUpdate(BaseModel):
    rental_plan_id: int | None = None
    quantity: int | None = None


class CartItemOut(BaseModel):
    model_config = ConfigDict(from_attributes=True)
    id: int
    product: ProductListOut
    rental_plan: RentalPlanOut
    quantity: int
```

➢ order.py (Order schemas)

Path: backend/app/schemas/order.py

Code:

```
from datetime import datetime

from pydantic import BaseModel, ConfigDict

from app.schemas.product import ProductListOut
from app.schemas.rental_plan import RentalPlanOut
from app.schemas.user import UserOut


class CheckoutRequest(BaseModel):
    shipping_address: str
    payment_method: str = "CARD"
    payment_reference: str | None = None


class OrderItemOut(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    id: int
    product: ProductListOut
    rental_plan: RentalPlanOut
    quantity: int
    price_at_order: float


class OrderOut(BaseModel):
    model_config = ConfigDict(from_attributes=True)
```

```
    id: int
    status: str
    total_amount: float
    shipping_address: str
    payment_method: str
    payment_reference: str | None = None
    created_at: datetime
    items: list[OrderItemOut] = []


class OrderAdminOut(OrderOut):
    user: UserOut | None = None
```

➢ auth.py (Register / Login API)

Path: backend/app/routers/auth.py

Code:

```
from fastapi import APIRouter, Depends, HTTPException, status
from fastapi.security import OAuth2PasswordRequestForm
from sqlalchemy.orm import Session

from app.core.deps import get_current_user
from app.core.security import create_access_token, hash_password, verify_password
from app.database import get_db
from app.models.user import User
from app.schemas.user import Token, UserCreate, UserLogin, UserOut

router = APIRouter(prefix="/api/auth", tags=["auth"])


@router.post("/register", response_model=Token, status_code=status.HTTP_201_CREATED)
def register(payload: UserCreate, db: Session = Depends(get_db)):
    existing = db.query(User).filter(User.email == payload.email).first()
    if existing:
        raise HTTPException(status_code=400, detail="Email already registered")

    user = User(
        name=payload.name,
        email=payload.email,
        phone=payload.phone,
        address=payload.address,
        hashed_password=hash_password(payload.password),
    )
    db.add(user)
    db.commit()
    db.refresh(user)

    token = create_access_token(subject=str(user.id))
    return Token(access_token=token, user=UserOut.model_validate(user))


@router.post("/login", response_model=Token)
def login(payload: UserLogin, db: Session = Depends(get_db)):
    user = db.query(User).filter(User.email == payload.email).first()
    if not user or not verify_password(payload.password, user.hashed_password):
        raise HTTPException(status_code=401, detail="Incorrect email or password")
    token = create_access_token(subject=str(user.id))
    return Token(access_token=token, user=UserOut.model_validate(user))
```

```
@router.post("/token", include_in_schema=False)
def login_for_swagger(form_data: OAuth2PasswordRequestForm = Depends(), db: Session =
    Depends(get_db)):
    """Enables the 'Authorize' button in Swagger UI (OAuth2 password flow)."""
    user = db.query(User).filter(User.email == form_data.username).first()
    if not user or not verify_password(form_data.password, user.hashed_password):
        raise HTTPException(status_code=401, detail="Incorrect email or password")
    token = create_access_token(subject=str(user.id))
    return {"access_token": token, "token_type": "bearer"}


@router.get("/me", response_model=UserOut)
def get_me(current_user: User = Depends(get_current_user)):
    return current_user
```

➢ users.py (User profile API)

Path: backend/app/routers/users.py

Code:

```
from fastapi import APIRouter, Depends
from sqlalchemy.orm import Session

from app.core.deps import get_current_user
from app.database import get_db
from app.models.user import User
from app.schemas.user import UserOut, UserUpdate

router = APIRouter(prefix="/api/users", tags=["users"])


@router.put("/me", response_model=UserOut)
def update_my_profile(
    payload: UserUpdate,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
):
    for field, value in payload.model_dump(exclude_unset=True).items():
        setattr(current_user, field, value)
    db.commit()
    db.refresh(current_user)
    return current_user
```

➢ categories.py (Categories API)

Path: backend/app/routers/categories.py

Code:

```
from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy.orm import Session

from app.core.deps import get_current_admin
from app.database import get_db
from app.models.category import Category
from app.schemas.category import CategoryCreate, CategoryOut, CategoryUpdate

router = APIRouter(prefix="/api/categories", tags=["categories"])


@router.get("", response_model=list[CategoryOut])
def list_categories(db: Session = Depends(get_db)):
    return db.query(Category).order_by(Category.name).all()


@router.get("/{category_id}", response_model=CategoryOut)
def get_category(category_id: int, db: Session = Depends(get_db)):
    category = db.get(Category, category_id)
    if not category:
        raise HTTPException(status_code=404, detail="Category not found")
    return category


@router.post("", response_model=CategoryOut, status_code=201,
    dependencies=[Depends(get_current_admin)])
def create_category(payload: CategoryCreate, db: Session = Depends(get_db)):
    if db.query(Category).filter(Category.slug == payload.slug).first():
        raise HTTPException(status_code=400, detail="Category slug already exists")
    category = Category(**payload.model_dump())
    db.add(category)
    db.commit()
    db.refresh(category)
    return category


@router.put("/{category_id}", response_model=CategoryOut,
    dependencies=[Depends(get_current_admin)])
def update_category(category_id: int, payload: CategoryUpdate, db: Session =
    Depends(get_db)):
    category = db.get(Category, category_id)
    if not category:
        raise HTTPException(status_code=404, detail="Category not found")
    for field, value in payload.model_dump(exclude_unset=True).items():
        setattr(category, field, value)
    db.commit()
    db.refresh(category)
    return category


@router.delete("/{category_id}", status_code=204, dependencies=[Depends(get_current_admin)])
def delete_category(category_id: int, db: Session = Depends(get_db)):
    category = db.get(Category, category_id)
    if not category:
        raise HTTPException(status_code=404, detail="Category not found")
    db.delete(category)
    db.commit()
```

➢ products.py (Products API (search, filter, sort))

Path: backend/app/routers/products.py

Code:

```
from fastapi import APIRouter, Depends, HTTPException, Query
from sqlalchemy import or_
from sqlalchemy.orm import Session, joinedload

from app.core.deps import get_current_admin
from app.database import get_db
from app.models.category import Category
from app.models.product import Product
from app.schemas.product import ProductCreate, ProductListOut, ProductOut, ProductUpdate

router = APIRouter(prefix="/api/products", tags=["products"])


@router.get("", response_model=list[ProductListOut])
def list_products(
    db: Session = Depends(get_db),
    category_id: int | None = None,
    category_slug: str | None = None,
    search: str | None = None,
    min_price: float | None = None,
    max_price: float | None = None,
    featured: bool | None = None,
    sort: str | None = Query(None, description="price_asc | price_desc | newest"),
):
    query = db.query(Product)

    if category_slug:
        category = db.query(Category).filter(Category.slug == category_slug).first()
        category_id = category.id if category else -1

    if category_id is not None:
        query = query.filter(Product.category_id == category_id)

    if search:
        like = f"%{search}%"
        query = query.filter(or_(Product.name.ilike(like), Product.description.ilike(like)))

    if min_price is not None:
        query = query.filter(Product.monthly_price >= min_price)
    if max_price is not None:
        query = query.filter(Product.monthly_price <= max_price)
    if featured is not None:
        query = query.filter(Product.is_featured == featured)

    if sort == "price_asc":
        query = query.order_by(Product.monthly_price.asc())
    elif sort == "price_desc":
        query = query.order_by(Product.monthly_price.desc())
    else:
        query = query.order_by(Product.created_at.desc())

    return query.all()


@router.get("/{product_id}", response_model=ProductOut)
def get_product(product_id: int, db: Session = Depends(get_db)):
    product = (
```

```
        db.query(Product)
        .options(joinedload(Product.category))
        .filter(Product.id == product_id)
        .first()
    )
    if not product:
        raise HTTPException(status_code=404, detail="Product not found")
    return product


@router.post("", response_model=ProductOut, status_code=201,
    dependencies=[Depends(get_current_admin)])
def create_product(payload: ProductCreate, db: Session = Depends(get_db)):
    if not db.get(Category, payload.category_id):
        raise HTTPException(status_code=400, detail="Invalid category_id")
    product = Product(**payload.model_dump())
    db.add(product)
    db.commit()
    db.refresh(product)
    return product


@router.put("/{product_id}", response_model=ProductOut,
    dependencies=[Depends(get_current_admin)])
def update_product(product_id: int, payload: ProductUpdate, db: Session = Depends(get_db)):
    product = db.get(Product, product_id)
    if not product:
        raise HTTPException(status_code=404, detail="Product not found")
    for field, value in payload.model_dump(exclude_unset=True).items():
        setattr(product, field, value)
    db.commit()
    db.refresh(product)
    return product


@router.delete("/{product_id}", status_code=204, dependencies=[Depends(get_current_admin)])
def delete_product(product_id: int, db: Session = Depends(get_db)):
    product = db.get(Product, product_id)
    if not product:
        raise HTTPException(status_code=404, detail="Product not found")
    db.delete(product)
    db.commit()
```

➢ rental_plans.py (Rental plans API)

Path: backend/app/routers/rental_plans.py

Code:

```
from fastapi import APIRouter, Depends
from sqlalchemy.orm import Session

from app.database import get_db
from app.models.rental_plan import RentalPlan
from app.schemas.rental_plan import RentalPlanOut

router = APIRouter(prefix="/api/rental-plans", tags=["rental-plans"])


@router.get("", response_model=list[RentalPlanOut])
def list_rental_plans(db: Session = Depends(get_db)):
    return db.query(RentalPlan).order_by(RentalPlan.duration_months).all()
```

➢ cart.py (Cart API)

Path: backend/app/routers/cart.py

Code:

```
from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy.orm import Session, joinedload

from app.core.deps import get_current_user
from app.database import get_db
from app.models.cart import CartItem
from app.models.product import Product
from app.models.rental_plan import RentalPlan
from app.models.user import User
from app.schemas.cart import CartItemCreate, CartItemOut, CartItemUpdate

router = APIRouter(prefix="/api/cart", tags=["cart"])


def _cart_query(db: Session):
    return db.query(CartItem).options(
        joinedload(CartItem.product), joinedload(CartItem.rental_plan)
    )
@router.get("", response_model=list[CartItemOut])
def get_cart(db: Session = Depends(get_db), current_user: User = Depends(get_current_user)):
    return _cart_query(db).filter(CartItem.user_id == current_user.id).all()


@router.post("", response_model=CartItemOut, status_code=201)
def add_to_cart(
    payload: CartItemCreate,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
):
    if not db.get(Product, payload.product_id):
        raise HTTPException(status_code=400, detail="Invalid product_id")
    if not db.get(RentalPlan, payload.rental_plan_id):
        raise HTTPException(status_code=400, detail="Invalid rental_plan_id")

    existing = (
        db.query(CartItem)
        .filter(
            CartItem.user_id == current_user.id,
            CartItem.product_id == payload.product_id,
            CartItem.rental_plan_id == payload.rental_plan_id,
        )
        .first()
    )
    if existing:
        existing.quantity += payload.quantity
        db.commit()
        db.refresh(existing)
        return existing

    item = CartItem(user_id=current_user.id, **payload.model_dump())
    db.add(item)
    db.commit()
    db.refresh(item)
    return item


@router.put("/{item_id}", response_model=CartItemOut)
```

```
def update_cart_item(
    item_id: int,
    payload: CartItemUpdate,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
):
    item = db.get(CartItem, item_id)
    if not item or item.user_id != current_user.id:
        raise HTTPException(status_code=404, detail="Cart item not found")
    for field, value in payload.model_dump(exclude_unset=True).items():
        setattr(item, field, value)
    db.commit()
    db.refresh(item)
    return item


@router.delete("/{item_id}", status_code=204)
def remove_cart_item(
    item_id: int,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
):
    item = db.get(CartItem, item_id)
    if not item or item.user_id != current_user.id:
        raise HTTPException(status_code=404, detail="Cart item not found")
    db.delete(item)
    db.commit()


@router.delete("", status_code=204)
def clear_cart(db: Session = Depends(get_db), current_user: User =
    Depends(get_current_user)):
    db.query(CartItem).filter(CartItem.user_id == current_user.id).delete()
    db.commit()
```

➢ orders.py (Checkout and Orders API)

Path: backend/app/routers/orders.py

Code:

```
from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy.orm import Session, joinedload

from app.core.deps import get_current_user
from app.database import get_db
from app.models.cart import CartItem
from app.models.order import Order, OrderItem
from app.models.user import User
from app.schemas.order import CheckoutRequest, OrderOut

router = APIRouter(prefix="/api/orders", tags=["orders"])

PLAN_PRICE_FIELD = {
    "Monthly": "monthly_price",
    "Quarterly": "quarterly_price",
    "Yearly": "yearly_price",
}

PAYMENT_METHODS = {"CARD", "UPI", "GPAY_QR"}
def _order_query(db: Session):
    return db.query(Order).options(
        joinedload(Order.items).joinedload(OrderItem.product),
```

```
        joinedload(Order.items).joinedload(OrderItem.rental_plan),
    )


@router.post("/checkout", response_model=OrderOut, status_code=201)
def checkout(
    payload: CheckoutRequest,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
):
    cart_items = (
        db.query(CartItem)
        .options(joinedload(CartItem.product), joinedload(CartItem.rental_plan))
        .filter(CartItem.user_id == current_user.id)
        .all()
    )
    if not cart_items:
        raise HTTPException(status_code=400, detail="Cart is empty")

    if payload.payment_method not in PAYMENT_METHODS:
        raise HTTPException(status_code=400, detail=f"payment_method must be one of
            {sorted(PAYMENT_METHODS)}")

    total = 0.0
    order_items = []
    for cart_item in cart_items:
        price_field = PLAN_PRICE_FIELD.get(cart_item.rental_plan.name, "monthly_price")
        unit_price = float(getattr(cart_item.product, price_field))
        total += unit_price * cart_item.quantity
        order_items.append(
            OrderItem(
                product_id=cart_item.product_id,
                rental_plan_id=cart_item.rental_plan_id,
                quantity=cart_item.quantity,
                price_at_order=unit_price,
            )
        )

    order = Order(
        user_id=current_user.id,
        total_amount=total,
        shipping_address=payload.shipping_address,
        payment_method=payload.payment_method,
        payment_reference=payload.payment_reference,
        items=order_items,
    )
    db.add(order)
    db.query(CartItem).filter(CartItem.user_id == current_user.id).delete()
    db.commit()
    db.refresh(order)
    return _order_query(db).filter(Order.id == order.id).first()


@router.get("", response_model=list[OrderOut])
def list_my_orders(db: Session = Depends(get_db), current_user: User =
    Depends(get_current_user)):
    return (
        _order_query(db)
        .filter(Order.user_id == current_user.id)
        .order_by(Order.created_at.desc())
        .all()
    )
```

```
@router.get("/{order_id}", response_model=OrderOut)
def get_order(
    order_id: int, db: Session = Depends(get_db), current_user: User =
        Depends(get_current_user)
):
    order = _order_query(db).filter(Order.id == order_id).first()
    if not order or (order.user_id != current_user.id and not current_user.is_admin):
        raise HTTPException(status_code=404, detail="Order not found")
    return order
```

➢ admin.py (Admin API (dashboard, upload, orders, users))

Path: backend/app/routers/admin.py

Code:

```
import re
import uuid

from fastapi import APIRouter, Depends, File, Form, HTTPException, UploadFile
from pydantic import BaseModel
from sqlalchemy.orm import Session, joinedload

from app.config import IMAGES_DIR
from app.core.deps import get_current_admin
from app.database import get_db
from app.models.category import Category
from app.models.order import Order, OrderItem
from app.models.product import Product
from app.models.user import User
from app.schemas.order import OrderAdminOut
from app.schemas.user import UserOut

ORDER_STATUSES = {"PLACED", "CONFIRMED", "DELIVERED", "RETURNED", "CANCELLED"}

ALLOWED_IMAGE_EXTENSIONS = {".jpg", ".jpeg", ".png", ".webp", ".gif"}
MAX_UPLOAD_BYTES = 5 * 1024 * 1024  # 5MB
SLUG_PATTERN = re.compile(r"^[a-z0-9-]+$")


class OrderStatusUpdate(BaseModel):
    status: str

router = APIRouter(prefix="/api/admin", tags=["admin"],
    dependencies=[Depends(get_current_admin)])


@router.post("/upload-image")
async def upload_image(category_slug: str = Form(...), file: UploadFile = File(...)):
    """Saves an admin-uploaded image into static/images/<category_slug>/ and
    returns its served URL. Used for both product and category images — the
    frontend groups everything by category folder for easy browsing on disk."""
    if not SLUG_PATTERN.match(category_slug):
        raise HTTPException(status_code=400, detail="category_slug must be lowercase
            letters, numbers and hyphens")
    extension = "." + (file.filename or "").rsplit(".", 1)[-1].lower() if "." in
        (file.filename or "") else ""
    if extension not in ALLOWED_IMAGE_EXTENSIONS:
        raise HTTPException(
            status_code=400,
            detail=f"Unsupported file type. Allowed: {sorted(ALLOWED_IMAGE_EXTENSIONS)}",
```

```
        )

    contents = await file.read()
    if len(contents) > MAX_UPLOAD_BYTES:
        raise HTTPException(status_code=400, detail="File too large (max 5MB)")

    folder = IMAGES_DIR / category_slug
    folder.mkdir(parents=True, exist_ok=True)

    filename = f"{uuid.uuid4().hex}{extension}"
    (folder / filename).write_bytes(contents)

    return {"url": f"/static/images/{category_slug}/{filename}"}
@router.get("/dashboard")
def dashboard_stats(db: Session = Depends(get_db)):
    return {
        "total_products": db.query(Product).count(),
        "total_categories": db.query(Category).count(),
        "total_users": db.query(User).count(),
        "total_orders": db.query(Order).count(),
        "revenue": float(sum(o.total_amount for o in db.query(Order).all())),
    }


@router.get("/users", response_model=list[UserOut])
def list_users(db: Session = Depends(get_db)):
    return db.query(User).order_by(User.created_at.desc()).all()


@router.get("/orders", response_model=list[OrderAdminOut])
def list_all_orders(db: Session = Depends(get_db)):
    return (
        db.query(Order)
        .options(
            joinedload(Order.user),
            joinedload(Order.items).joinedload(OrderItem.product),
            joinedload(Order.items).joinedload(OrderItem.rental_plan),
        )
        .order_by(Order.created_at.desc())
        .all()
    )


@router.patch("/orders/{order_id}/status", response_model=OrderAdminOut)
def update_order_status(order_id: int, payload: OrderStatusUpdate, db: Session =
    Depends(get_db)):
    if payload.status not in ORDER_STATUSES:
        raise HTTPException(status_code=400, detail=f"Status must be one of
            {sorted(ORDER_STATUSES)}")

    order = (
        db.query(Order)
        .options(
            joinedload(Order.user),
            joinedload(Order.items).joinedload(OrderItem.product),
            joinedload(Order.items).joinedload(OrderItem.rental_plan),
        )
        .filter(Order.id == order_id)
        .first()
    )
    if not order:
        raise HTTPException(status_code=404, detail="Order not found")
```

```
    order.status = payload.status
    db.commit()
    db.refresh(order)
    return order
```

➢ seed.py (Database seed script)

Path: backend/app/seed.py

Code:

```
"""
Seeds the database with rental plans, categories, ~63 demo products,
one admin account and one demo customer account.

Run with: python -m app.seed
Safe to re-run — it skips seeding if categories already exist.
"""

from app.core.security import hash_password
from app.database import Base, SessionLocal, engine
from app.models.category import Category
from app.models.product import Product
from app.models.rental_plan import RentalPlan
from app.models.user import User

# Demo images live under backend/static/images/<category-slug>/img-N.jpg and are
# served at /static/images/<category-slug>/img-N.jpg (see main.py's static mount).
# Admin-uploaded images land in the same per-category folders (see routers/admin.py).
CATEGORY_IMAGE_FILES: dict[str, list[str]] = {
    "Bedroom": [f"img-{i}.jpg" for i in range(1, 7)],
    "BHK Combos": [f"img-{i}.jpg" for i in range(1, 7)],
    "Living Room": [f"img-{i}.jpg" for i in range(1, 7)],
    "Steal Deals": [f"img-{i}.jpg" for i in range(1, 7)],
    "Storage": [f"img-{i}.jpg" for i in range(1, 7)],
    "Kids": [f"img-{i}.jpg" for i in range(1, 7)],
    "Study": [f"img-{i}.jpg" for i in range(1, 7)],
    "Dining": [f"img-{i}.jpg" for i in range(1, 7)],
    "Luxury": [f"img-{i}.jpg" for i in range(1, 6)],
}


def _local_image(slug: str, filename: str) -> str:
    return f"/static/images/{slug}/{filename}"


def _images_for_category(category_name: str, slug: str, index: int, count: int = 2) ->
    list[str]:
    files = CATEGORY_IMAGE_FILES[category_name]
    return [_local_image(slug, files[(index + i) % len(files)]) for i in range(count)]
# (name, monthly_price_inr, description)
CATEGORY_PRODUCTS: dict[str, list[tuple[str, int, str]]] = {
    "Bedroom": [
        ("Queen Bed with Storage", 1499, "Solid engineered-wood queen bed with hydraulic
            storage, includes a plush headboard for extra comfort."),
        ("King Size Bed", 1899, "Spacious king-size bed frame with a sturdy build, perfect
            for large bedrooms."),
        ("Bedside Table", 399, "Compact bedside table with a drawer and open shelf,
            finished in warm walnut."),
        ("Wooden Wardrobe 3-Door", 1299, "3-door wardrobe with hanging rails, shelves and a
            full-length mirror."),
        ("Dressing Table with Mirror", 799, "Elegant dressing table with a large mirror and
```

```
        multiple storage drawers."),
    ("Bunk Bed", 1599, "Space-saving bunk bed with sturdy metal frame and safety rails,
        ideal for shared rooms."),
    ("Bedroom Bench", 549, "Cushioned bench for the foot of the bed, adds a touch of
        luxury to any bedroom."),
    ("Chest of Drawers", 899, "5-drawer chest offering ample storage for clothes and
        accessories."),
],
"BHK Combos": [
    ("1BHK Starter Combo", 4999, "Complete furniture set for a 1BHK — bed, wardrobe,
        sofa, dining table and TV unit."),
    ("2BHK Complete Combo", 8999, "Fully furnish a 2BHK home with beds, wardrobes,
        living room and dining furniture."),
    ("3BHK Premium Combo", 13999, "Premium furniture combo covering all bedrooms,
        living and dining areas of a 3BHK."),
    ("Bachelor Studio Combo", 3499, "Compact combo designed for studio apartments —
        bed, desk, wardrobe and mini sofa."),
    ("Newlywed Combo Pack", 6499, "Curated combo with a queen bed, wardrobe, dining set
        and living room essentials."),
    ("Work-From-Home Combo", 2999, "Ergonomic study desk, chair, bookshelf and storage
        unit bundled for home offices."),
    ("Compact 1BHK Combo", 4499, "Space-efficient combo tailored for compact 1BHK
        apartments."),
    ("Family 2BHK Combo", 9499, "Spacious combo with kids furniture, master bedroom and
        living room sets."),
],
"Living Room": [
    ("3-Seater Sofa", 1699, "Comfortable fabric 3-seater sofa with cushioned armrests
        and durable upholstery."),
    ("L-Shaped Sectional Sofa", 2499, "Large L-shaped sectional sofa, perfect for
        spacious living rooms and family time."),
    ("Recliner Chair", 1299, "Single-seat manual recliner with a footrest for ultimate
        relaxation."),
    ("Coffee Table", 599, "Modern wooden coffee table with a lower storage shelf."),
    ("TV Entertainment Unit", 899, "Wall-style TV unit with storage cabinets for
        set-top boxes and accessories."),
    ("Bookshelf", 499, "5-tier open bookshelf, great for books, decor and plants."),
    ("Accent Armchair", 999, "Stylish accent armchair to complement any living room
        decor."),
    ("Center Rug & Table Set", 749, "Coffee table bundled with a soft area rug for a
        cozy living space."),
],
"Steal Deals": [
    ("Clearance Sofa Set", 999, "Gently refurbished 3-seater sofa set at a steal price,
        limited stock."),
    ("Refurbished Dining Table", 799, "Quality-checked 4-seater dining table,
        refurbished and ready to use."),
    ("Budget Bed Frame", 599, "No-frills sturdy bed frame at an unbeatable monthly
        rate."),
    ("Discounted Wardrobe", 699, "2-door wardrobe available at a special clearance
        price."),
    ("Last-Piece Recliner", 899, "Single last-piece recliner chair, heavily
        discounted."),
    ("Value Study Desk", 349, "Compact study desk offered at our best value pricing."),
    ("Sale Bookshelf", 299, "3-tier bookshelf at a steal-deal monthly price."),
],
"Storage": [
    ("Modular Wardrobe", 999, "Customizable modular wardrobe with adjustable shelves
        and drawers."),
    ("Shoe Rack", 349, "Multi-layer shoe rack with a bench top, fits up to 18 pairs."),
    ("Storage Cabinet", 599, "Multi-purpose storage cabinet with lockable doors."),
    ("Utility Cabinet", 449, "Compact utility cabinet for storing household
```

```
            essentials."),
        ("Bookshelf Storage Unit", 499, "Combination bookshelf and storage unit with mixed
            open/closed sections."),
        ("Under-bed Storage Box", 249, "Rolling under-bed storage box, maximizes hidden
            storage space."),
        ("Multi-purpose Rack", 379, "Sturdy metal rack for kitchen, garage or balcony
            storage."),
    ],
    "Kids": [
        ("Kids Study Table", 449, "Colorful adjustable-height study table designed for
            children."),
        ("Bunk Bed for Kids", 1399, "Fun and safe bunk bed with guard rails and a sturdy
            ladder."),
        ("Toy Storage Unit", 349, "Bright toy storage unit with bins to keep the playroom
            tidy."),
        ("Kids Wardrobe", 799, "Compact wardrobe sized perfectly for a child's room."),
        ("Kids Bean Bag", 299, "Soft and durable bean bag in fun colors for kids' rooms."),
        ("Kids Bed with Rails", 999, "Single bed with safety rails, designed for young
            children."),
        ("Kids Bookshelf", 349, "Low-height bookshelf so kids can reach their favorite
            books."),
    ],
    "Study": [
        ("Study Table with Drawers", 549, "Spacious study table with 3 drawers for
            stationery and files."),
        ("Ergonomic Office Chair", 699, "Adjustable-height ergonomic chair with lumbar
            support for long work hours."),
        ("Bookshelf", 449, "Tall 6-shelf bookshelf, ideal for books and study material."),
        ("Computer Table", 599, "Study table with a keyboard tray and CPU stand, built for
            desktop setups."),
        ("Study Lamp Desk Set", 299, "Desk bundled with an adjustable LED study lamp."),
        ("L-Shaped Study Desk", 899, "Large L-shaped desk offering ample workspace for
            study or WFH."),
    ],
    "Dining": [
        ("4-Seater Dining Set", 1199, "Elegant 4-seater dining table set with cushioned
            chairs."),
        ("6-Seater Dining Set", 1799, "Spacious 6-seater dining set, perfect for family
            gatherings."),
        ("Dining Bench", 599, "Wooden dining bench that pairs well with any dining table."),
        ("Bar Stool Set", 699, "Set of 2 adjustable-height bar stools for kitchen
            counters."),
        ("Crockery Unit", 899, "Glass-front crockery unit for elegant dinnerware display."),
        ("Round Dining Table", 999, "Compact round dining table, great for small
            families."),
    ],
    "Luxury": [
        ("Premium Leather Sofa", 3499, "Genuine leather 3-seater sofa with premium
            stitching and comfort."),
        ("Designer King Bed", 3999, "Designer upholstered king bed with a tufted
            headboard."),
        ("Luxury Recliner Set", 4499, "Set of 2 premium power recliners with USB charging
            ports."),
        ("Chandelier Dining Set", 5299, "8-seater dining set paired with an elegant
            chandelier."),
        ("Premium Wardrobe", 2999, "4-door premium wardrobe with soft-close hinges and
            mirror panels."),
        ("Luxury Lounge Chair", 2499, "Designer lounge chair, a statement piece for any
            living room."),
    ],
}
```

```
CATEGORY_META = {
    "Bedroom": "Beds, wardrobes and everything for a restful bedroom.",
    "BHK Combos": "Complete furniture combos for 1BHK, 2BHK and 3BHK homes.",
    "Living Room": "Sofas, recliners and decor to furnish your living room.",
    "Steal Deals": "Refurbished and clearance furniture at unbeatable prices.",
    "Storage": "Wardrobes, racks and cabinets to organize your space.",
    "Kids": "Safe, fun and functional furniture for children's rooms.",
    "Study": "Desks, chairs and shelves for productive study or work-from-home.",
    "Dining": "Dining tables, chairs and sets for every family size.",
    "Luxury": "Premium, designer furniture for a touch of luxury.",
}


def slugify(name: str) -> str:
    return name.lower().replace(" ", "-")


def seed():
    Base.metadata.create_all(bind=engine)
    db = SessionLocal()
    try:
        if db.query(Category).first():
            print("Database already seeded — skipping.")
            return

        # Rental plans
        plans = [
            RentalPlan(name="Monthly", duration_months=1, discount_percent=0),
            RentalPlan(name="Quarterly", duration_months=3, discount_percent=10),
            RentalPlan(name="Yearly", duration_months=12, discount_percent=25),
        ]
        db.add_all(plans)
        db.flush()

        # Categories
        categories: dict[str, Category] = {}
        for name, description in CATEGORY_META.items():
            slug = slugify(name)
            cover_file = CATEGORY_IMAGE_FILES[name][0]
            category = Category(
                name=name,
                slug=slug,
                description=description,
                image_url=_local_image(slug, cover_file),
            )
            db.add(category)
            categories[name] = category
        db.flush()

        # Products
        product_count = 0
        for category_name, items in CATEGORY_PRODUCTS.items():
            category = categories[category_name]
            for position, (name, monthly, description) in enumerate(items):
                quarterly = round(monthly * 3 * 0.90 / 10) * 10
                yearly = round(monthly * 12 * 0.75 / 10) * 10
                deposit = round(monthly * 2 / 10) * 10

                product = Product(
                    name=name,
                    category_id=category.id,
                    description=description,
```

```
                    image_urls=_images_for_category(category_name, category.slug, position
                        * 2),
                    monthly_price=monthly,
                    quarterly_price=quarterly,
                    yearly_price=yearly,
                    deposit=deposit,
                    stock_qty=15,
                    is_featured=(position == 0),
                )
                db.add(product)
                product_count += 1

        # Users
        admin = User(
            name="Admin",
            email="admin@furniflex.com",
            hashed_password=hash_password("Admin@123"),
            is_admin=True,
        )
        demo_user = User(
            name="Demo Customer",
            email="demo@furniflex.com",
            hashed_password=hash_password("Demo@123"),
            phone="9876543210",
            address="221B Baker Street, Bengaluru, Karnataka",
            is_admin=False,
        )
        db.add_all([admin, demo_user])

        db.commit()
        print(f"Seeded {len(categories)} categories, {product_count} products, 3 rental
            plans, 2 users.")
    finally:
        db.close()


if __name__ == "__main__":
    seed()
```

4.2 Frontend Code (React):-

➢ package.json (Frontend dependencies)

Path: frontend/package.json

Code:

```
{
  "name": "frontend",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "lint": "oxlint",
    "preview": "vite preview"
  },
  "dependencies": {
    "@emotion/react": "^11.14.0",
    "@emotion/styled": "^11.14.1",
    "@fontsource/inter": "^5.3.0",
    "@fontsource/poppins": "^5.3.0",
    "@mui/icons-material": "^7.3.11",
    "@mui/material": "^7.3.11",
    "axios": "^1.19.0",
    "react": "^19.2.8",
    "react-dom": "^19.2.8",
    "react-router-dom": "^7.18.2"
  },
  "devDependencies": {
    "@types/react": "^19.2.17",
    "@types/react-dom": "^19.2.3",
    "@vitejs/plugin-react": "^6.0.4",
    "oxlint": "^1.75.0",
    "vite": "^8.2.0"
  }
}
```

➢ vite.config.js (Vite configuration)

Path: frontend/vite.config.js

Code:

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

// https://vite.dev/config/
export default defineConfig({
  plugins: [react()],
})
```

➢ index.html (HTML entry page)

Path: frontend/index.html

Code:

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" href="data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg'
        viewBox='0 0 100 100'%3E%3Ccircle cx='50' cy='50' r='50'
            fill='%23E8963F'/%3E%3Ctext x='50' y='68' font-size='60' text-anchor='middle'
                fill='white' font-family='sans-serif'%3EF%3C/text%3E%3C/svg%3E" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <meta name="description" content="FurniFlex - Rent quality furniture and appliances for
        your home." />
    <title>FurniFlex | Furniture Rental</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

➢ main.jsx (React entry point)

Path: frontend/src/main.jsx

Code:

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import { BrowserRouter } from "react-router-dom";

import "@fontsource/poppins/400.css";
import "@fontsource/poppins/600.css";
import "@fontsource/poppins/700.css";
import "@fontsource/inter/400.css";
import "@fontsource/inter/500.css";
import "@fontsource/inter/600.css";

import { ThemeModeProvider } from "./context/ThemeModeContext.jsx";
import { AuthProvider } from "./context/AuthContext.jsx";
import { CartProvider } from "./context/CartContext.jsx";
import App from "./App.jsx";

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <BrowserRouter>
      <ThemeModeProvider>
        <AuthProvider>
          <CartProvider>
            <App />
          </CartProvider>
        </AuthProvider>
      </ThemeModeProvider>
    </BrowserRouter>
  </StrictMode>
);
```

➢ App.jsx (Root component)

Path: frontend/src/App.jsx

Code:

```
import { Box } from "@mui/material";
import Navbar from "./components/layout/Navbar";
import Footer from "./components/layout/Footer";
import AppRoutes from "./routes/AppRoutes";

function App() {
  return (
    <Box sx={{ display: "flex", flexDirection: "column", minHeight: "100vh" }}>
      <Navbar />
      <Box component="main" sx={{ flexGrow: 1 }}>
        <AppRoutes />
      </Box>
      <Footer />
    </Box>
  );
}

export default App;
```

➢ AppRoutes.jsx (Route table)

Path: frontend/src/routes/AppRoutes.jsx

Code:

```
import { Routes, Route } from "react-router-dom";

import Home from "../pages/Home";
import Categories from "../pages/Categories";
import CategoryPage from "../pages/CategoryPage";
import SearchResults from "../pages/SearchResults";
import ProductDetails from "../pages/ProductDetails";
import Cart from "../pages/Cart";
import Checkout from "../pages/Checkout";
import OrderHistory from "../pages/OrderHistory";
import Profile from "../pages/Profile";
import Login from "../pages/Login";
import Register from "../pages/Register";
import NotFound from "../pages/NotFound";

import AdminDashboard from "../pages/admin/AdminDashboard";
import AdminProducts from "../pages/admin/AdminProducts";
import AdminCategories from "../pages/admin/AdminCategories";
import AdminOrders from "../pages/admin/AdminOrders";
import AdminUsers from "../pages/admin/AdminUsers";
import ProtectedRoute from "../components/layout/ProtectedRoute";
import AdminRoute from "../components/layout/AdminRoute";

export default function AppRoutes() {
  return (
    <Routes>
      <Route path="/" element={<Home />} />
      <Route path="/categories" element={<Categories />} />
      <Route path="/categories/:slug" element={<CategoryPage />} />
      <Route path="/search" element={<SearchResults />} />
      <Route path="/products/:id" element={<ProductDetails />} />
      <Route path="/login" element={<Login />} />
```

```
<Route path="/register" element={<Register />} />

<Route
  path="/cart"
  element={
    <ProtectedRoute>
      <Cart />
    </ProtectedRoute>
  }
/>
<Route
  path="/checkout"
  element={
    <ProtectedRoute>
      <Checkout />
    </ProtectedRoute>
  }
/>
<Route
  path="/orders"
  element={
    <ProtectedRoute>
      <OrderHistory />
    </ProtectedRoute>
  }
/>
<Route
  path="/profile"
  element={
    <ProtectedRoute>
      <Profile />
    </ProtectedRoute>
  }
/>

<Route
  path="/admin"
  element={
    <AdminRoute>
      <AdminDashboard />
    </AdminRoute>
  }
/>
<Route
  path="/admin/products"
  element={
    <AdminRoute>
      <AdminProducts />
    </AdminRoute>
  }
/>
<Route
  path="/admin/categories"
  element={
    <AdminRoute>
      <AdminCategories />
    </AdminRoute>
  }
/>
<Route
  path="/admin/orders"
  element={
```

```
          <AdminRoute>
            <AdminOrders />
          </AdminRoute>
        }
      />
      <Route
        path="/admin/users"
        element={
          <AdminRoute>
            <AdminUsers />
          </AdminRoute>
        }
      />

      <Route path="*" element={<NotFound />} />
    </Routes>
  );
}
```

➢ theme.js (Material UI theme (light and dark))

Path: frontend/src/theme/theme.js

Code:

```
import { createTheme } from "@mui/material/styles";

// Palette derived from the FurniFlex mobile reference: warm amber/mustard
// accents on a soft cream base (light) or warm charcoal base (dark),
// pill-shaped buttons, rounded cards.
const amber = {
  main: "#E8963F",
  dark: "#C97A2B",
  light: "#F3B871",
  contrastText: "#FFFFFF",
};

const PALETTES = {
  light: {
    mode: "light",
    primary: amber,
    secondary: { main: "#2B2B2B", contrastText: "#FFFFFF" },
    background: { default: "#FFF8EE", paper: "#FFFBF3" },
    text: { primary: "#2B2B2B", secondary: "#7A7368" },
    divider: "#F0E4D0",
    error: { main: "#D64545" },
    success: { main: "#3F9A5B" },
  },
  dark: {
    mode: "dark",
    primary: amber,
    secondary: { main: "#EDE6D8", contrastText: "#211C15" },
    background: { default: "#17140F", paper: "#211C15" },
    text: { primary: "#F5EFE4", secondary: "#B7AC9A" },
    divider: "#3A3226",
    error: { main: "#E57373" },
    success: { main: "#66BB6A" },
  },
};

export function createAppTheme(mode = "light") {
  const palette = PALETTES[mode] ?? PALETTES.light;
```

```
return createTheme({
  palette,
  shape: {
    borderRadius: 16,
  },
  typography: {
    fontFamily: '"Inter", "Poppins", "Helvetica", "Arial", sans-serif',
    h1: { fontFamily: '"Poppins", sans-serif', fontWeight: 700 },
    h2: { fontFamily: '"Poppins", sans-serif', fontWeight: 700 },
    h3: { fontFamily: '"Poppins", sans-serif', fontWeight: 700 },
    h4: { fontFamily: '"Poppins", sans-serif', fontWeight: 600 },
    h5: { fontFamily: '"Poppins", sans-serif', fontWeight: 600 },
    h6: { fontFamily: '"Poppins", sans-serif', fontWeight: 600 },
    button: { textTransform: "none", fontWeight: 600 },
  },
  components: {
    MuiButton: {
      styleOverrides: {
        root: {
          borderRadius: 999,
          paddingLeft: 22,
          paddingRight: 22,
          paddingTop: 10,
          paddingBottom: 10,
        },
        containedPrimary: {
          boxShadow: "none",
          "&:hover": { boxShadow: "0 6px 16px rgba(232, 150, 63, 0.35)" },
        },
        outlined: {
          borderWidth: 1.5,
          "&:hover": { borderWidth: 1.5 },
        },
      },
    },
    MuiCard: {
      styleOverrides: {
        root: {
          borderRadius: 20,
          boxShadow:
            mode === "dark"
              ? "0 4px 20px rgba(0, 0, 0, 0.35)"
              : "0 4px 20px rgba(43, 43, 43, 0.06)",
        },
      },
    },
    MuiChip: {
      styleOverrides: {
        root: {
          borderRadius: 999,
          fontWeight: 600,
        },
      },
    },
    MuiPaper: {
      styleOverrides: {
        root: {
          backgroundImage: "none",
        },
      },
    },
    MuiAppBar: {
```

```
        styleOverrides: {
          root: {
            boxShadow:
              mode === "dark"
                ? "0 2px 12px rgba(0, 0, 0, 0.4)"
                : "0 2px 12px rgba(43, 43, 43, 0.06)",
          },
        },
      },
      MuiTextField: {
        defaultProps: {
          variant: "outlined",
        },
      },
      MuiOutlinedInput: {
        styleOverrides: {
          root: {
            borderRadius: 14,
          },
        },
      },
    },
  });
}
```

➢ AuthContext.jsx (Authentication context)

Path: frontend/src/context/AuthContext.jsx

Code:

```
import { createContext, useContext, useEffect, useState } from "react";
import { authService } from "../services/authService";

const AuthContext = createContext(null);

export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const token = localStorage.getItem("furniflex_token");
    if (!token) {
      setLoading(false);
      return;
    }
    authService
      .me()
      .then(setUser)
      .catch(() => {
        localStorage.removeItem("furniflex_token");
      })
      .finally(() => setLoading(false));
  }, []);

  const login = async (credentials) => {
    const data = await authService.login(credentials);
    localStorage.setItem("furniflex_token", data.access_token);
    setUser(data.user);
    return data.user;
  };

  const register = async (payload) => {
```

```
    const data = await authService.register(payload);
    localStorage.setItem("furniflex_token", data.access_token);
    setUser(data.user);
    return data.user;
  };

  const logout = () => {
    localStorage.removeItem("furniflex_token");
    setUser(null);
  };

  const updateProfile = async (payload) => {
    const updated = await authService.updateProfile(payload);
    setUser(updated);
    return updated;
  };

  return (
    <AuthContext.Provider value={{ user, loading, login, register, logout, updateProfile }}>
      {children}
    </AuthContext.Provider>
  );
}

export function useAuth() {
  const ctx = useContext(AuthContext);
  if (!ctx) throw new Error("useAuth must be used within AuthProvider");
  return ctx;
}
```

➢ CartContext.jsx (Cart context)

Path: frontend/src/context/CartContext.jsx

Code:

```
import { createContext, useCallback, useContext, useEffect, useState } from "react";
import { cartService } from "../services/cartService";
import { useAuth } from "./AuthContext";

const CartContext = createContext(null);

export function CartProvider({ children }) {
  const { user } = useAuth();
  const [items, setItems] = useState([]);
  const [loading, setLoading] = useState(false);

  const refresh = useCallback(async () => {
    if (!user) {
      setItems([]);
      return;
    }
    setLoading(true);
    try {
      const data = await cartService.list();
      setItems(data);
    } finally {
      setLoading(false);
    }
  }, [user]);

  useEffect(() => {
    refresh();
```

```
  }, [refresh]);

  const addToCart = async (productId, rentalPlanId, quantity = 1) => {
    await cartService.add({ product_id: productId, rental_plan_id: rentalPlanId, quantity
        });
    await refresh();
  };

  const updateItem = async (itemId, payload) => {
    await cartService.update(itemId, payload);
    await refresh();
  };
  const removeItem = async (itemId) => {
    await cartService.remove(itemId);
    await refresh();
  };

  const clearCart = async () => {
    await cartService.clear();
    setItems([]);
  };

  const priceFor = (item) => {
    const field = {
      Monthly: "monthly_price",
      Quarterly: "quarterly_price",
      Yearly: "yearly_price",
    }[item.rental_plan.name];
    return item.product[field] * item.quantity;
  };

  const total = items.reduce((sum, item) => sum + priceFor(item), 0);
  return (
    <CartContext.Provider
      value={{ items, loading, refresh, addToCart, updateItem, removeItem, clearCart,
          priceFor, total }}
    >
      {children}
    </CartContext.Provider>
  );
}

export function useCart() {
  const ctx = useContext(CartContext);
  if (!ctx) throw new Error("useCart must be used within CartProvider");
  return ctx;
}
```

➢ ThemeModeContext.jsx (Light / dark mode context)

Path: frontend/src/context/ThemeModeContext.jsx

Code:

```
import { createContext, useContext, useEffect, useMemo, useState } from "react";
import { ThemeProvider } from "@mui/material/styles";
import CssBaseline from "@mui/material/CssBaseline";

import { createAppTheme } from "../theme/theme";

const STORAGE_KEY = "furniflex_theme_mode";

const ThemeModeContext = createContext(null);

function getInitialMode() {
  const stored = localStorage.getItem(STORAGE_KEY);
  if (stored === "light" || stored === "dark") return stored;
  return window.matchMedia?.("(prefers-color-scheme: dark)").matches ? "dark" : "light";
}

export function ThemeModeProvider({ children }) {
  const [mode, setMode] = useState(getInitialMode);

  useEffect(() => {
    localStorage.setItem(STORAGE_KEY, mode);
  }, [mode]);

  const toggleMode = () => setMode((m) => (m === "light" ? "dark" : "light"));

  const theme = useMemo(() => createAppTheme(mode), [mode]);

  return (
    <ThemeModeContext.Provider value={{ mode, toggleMode }}>
      <ThemeProvider theme={theme}>
        <CssBaseline />
        {children}
      </ThemeProvider>
    </ThemeModeContext.Provider>
  );
}

export function useThemeMode() {
  const ctx = useContext(ThemeModeContext);
  if (!ctx) throw new Error("useThemeMode must be used within ThemeModeProvider");
  return ctx;
}
```

➢ api.js (Axios API client)

Path: frontend/src/services/api.js

Code:

```
import axios from "axios";

const API_BASE_URL = import.meta.env.VITE_API_BASE_URL || "http://localhost:8000/api";

// The backend serves uploaded/seeded images from /static, alongside the /api routes.
export const ASSET_BASE_URL = API_BASE_URL.replace(/\/api\/?$/, "");

// Product/category images are stored as paths like "/static/images/bedroom/img-1.jpg".
// This resolves them against the backend origin; absolute URLs pass through unchanged.
export function resolveImageUrl(path) {
  if (!path) return path;
  if (/^https?:\/\//i.test(path)) return path;
  return `${ASSET_BASE_URL}${path}`;
}

export const api = axios.create({
  baseURL: API_BASE_URL,
});

api.interceptors.request.use((config) => {
  const token = localStorage.getItem("furniflex_token");
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});
```

➢ authService.js (Auth service)

Path: frontend/src/services/authService.js

Code:

```
import { api } from "./api";

export const authService = {
  register: (payload) => api.post("/auth/register", payload).then((res) => res.data),
  login: (payload) => api.post("/auth/login", payload).then((res) => res.data),
  me: () => api.get("/auth/me").then((res) => res.data),
  updateProfile: (payload) => api.put("/users/me", payload).then((res) => res.data),
};
```

➢ productService.js (Product service)

Path: frontend/src/services/productService.js

Code:

```
import { api } from "./api";

export const productService = {
  list: (params) => api.get("/products", { params }).then((res) => res.data),
  get: (id) => api.get(`/products/${id}`).then((res) => res.data),
  create: (payload) => api.post("/products", payload).then((res) => res.data),
  update: (id, payload) => api.put(`/products/${id}`, payload).then((res) => res.data),
  remove: (id) => api.delete(`/products/${id}`),
};

export const rentalPlanService = {
  list: () => api.get("/rental-plans").then((res) => res.data),
};
```

➢ categoryService.js (Category service)

Path: frontend/src/services/categoryService.js

Code:

```
import { api } from "./api";

export const categoryService = {
  list: () => api.get("/categories").then((res) => res.data),
  get: (id) => api.get(`/categories/${id}`).then((res) => res.data),
  create: (payload) => api.post("/categories", payload).then((res) => res.data),
  update: (id, payload) => api.put(`/categories/${id}`, payload).then((res) => res.data),
  remove: (id) => api.delete(`/categories/${id}`),
};
```

➢ cartService.js (Cart service)

Path: frontend/src/services/cartService.js

Code:

```
import { api } from "./api";

export const cartService = {
  list: () => api.get("/cart").then((res) => res.data),
  add: (payload) => api.post("/cart", payload).then((res) => res.data),
  update: (itemId, payload) => api.put(`/cart/${itemId}`, payload).then((res) => res.data),
  remove: (itemId) => api.delete(`/cart/${itemId}`),
  clear: () => api.delete("/cart"),
};
```

➢ orderService.js (Order service)

Path: frontend/src/services/orderService.js

Code:

```
import { api } from "./api";

export const orderService = {
  checkout: (payload) => api.post("/orders/checkout", payload).then((res) => res.data),
  myOrders: () => api.get("/orders").then((res) => res.data),
  get: (id) => api.get(`/orders/${id}`).then((res) => res.data),
};

export const adminService = {
  dashboard: () => api.get("/admin/dashboard").then((res) => res.data),
  users: () => api.get("/admin/users").then((res) => res.data),
  orders: () => api.get("/admin/orders").then((res) => res.data),
  updateOrderStatus: (id, status) =>
    api.patch(`/admin/orders/${id}/status`, { status }).then((res) => res.data),
};
```

➢ uploadService.js (Image upload service)

Path: frontend/src/services/uploadService.js

Code:

```
import { api } from "./api";

export const uploadService = {
  uploadImage: (file, categorySlug) => {
    const formData = new FormData();
    formData.append("file", file);
    formData.append("category_slug", categorySlug);
    return api
      .post("/admin/upload-image", formData, {
        headers: { "Content-Type": "multipart/form-data" },
      })
      .then((res) => res.data.url);
  },
};
```

➢ Navbar.jsx (Navigation bar)

Path: frontend/src/components/layout/Navbar.jsx

Code:

```
import { useState } from "react";
import { Link as RouterLink, useNavigate } from "react-router-dom";
import {
  AppBar,
  Avatar,
  Badge,
  Box,
  Button,
  Drawer,
  IconButton,
  InputAdornment,
  List,
  ListItemButton,
  ListItemText,
  Menu,
  MenuItem,
  TextField,
  Toolbar,
  Tooltip,
  Typography,
  useMediaQuery,
  useTheme,
} from "@mui/material";
import MenuIcon from "@mui/icons-material/Menu";
import SearchIcon from "@mui/icons-material/Search";
import ShoppingCartOutlinedIcon from "@mui/icons-material/ShoppingCartOutlined";
import ChairAltIcon from "@mui/icons-material/ChairAlt";
import LightModeIcon from "@mui/icons-material/LightMode";
import DarkModeIcon from "@mui/icons-material/DarkMode";
import { useAuth } from "../../context/AuthContext";
import { useCart } from "../../context/CartContext";
import { useThemeMode } from "../../context/ThemeModeContext";

const NAV_LINKS = [
  { label: "Home", to: "/" },
  { label: "Categories", to: "/categories" },
];

export default function Navbar() {
  const theme = useTheme();
  const isMobile = useMediaQuery(theme.breakpoints.down("md"));
  const navigate = useNavigate();
  const { user, logout } = useAuth();
  const { items } = useCart();
  const { mode, toggleMode } = useThemeMode();

  const [drawerOpen, setDrawerOpen] = useState(false);
  const [anchorEl, setAnchorEl] = useState(null);
  const [searchValue, setSearchValue] = useState("");

  const cartCount = items.reduce((sum, item) => sum + item.quantity, 0);

  const handleLogout = () => {
    setAnchorEl(null);
    logout();
    navigate("/");
  };
```

```
const handleSearchSubmit = (e) => {
  e.preventDefault();
  navigate(`/search${searchValue.trim() ? `?q=${encodeURIComponent(searchValue.trim())}`
      : ""}`);
};

return (
  <AppBar position="sticky" color="inherit" sx={{ bgcolor: "background.paper" }}>
    <Toolbar sx={{ maxWidth: 1280, width: "100%", mx: "auto" }}>
      {isMobile && (
        <IconButton edge="start" onClick={() => setDrawerOpen(true)} sx={{ mr: 1 }}>
          <MenuIcon />
        </IconButton>
      )}

      <Box
        component={RouterLink}
        to="/"
        sx={{ display: "flex", alignItems: "center", gap: 1, textDecoration: "none",
            flexGrow: isMobile ? 1 : 0 }}
      >
        <ChairAltIcon sx={{ color: "primary.main", fontSize: 32 }} />
        <Typography variant="h6" sx={{ color: "text.primary", fontWeight: 700 }}>
          FurniFlex
        </Typography>
      </Box>

      {!isMobile && (
        <Box sx={{ display: "flex", alignItems: "center", gap: 1, ml: 4, flexGrow: 1 }}>
          {NAV_LINKS.map((link) => (
            <Button key={link.to} component={RouterLink} to={link.to} color="inherit">
              {link.label}
            </Button>
          ))}
          <Box component="form" onSubmit={handleSearchSubmit} sx={{ ml: 2, flexGrow: 1,
              maxWidth: 320 }}>
            <TextField
              size="small"
              fullWidth
              placeholder="Search furniture..."
              value={searchValue}
              onChange={(e) => setSearchValue(e.target.value)}
              slotProps={{
                input: {
                  startAdornment: (
                    <InputAdornment position="start">
                      <SearchIcon fontSize="small" color="action" />
                    </InputAdornment>
                  ),
                },
              }}
              sx={{ bgcolor: "background.default", borderRadius: 3 }}
            />
          </Box>
        </Box>
      )}

      <Box sx={{ display: "flex", alignItems: "center", gap: 1, ml: isMobile ? "auto" : 0
          }}>
        {isMobile && (
          <IconButton component={RouterLink} to="/search" aria-label="search">
```

```
            <SearchIcon />
          </IconButton>
        )}

        <Tooltip title={mode === "light" ? "Switch to dark mode" : "Switch to light
            mode"}>
          <IconButton onClick={toggleMode} aria-label="toggle color mode">
            {mode === "light" ? <DarkModeIcon /> : <LightModeIcon />}
          </IconButton>
        </Tooltip>

        <IconButton component={RouterLink} to="/cart" aria-label="cart">
          <Badge badgeContent={cartCount} color="primary">
            <ShoppingCartOutlinedIcon />
          </Badge>
        </IconButton>

        {user ? (
          <>
            <IconButton onClick={(e) => setAnchorEl(e.currentTarget)}>
              <Avatar sx={{ width: 34, height: 34, bgcolor: "primary.main" }}>
                {user.name?.[0]?.toUpperCase() || "U"}
              </Avatar>
            </IconButton>
            <Menu anchorEl={anchorEl} open={Boolean(anchorEl)} onClose={() =>
                setAnchorEl(null)}>
              <MenuItem component={RouterLink} to="/profile" onClick={() =>
                  setAnchorEl(null)}>
                Profile
              </MenuItem>
              <MenuItem component={RouterLink} to="/orders" onClick={() =>
                  setAnchorEl(null)}>
                Order History
              </MenuItem>
              {user.is_admin && (
                <MenuItem component={RouterLink} to="/admin" onClick={() =>
                    setAnchorEl(null)}>
                  Admin Panel
                </MenuItem>
              )}
              <MenuItem onClick={handleLogout}>Logout</MenuItem>
            </Menu>
          </>
        ) : (
          <Button component={RouterLink} to="/login" variant="contained" color="primary">
            Login
          </Button>
        )}
      </Box>
    </Toolbar>

    <Drawer anchor="left" open={drawerOpen} onClose={() => setDrawerOpen(false)}>
      <Box sx={{ width: 240 }} role="presentation" onClick={() => setDrawerOpen(false)}>
        <List>
          {NAV_LINKS.map((link) => (
            <ListItemButton key={link.to} component={RouterLink} to={link.to}>
              <ListItemText primary={link.label} />
            </ListItemButton>
          ))}
        </List>
      </Box>
```

```
      </Drawer>
    </AppBar>
  );
}
```

➢ Footer.jsx (Footer)

Path: frontend/src/components/layout/Footer.jsx

Code:

```
import { Box, Container, Grid, Link, Typography } from "@mui/material";
import ChairAltIcon from "@mui/icons-material/ChairAlt";

export default function Footer() {
  return (
    <Box component="footer" sx={{ bgcolor: "#211C15", color: "#fff", mt: 8, py: 5 }}>
      <Container maxWidth="lg">
        <Grid container spacing={4}>
          <Grid size={{ xs: 12, sm: 4 }}>
            <Box sx={{ display: "flex", alignItems: "center", gap: 1, mb: 1 }}>
              <ChairAltIcon sx={{ color: "primary.main" }} />
              <Typography variant="h6" fontWeight={700}>
                FurniFlex
              </Typography>
            </Box>
            <Typography variant="body2" sx={{ opacity: 0.8 }}>
              Rent quality furniture &amp;amp; appliances for your home, flexibly and
                  affordably.
            </Typography>
          </Grid>
          <Grid size={{ xs: 6, sm: 4 }}>
            <Typography variant="subtitle2" sx={{ mb: 1, opacity: 0.9 }}>
              Company
            </Typography>
            <Link href="#" underline="hover" color="inherit" display="block" sx={{ opacity:
                0.8, mb: 0.5 }}>
              About Us
            </Link>
            <Link href="#" underline="hover" color="inherit" display="block" sx={{ opacity:
                0.8, mb: 0.5 }}>
              Careers
            </Link>
            <Link href="#" underline="hover" color="inherit" display="block" sx={{ opacity:
                0.8 }}>
              Contact
            </Link>
          </Grid>
          <Grid size={{ xs: 6, sm: 4 }}>
            <Typography variant="subtitle2" sx={{ mb: 1, opacity: 0.9 }}>
              Support
            </Typography>
            <Link href="#" underline="hover" color="inherit" display="block" sx={{ opacity:
                0.8, mb: 0.5 }}>
              FAQs
            </Link>
            <Link href="#" underline="hover" color="inherit" display="block" sx={{ opacity:
                0.8, mb: 0.5 }}>
              Terms &amp;amp; Conditions
            </Link>
            <Link href="#" underline="hover" color="inherit" display="block" sx={{ opacity:
                0.8 }}>
              Privacy Policy
```

```
            </Link>
          </Grid>
        </Grid>
        <Typography variant="caption" sx={{ display: "block", mt: 4, opacity: 0.6,
            textAlign: "center" }}>
          © {new Date().getFullYear()} FurniFlex. All rights reserved.
        </Typography>
        <Typography variant="caption" sx={{ display: "block", mt: 0.5, opacity: 0.8,
            textAlign: "center", fontWeight: 600 }}>
          Built for a B.N.N College Project By Rehana Mansuri
        </Typography>
      </Container>
    </Box>
  );
}
```

➢ ProtectedRoute.jsx (Protected route guard)

Path: frontend/src/components/layout/ProtectedRoute.jsx

Code:

```
import { Navigate, useLocation } from "react-router-dom";
import { useAuth } from "../../context/AuthContext";
import LoadingSpinner from "../common/LoadingSpinner";

export default function ProtectedRoute({ children }) {
  const { user, loading } = useAuth();
  const location = useLocation();

  if (loading) return <LoadingSpinner />;
  if (!user) return <Navigate to="/login" state={{ from: location }} replace />;
  return children;
}
```

➢ AdminRoute.jsx (Admin route guard)

Path: frontend/src/components/layout/AdminRoute.jsx

Code:

```
import { Navigate } from "react-router-dom";
import { useAuth } from "../../context/AuthContext";
import LoadingSpinner from "../common/LoadingSpinner";

export default function AdminRoute({ children }) {
  const { user, loading } = useAuth();

  if (loading) return <LoadingSpinner />;
  if (!user) return <Navigate to="/login" replace />;
  if (!user.is_admin) return <Navigate to="/" replace />;
  return children;
}
```

➢ SearchBar.jsx (Search bar)

Path: frontend/src/components/common/SearchBar.jsx

Code:

```
import { InputAdornment, TextField } from "@mui/material";
import SearchIcon from "@mui/icons-material/Search";

export default function SearchBar({ value, onChange, placeholder = "Search furniture..." })
    {
  return (
    <TextField
      fullWidth
      value={value}
      onChange={(e) => onChange(e.target.value)}
      placeholder={placeholder}
      slotProps={{
        input: {
          startAdornment: (
            <InputAdornment position="start">
              <SearchIcon color="action" />
            </InputAdornment>
          ),
        },
      }}
      sx={{ bgcolor: "background.paper", borderRadius: 3 }}
    />
  );
}
```

➢ CategoryTile.jsx (Category tile)

Path: frontend/src/components/common/CategoryTile.jsx

Code:

```
import { Link as RouterLink } from "react-router-dom";
import { Box, Typography } from "@mui/material";
import { resolveImageUrl } from "../../services/api";

export default function CategoryTile({ category }) {
  return (
    <Box
      component={RouterLink}
      to={`/categories/${category.slug}`}
      sx={{
        textDecoration: "none",
        display: "flex",
        flexDirection: "column",
        alignItems: "center",
        gap: 1,
        transition: "transform 0.15s ease",
        "&:hover": { transform: "translateY(-4px)" },
      }}
    >
      <Box
        sx={{
          width: 88,
          height: 88,
          borderRadius: "50%",
          overflow: "hidden",
          border: "3px solid",
```

```
          borderColor: "background.paper",
          boxShadow: "0 4px 16px rgba(43,43,43,0.1)",
        }}
      >
        <Box
          component="img"
          src={resolveImageUrl(category.image_url)}
          alt={category.name}
          sx={{ width: "100%", height: "100%", objectFit: "cover" }}
        />
      </Box>
      <Typography variant="body2" fontWeight={600} color="text.primary" textAlign="center">
        {category.name}
      </Typography>
    </Box>
  );
}
```

➢ LoadingSpinner.jsx (Loading spinner)

Path: frontend/src/components/common/LoadingSpinner.jsx

Code:

```
import { Box, CircularProgress } from "@mui/material";

export default function LoadingSpinner({ minHeight = "50vh" }) {
  return (
    <Box display="flex" alignItems="center" justifyContent="center" minHeight={minHeight}>
      <CircularProgress color="primary" />
    </Box>
  );
}
```

➢ ProductCard.jsx (Product card)

Path: frontend/src/components/product/ProductCard.jsx

Code:

```
import { Link as RouterLink } from "react-router-dom";
import { Box, Button, Card, CardContent, CardMedia, Chip, Stack, Typography } from
    "@mui/material";
import { resolveImageUrl } from "../../services/api";

const currency = new Intl.NumberFormat("en-IN", {
  style: "currency",
  currency: "INR",
  maximumFractionDigits: 0,
});

export default function ProductCard({ product }) {
  return (
    <Card sx={{ height: "100%", display: "flex", flexDirection: "column" }}>
      <Box sx={{ position: "relative" }}>
        <CardMedia
          component="img"
          image={resolveImageUrl(product.image_urls?.[0])}
          alt={product.name}
          sx={{ height: 200, objectFit: "contain", bgcolor: "grey.100" }}
        />
        {product.is_featured && (
          <Chip
            label="Featured"
```

```
            size="small"
            color="primary"
            sx={{ position: "absolute", top: 12, left: 12, fontWeight: 700 }}
          />
        )}
      </Box>
      <CardContent sx={{ display: "flex", flexDirection: "column", flexGrow: 1 }}>
        <Typography variant="subtitle1" fontWeight={600} noWrap>
          {product.name}
        </Typography>
        <Stack direction="row" alignItems="baseline" spacing={0.5} sx={{ mt: 0.5, mb: 2 }}>
          <Typography variant="h6" color="primary.dark" fontWeight={700}>
            {currency.format(product.monthly_price)}
          </Typography>
          <Typography variant="body2" color="text.secondary">
            /month
          </Typography>
        </Stack>
        <Box sx={{ flexGrow: 1 }} />
        <Button
          component={RouterLink}
          to={`/products/${product.id}`}
          variant="outlined"
          color="primary"
          fullWidth
        >
          View Details
        </Button>
      </CardContent>
    </Card>
  );
}
```

➢ RentalPlanSelector.jsx (Rental plan selector)

Path: frontend/src/components/product/RentalPlanSelector.jsx

Code:

```
import { Box, Stack, Typography } from "@mui/material";

const currency = new Intl.NumberFormat("en-IN", {
  style: "currency",
  currency: "INR",
  maximumFractionDigits: 0,
});

const PLAN_PRICE_FIELD = {
  Monthly: "monthly_price",
  Quarterly: "quarterly_price",
  Yearly: "yearly_price",
};

export default function RentalPlanSelector({ plans, product, selectedPlanId, onChange }) {
  return (
    <Stack direction="row" spacing={1.5} flexWrap="wrap" useFlexGap>
      {plans.map((plan) => {
        const price = product[PLAN_PRICE_FIELD[plan.name]];
        const selected = plan.id === selectedPlanId;
        return (
          <Box
            key={plan.id}
            onClick={() => onChange(plan.id)}
```

```
            sx={{
              cursor: "pointer",
              borderRadius: 3,
              border: "1.5px solid",
              borderColor: selected ? "primary.main" : "divider",
              bgcolor: selected ? "primary.main" : "background.paper",
              color: selected ? "primary.contrastText" : "text.primary",
              px: 2.5,
              py: 1.5,
              minWidth: 128,
              textAlign: "center",
              transition: "all 0.15s ease",
            }}
          >
            <Typography variant="subtitle2" fontWeight={700}>
              {plan.name}
            </Typography>
            <Typography variant="body2" sx={{ opacity: selected ? 0.95 : 0.7 }}>
              {currency.format(price)}
            </Typography>
            {plan.discount_percent > 0 && (
              <Typography variant="caption" sx={{ opacity: selected ? 0.85 : 0.6 }}>
                Save {plan.discount_percent}%
              </Typography>
            )}
          </Box>
        );
      })}
    </Stack>
  );
}
```

➢ PaymentMethodSelector.jsx (Payment method selector)

Path: frontend/src/components/checkout/PaymentMethodSelector.jsx

Code:

```
import { Box, Stack, TextField, ToggleButton, ToggleButtonGroup, Typography } from
    "@mui/material";
import CreditCardIcon from "@mui/icons-material/CreditCard";
import AccountBalanceWalletIcon from "@mui/icons-material/AccountBalanceWallet";
import QrCode2Icon from "@mui/icons-material/QrCode2";

const METHODS = [
  { value: "CARD", label: "Credit / Debit Card", icon: <CreditCardIcon fontSize="small" />
      },
  { value: "UPI", label: "UPI", icon: <AccountBalanceWalletIcon fontSize="small" /> },
  { value: "GPAY_QR", label: "GPay QR", icon: <QrCode2Icon fontSize="small" /> },
];

export default function PaymentMethodSelector({ method, onMethodChange, card, onCardChange,
    upiId, onUpiIdChange }) {
  return (
    <Box>
      <ToggleButtonGroup
        exclusive
        value={method}
        onChange={(_, value) => value && onMethodChange(value)}
        sx={{ mb: 3, flexWrap: "wrap" }}
      >
        {METHODS.map((m) => (
          <ToggleButton key={m.value} value={m.value} sx={{ gap: 1, px: 2.5 }}>
```

```
            {m.icon}
            {m.label}
          </ToggleButton>
        ))}
      </ToggleButtonGroup>

      {method === "CARD" && (
        <Stack spacing={2}>
          <TextField
            fullWidth
            label="Card Number"
            placeholder="1234 5678 9012 3456"
            value={card.number}
            onChange={(e) => onCardChange({ ...card, number: e.target.value })}
          />
          <TextField
            fullWidth
            label="Name on Card"
            value={card.name}
            onChange={(e) => onCardChange({ ...card, name: e.target.value })}
          />
          <Stack direction="row" spacing={2}>
            <TextField
              fullWidth
              label="Expiry (MM/YY)"
              placeholder="12/28"
              value={card.expiry}
              onChange={(e) => onCardChange({ ...card, expiry: e.target.value })}
            />
            <TextField
              fullWidth
              label="CVV"
              placeholder="123"
              type="password"
              value={card.cvv}
              onChange={(e) => onCardChange({ ...card, cvv: e.target.value })}
            />
          </Stack>
          <Typography variant="caption" color="text.secondary">
            Demo checkout — this is a simulated payment form for the college project and
                does not
            process any real transaction.
          </Typography>
        </Stack>
      )}

      {method === "UPI" && (
        <Stack spacing={2}>
          <TextField
            fullWidth
            label="UPI ID"
            placeholder="yourname@upi"
            value={upiId}
            onChange={(e) => onUpiIdChange(e.target.value)}
          />
          <Typography variant="caption" color="text.secondary">
            Demo checkout — enter any UPI ID. No real payment request is sent.
          </Typography>
        </Stack>
      )}

      {method === "GPAY_QR" && (
```

```
        <Stack alignItems="center" spacing={1.5} sx={{ py: 2 }}>
          <Box
            sx={{
              width: 200,
              height: 200,
              borderRadius: 3,
              border: "2px dashed",
              borderColor: "divider",
              display: "flex",
              alignItems: "center",
              justifyContent: "center",
              bgcolor: "background.default",
            }}
          >
            <QrCode2Icon sx={{ fontSize: 96, color: "text.secondary", opacity: 0.5 }} />
          </Box>
          <Typography variant="body2" fontWeight={600}>
            Scan with GPay or any UPI app
          </Typography>
          <Typography variant="caption" color="text.secondary" textAlign="center">
            Demo QR placeholder — no real transaction is made. Click "Place Order" once
                you've
            simulated the scan.
          </Typography>
        </Stack>
      )}
    </Box>
  );
}
```

➢ Home.jsx (Home Page of Website)

Path: frontend/src/pages/Home.jsx

Code:

```
import { useEffect, useState } from "react";
import { Link as RouterLink } from "react-router-dom";
import { Box, Button, Container, Grid, Stack, Typography } from "@mui/material";
import ArrowForwardIcon from "@mui/icons-material/ArrowForward";

import { categoryService } from "../services/categoryService";
import { productService } from "../services/productService";
import CategoryTile from "../components/common/CategoryTile";
import ProductCard from "../components/product/ProductCard";
import LoadingSpinner from "../components/common/LoadingSpinner";

export default function Home() {
  const [categories, setCategories] = useState([]);
  const [featured, setFeatured] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    Promise.all([categoryService.list(), productService.list({ featured: true })])
      .then(([categoryData, productData]) => {
        setCategories(categoryData);
        setFeatured(productData);
      })
      .finally(() => setLoading(false));
  }, []);

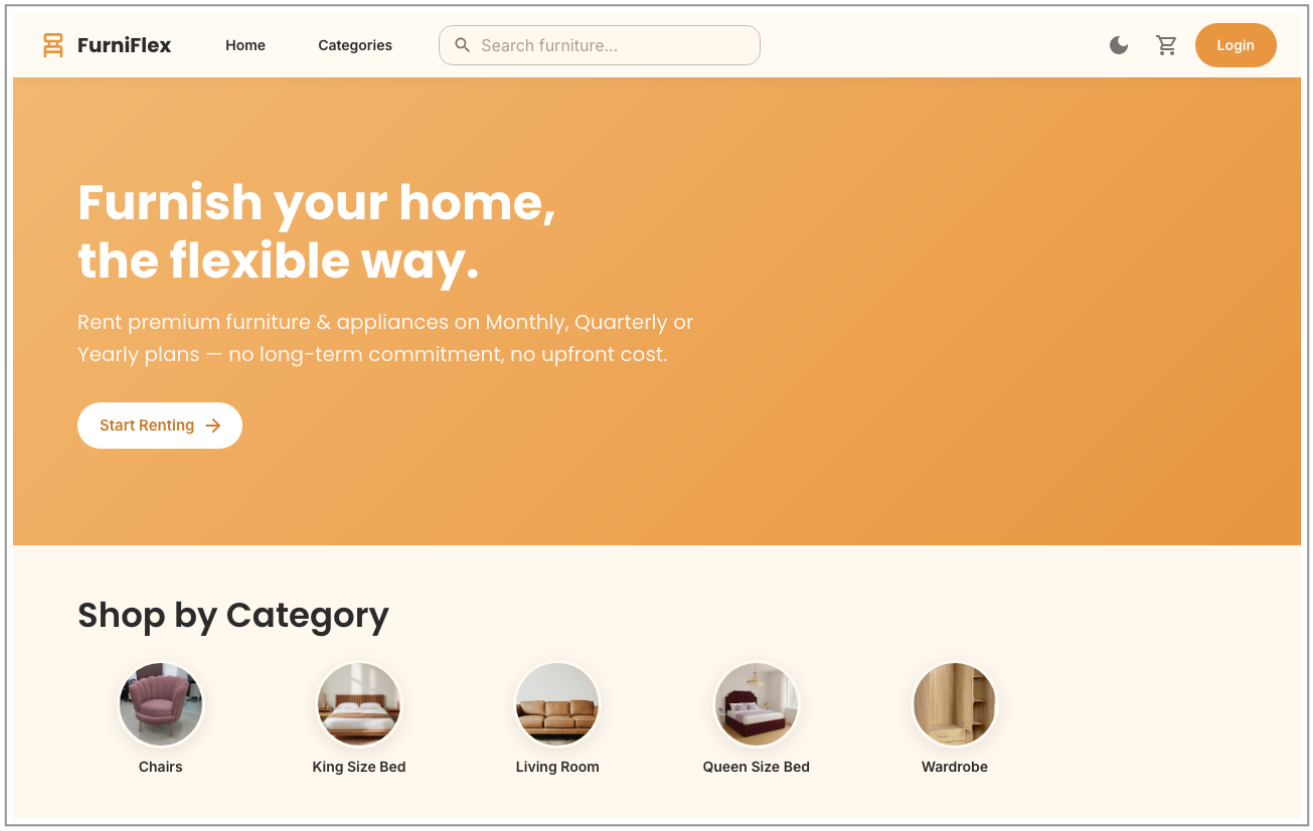
  if (loading) return <LoadingSpinner minHeight="80vh" />;
```

```
return (
  <Box>
    <Box
      sx={{
        background: "linear-gradient(135deg, #F3B871 0%, #E8963F 100%)",
        py: { xs: 8, md: 12 },
      }}
    >
      <Container maxWidth="lg">
        <Grid container spacing={4} alignItems="center">
          <Grid size={{ xs: 12, md: 7 }}>
            <Typography variant="h2" sx={{ color: "#fff", fontSize: { xs: 34, md: 48 },
                mb: 2 }}>
              Furnish your home,
              <br />
              the flexible way.
            </Typography>
            <Typography variant="h6" sx={{ color: "rgba(255,255,255,0.9)", fontWeight:
                400, mb: 4 }}>
              Rent premium furniture &amp;amp; appliances on Monthly, Quarterly or Yearly
              plans — no long-term commitment, no upfront cost.
            </Typography>
            <Button
              component={RouterLink}
              to="/categories"
              variant="contained"
              size="large"
              endIcon={<ArrowForwardIcon />}
              sx={{
                bgcolor: "#fff",
                color: "primary.dark",
                "&:hover": { bgcolor: "#FFF8EE" },
              }}
            >
              Start Renting
            </Button>
          </Grid>
        </Grid>
      </Container>
    </Box>

    <Container maxWidth="lg" sx={{ py: 6 }}>
      <Typography variant="h4" sx={{ mb: 3 }}>
        Shop by Category
      </Typography>
      <Grid container spacing={{ xs: 3, md: 4 }}>
        {categories.map((category) => (
          <Grid key={category.id} size={{ xs: 4, sm: 3, md: 2 }}>
            <CategoryTile category={category} />
          </Grid>
        ))}
      </Grid>
    </Container>
    {featured.length > 0 && (
      <Container maxWidth="lg" sx={{ py: 4 }}>
        <Stack direction="row" justifyContent="space-between" alignItems="center" sx={{
            mb: 3 }}>
          <Typography variant="h4">Featured Furniture</Typography>
          <Button component={RouterLink} to="/categories" endIcon={<ArrowForwardIcon />}>
            View All
          </Button>
        </Stack>
```

```
          <Grid container spacing={3}>
            {featured.slice(0, 8).map((product) => (
              <Grid key={product.id} size={{ xs: 12, sm: 6, md: 3 }}>
                <ProductCard product={product} />
              </Grid>
            ))}
          </Grid>
        </Container>
      )}
    </Box>
  );
}
```

Output:



➢ Login.jsx (Login Page)

Path: frontend/src/pages/Login.jsx

Code:

```
import { useState } from "react";
import { Link as RouterLink, useLocation, useNavigate } from "react-router-dom";
import {
  Alert,
  Box,
  Button,
  Container,
  Link,
  Paper,
  Stack,
  TextField,
  Typography,
} from "@mui/material";
import ChairAltIcon from "@mui/icons-material/ChairAlt";

import { useAuth } from "../context/AuthContext";

export default function Login() {
  const { login } = useAuth();
  const navigate = useNavigate();
  const location = useLocation();

  const [form, setForm] = useState({ email: "", password: "" });
  const [error, setError] = useState("");
  const [loading, setLoading] = useState(false);

  const handleChange = (field) => (e) => setForm({ ...form, [field]: e.target.value });

  const handleSubmit = async (e) => {
    e.preventDefault();
    setError("");
    setLoading(true);
    try {
      await login(form);
      const redirectTo = location.state?.from?.pathname || "/";
      navigate(redirectTo, { replace: true });
    } catch {
      setError("Invalid email or password.");
    } finally {
      setLoading(false);
    }
  };

  return (
    <Container maxWidth="xs" sx={{ py: 10 }}>
      <Stack alignItems="center" spacing={1} sx={{ mb: 4 }}>
        <ChairAltIcon sx={{ color: "primary.main", fontSize: 40 }} />
        <Typography variant="h5">Welcome back</Typography>
        <Typography variant="body2" color="text.secondary">
          Login to continue to FurniFlex
        </Typography>
      </Stack>

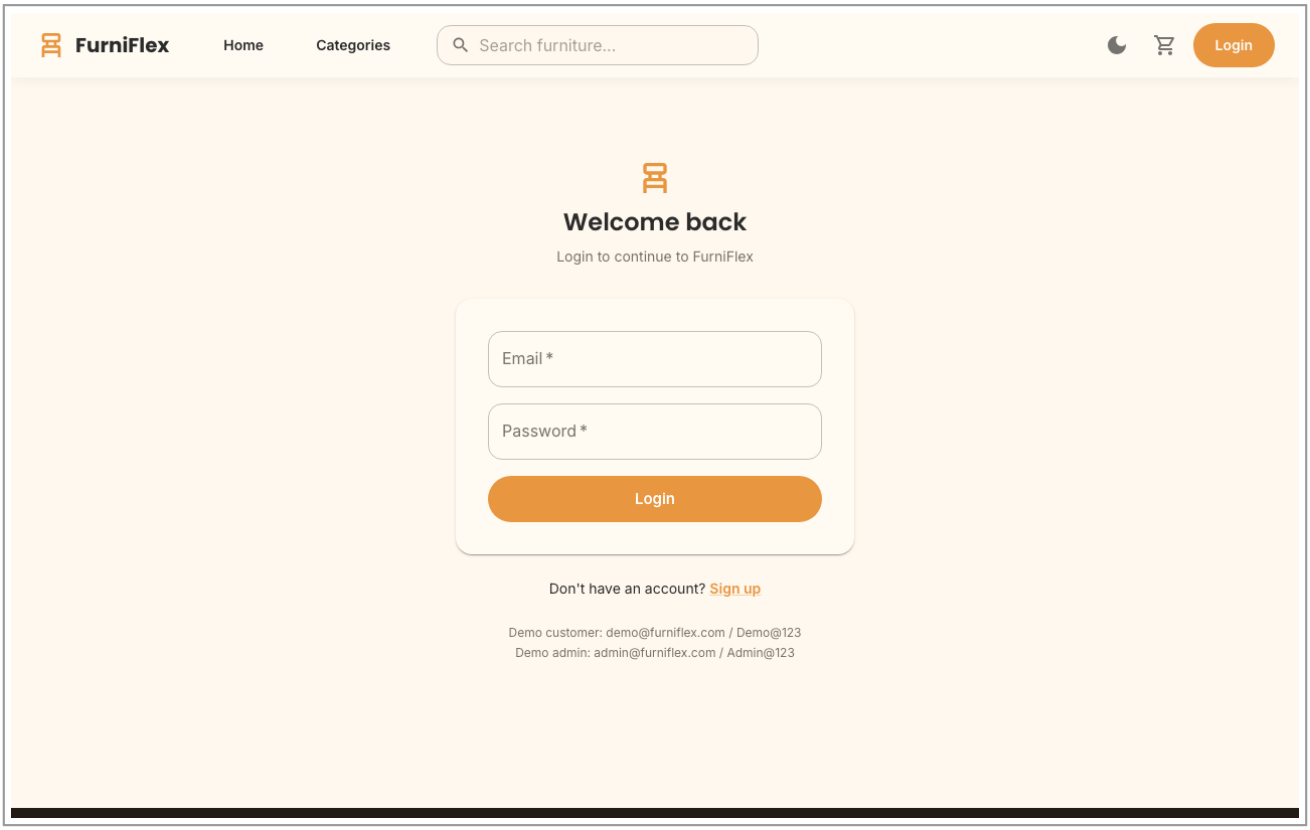
      <Paper sx={{ p: 4 }}>
        <Box component="form" onSubmit={handleSubmit}>
          {error && (
            <Alert severity="error" sx={{ mb: 2 }}>
```

```
            {error}
          </Alert>
        )}
        <Stack spacing={2}>
          <TextField
            type="email"
            label="Email"
            required
            value={form.email}
            onChange={handleChange("email")}
          />
          <TextField
            type="password"
            label="Password"
            required
            value={form.password}
            onChange={handleChange("password")}
          />
          <Button type="submit" variant="contained" color="primary" size="large"
              disabled={loading}>
            {loading ? "Logging in..." : "Login"}
          </Button>
        </Stack>
      </Box>
    </Paper>

    <Typography variant="body2" textAlign="center" sx={{ mt: 3 }}>
      Don't have an account?{" "}
      <Link component={RouterLink} to="/register" fontWeight={600}>
        Sign up
      </Link>
    </Typography>

    <Typography variant="caption" display="block" textAlign="center" sx={{ mt: 3 }}
        color="text.secondary">
      Demo customer: demo@furniflex.com / Demo@123
      <br />
      Demo admin: admin@furniflex.com / Admin@123
    </Typography>
  </Container>
 );
}
```

Output:



➢ Register.jsx (Registration Page)

Path: frontend/src/pages/Register.jsx

Code:

```
import { useState } from "react";
import { Link as RouterLink, useNavigate } from "react-router-dom";
import {
  Alert,
  Box,
  Button,
  Container,
  Link,
  Paper,
  Stack,
  TextField,
  Typography,
} from "@mui/material";
import ChairAltIcon from "@mui/icons-material/ChairAlt";

import { useAuth } from "../context/AuthContext";

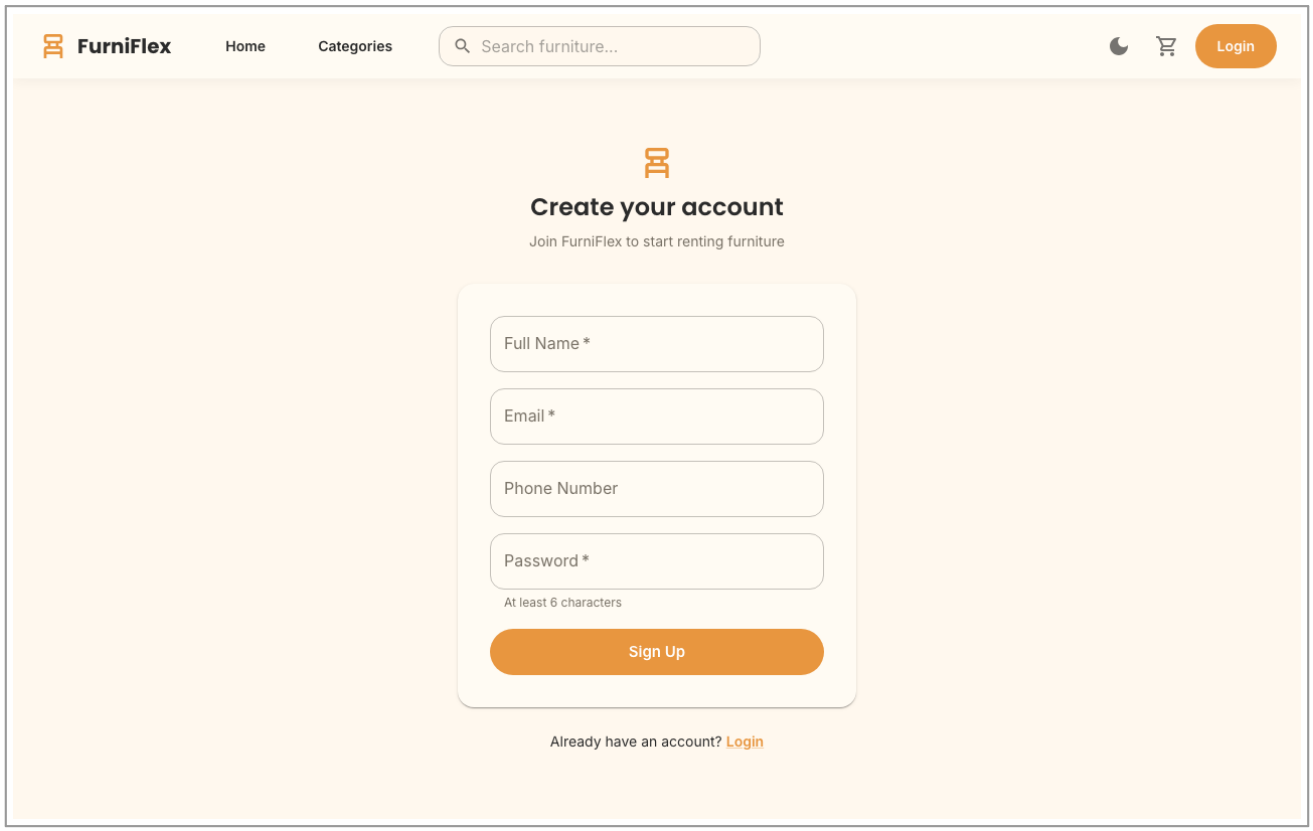
export default function Register() {
  const { register } = useAuth();
  const navigate = useNavigate();
  const [form, setForm] = useState({ name: "", email: "", phone: "", password: "" });
  const [error, setError] = useState("");
  const [loading, setLoading] = useState(false);

  const handleChange = (field) => (e) => setForm({ ...form, [field]: e.target.value });
```

```
  const handleSubmit = async (e) => {
    e.preventDefault();
    setError("");
    setLoading(true);
    try {
      await register(form);
      navigate("/", { replace: true });
    } catch (err) {
      setError(err?.response?.data?.detail || "Could not create account.");
    } finally {
      setLoading(false);
    }
  };
  return (
    <Container maxWidth="xs" sx={{ py: 8 }}>
      <Stack alignItems="center" spacing={1} sx={{ mb: 4 }}>
        <ChairAltIcon sx={{ color: "primary.main", fontSize: 40 }} />
        <Typography variant="h5">Create your account</Typography>
        <Typography variant="body2" color="text.secondary">
          Join FurniFlex to start renting furniture
        </Typography>
      </Stack>

      <Paper sx={{ p: 4 }}>
        <Box component="form" onSubmit={handleSubmit}>
          {error && (
            <Alert severity="error" sx={{ mb: 2 }}>
              {error}
            </Alert>
          )}
          <Stack spacing={2}>
            <TextField label="Full Name" required value={form.name}
                onChange={handleChange("name")} />
            <TextField type="email" label="Email" required value={form.email}
                onChange={handleChange("email")} />
            <TextField label="Phone Number" value={form.phone}
                onChange={handleChange("phone")} />
            <TextField
              type="password"
              label="Password"
              required
              helperText="At least 6 characters"
              value={form.password}
              onChange={handleChange("password")}
            />
            <Button type="submit" variant="contained" color="primary" size="large"
                disabled={loading}>
              {loading ? "Creating account..." : "Sign Up"}
            </Button>
          </Stack>
        </Box>
      </Paper>
      <Typography variant="body2" textAlign="center" sx={{ mt: 3 }}>
        Already have an account?{" "}
        <Link component={RouterLink} to="/login" fontWeight={600}>
          Login
        </Link>
      </Typography>
    </Container>
  );
}
```

Output:



➢ Categories.jsx (All Categories Page)

Path: frontend/src/pages/Categories.jsx

Code:

```
import { useEffect, useState } from "react";
import { Container, Grid, Typography } from "@mui/material";
import { categoryService } from "../services/categoryService";
import CategoryTile from "../components/common/CategoryTile";
import LoadingSpinner from "../components/common/LoadingSpinner";

export default function Categories() {
  const [categories, setCategories] = useState([]);
  const [loading, setLoading] = useState(true);

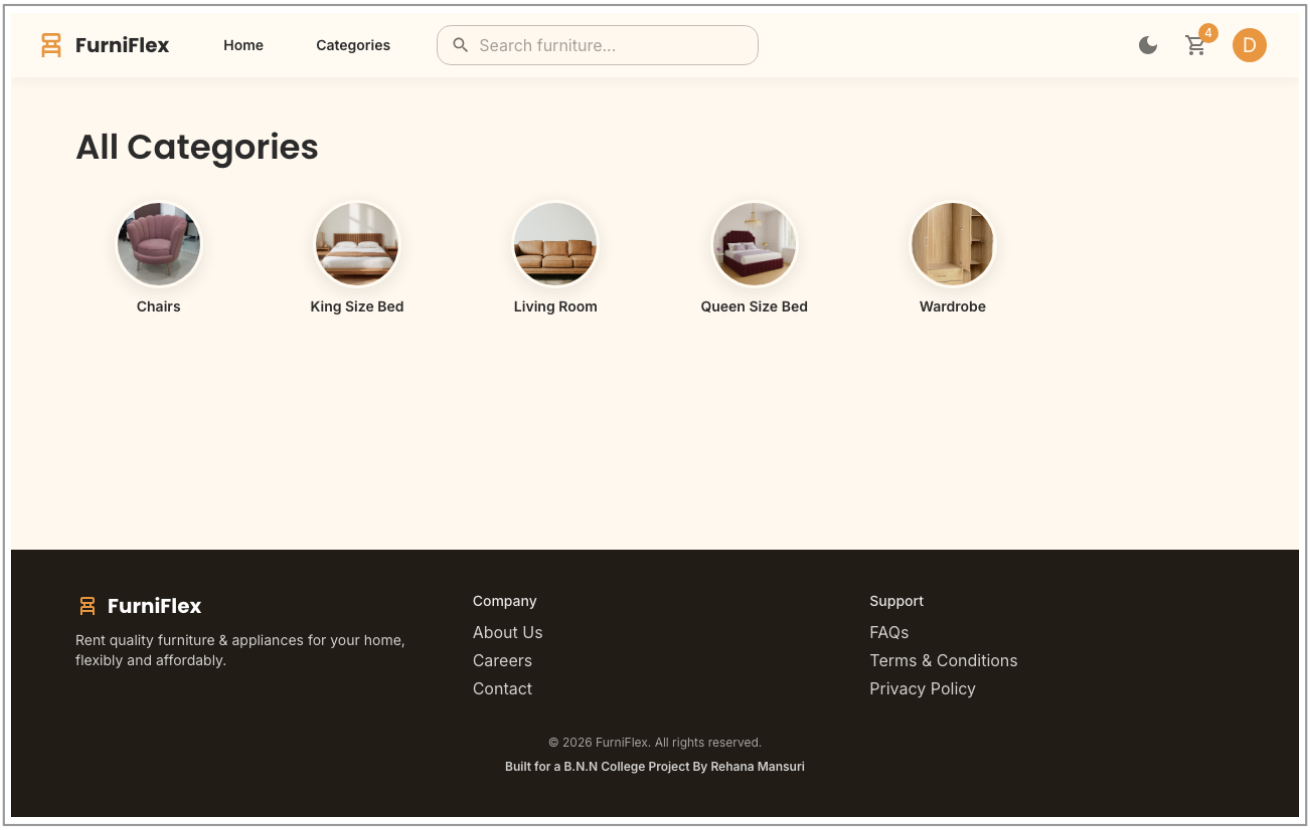
  useEffect(() => {
    categoryService.list().then(setCategories).finally(() => setLoading(false));
  }, []);

  if (loading) return <LoadingSpinner minHeight="70vh" />;

  return (
    <Container maxWidth="lg" sx={{ py: 6 }}>
      <Typography variant="h4" sx={{ mb: 4 }}>
        All Categories
      </Typography>
      <Grid container spacing={4}>
        {categories.map((category) => (
          <Grid key={category.id} size={{ xs: 4, sm: 3, md: 2 }}>
```

```
            <CategoryTile category={category} />
          </Grid>
        ))}
      </Grid>
    </Container>
  );
}
```

Output:



➢ CategoryPage.jsx (Category Products Page)

Path: frontend/src/pages/CategoryPage.jsx

Code:

```
import { useEffect, useMemo, useState } from "react";
import { useParams } from "react-router-dom";
import {
  Container,
  Grid,
  MenuItem,
  Stack,
  TextField,
  Typography,
} from "@mui/material";

import { categoryService } from "../services/categoryService";
import { productService } from "../services/productService";
import ProductCard from "../components/product/ProductCard";
import SearchBar from "../components/common/SearchBar";
import LoadingSpinner from "../components/common/LoadingSpinner";

export default function CategoryPage() {
  const { slug } = useParams();
  const [category, setCategory] = useState(null);
  const [products, setProducts] = useState([]);
  const [loading, setLoading] = useState(true);
  const [search, setSearch] = useState("");
  const [sort, setSort] = useState("newest");
  const [minPrice, setMinPrice] = useState("");
  const [maxPrice, setMaxPrice] = useState("");

  useEffect(() => {
    setLoading(true);
    categoryService
      .list()
      .then((categories) => {
        const found = categories.find((c) => c.slug === slug);
        setCategory(found || null);
        return productService.list({
          category_slug: slug,
          sort,
          min_price: minPrice || undefined,
          max_price: maxPrice || undefined,
        });
      })
      .then(setProducts)
      .finally(() => setLoading(false));
  }, [slug, sort, minPrice, maxPrice]);

  const filtered = useMemo(() => {
    if (!search.trim()) return products;
    const term = search.toLowerCase();
    return products.filter((p) => p.name.toLowerCase().includes(term));
  }, [products, search]);

  if (loading) return <LoadingSpinner minHeight="70vh" />;

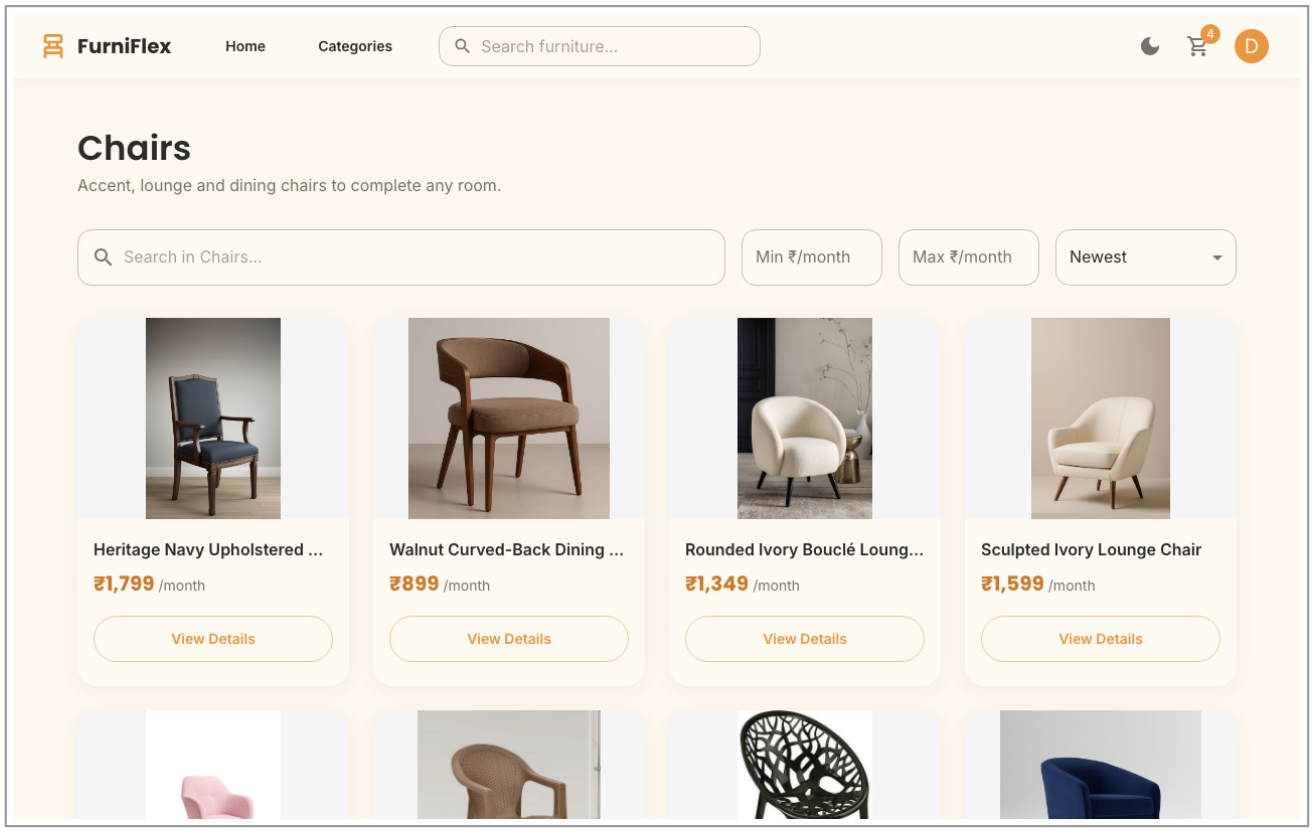
  return (
    <Container maxWidth="lg" sx={{ py: 6 }}>
      <Typography variant="h4" sx={{ mb: 0.5 }}>
        {category ? category.name : "Products"}
```

```
      </Typography>
      {category?.description && (
        <Typography variant="body1" color="text.secondary" sx={{ mb: 4 }}>
          {category.description}
        </Typography>
      )}

      <Stack direction={{ xs: "column", sm: "row" }} spacing={2} sx={{ mb: 4 }}>
        <SearchBar value={search} onChange={setSearch} placeholder={`Search in
            ${category?.name || "products"}...`} />
        <TextField
          type="number"
          label="Min ₹/month"
          value={minPrice}
          onChange={(e) => setMinPrice(e.target.value)}
          sx={{ minWidth: 140, bgcolor: "background.paper" }}
        />
        <TextField
          type="number"
          label="Max ₹/month"
          value={maxPrice}
          onChange={(e) => setMaxPrice(e.target.value)}
          sx={{ minWidth: 140, bgcolor: "background.paper" }}
        />
        <TextField
          select
          value={sort}
          onChange={(e) => setSort(e.target.value)}
          sx={{ minWidth: 180, bgcolor: "background.paper" }}
        >
          <MenuItem value="newest">Newest</MenuItem>
          <MenuItem value="price_asc">Price: Low to High</MenuItem>
          <MenuItem value="price_desc">Price: High to Low</MenuItem>
        </TextField>
      </Stack>

      {filtered.length === 0 ? (
        <Typography color="text.secondary">No products found.</Typography>
      ) : (
        <Grid container spacing={3}>
          {filtered.map((product) => (
            <Grid key={product.id} size={{ xs: 12, sm: 6, md: 3 }}>
              <ProductCard product={product} />
            </Grid>
          ))}
        </Grid>
      )}
    </Container>
  );
}
```

Output:



➢ SearchResults.jsx (Search Results Page)

Path: frontend/src/pages/SearchResults.jsx

Code:

```
import { useEffect, useState } from "react";
import { useSearchParams } from "react-router-dom";
import { Container, Grid, MenuItem, Stack, TextField, Typography } from "@mui/material";

import { categoryService } from "../services/categoryService";
import { productService } from "../services/productService";
import ProductCard from "../components/product/ProductCard";
import SearchBar from "../components/common/SearchBar";
import LoadingSpinner from "../components/common/LoadingSpinner";

export default function SearchResults() {
  const [searchParams, setSearchParams] = useSearchParams();
  const query = searchParams.get("q") || "";

  const [categories, setCategories] = useState([]);
  const [products, setProducts] = useState([]);
  const [loading, setLoading] = useState(true);
  const [categoryId, setCategoryId] = useState("");
  const [sort, setSort] = useState("newest");
  const [minPrice, setMinPrice] = useState("");
  const [maxPrice, setMaxPrice] = useState("");

  useEffect(() => {
    categoryService.list().then(setCategories);
  }, []);
```

```
useEffect(() => {
  setLoading(true);
  productService
    .list({
      search: query || undefined,
      category_id: categoryId || undefined,
      sort,
      min_price: minPrice || undefined,
      max_price: maxPrice || undefined,
    })
    .then(setProducts)
    .finally(() => setLoading(false));
}, [query, categoryId, sort, minPrice, maxPrice]);
const handleQueryChange = (value) => {
  setSearchParams(value ? { q: value } : {});
};

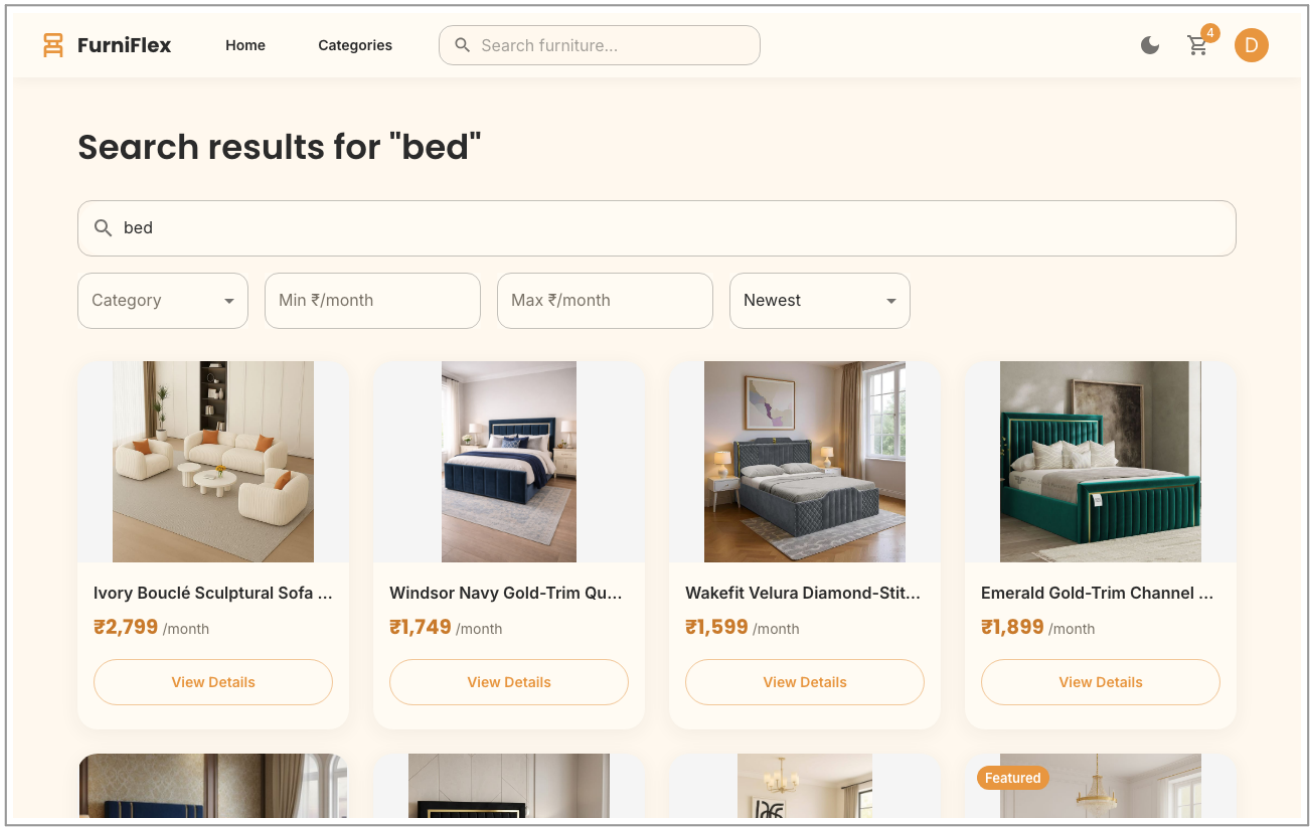
return (
  <Container maxWidth="lg" sx={{ py: 6 }}>
    <Typography variant="h4" sx={{ mb: 4 }}>
      {query ? `Search results for "${query}"` : "Search Furniture"}
    </Typography>

    <Stack direction={{ xs: "column", sm: "row" }} spacing={2} sx={{ mb: 4 }}
        flexWrap="wrap" useFlexGap>
      <SearchBar value={query} onChange={handleQueryChange} placeholder="Search all
          furniture..." />
      <TextField
        select
        label="Category"
        value={categoryId}
        onChange={(e) => setCategoryId(e.target.value)}
        sx={{ minWidth: 170, bgcolor: "background.paper" }}
      >
        <MenuItem value="">All Categories</MenuItem>
        {categories.map((c) => (
          <MenuItem key={c.id} value={c.id}>
            {c.name}
          </MenuItem>
        ))}
      </TextField>
      <TextField
        type="number"
        label="Min ₹/month"
        value={minPrice}
        onChange={(e) => setMinPrice(e.target.value)}
        sx={{ minWidth: 140, bgcolor: "background.paper" }}
      />
      <TextField
        type="number"
        label="Max ₹/month"
        value={maxPrice}
        onChange={(e) => setMaxPrice(e.target.value)}
        sx={{ minWidth: 140, bgcolor: "background.paper" }}
      />
      <TextField
        select
        value={sort}
        onChange={(e) => setSort(e.target.value)}
        sx={{ minWidth: 180, bgcolor: "background.paper" }}
      >
```

```
            <MenuItem value="newest">Newest</MenuItem>
            <MenuItem value="price_asc">Price: Low to High</MenuItem>
            <MenuItem value="price_desc">Price: High to Low</MenuItem>
          </TextField>
        </Stack>

        {loading ? (
          <LoadingSpinner minHeight="40vh" />
        ) : products.length === 0 ? (
          <Typography color="text.secondary">No products found. Try a different search or
              filter.</Typography>
        ) : (
          <Grid container spacing={3}>
            {products.map((product) => (
              <Grid key={product.id} size={{ xs: 12, sm: 6, md: 3 }}>
                <ProductCard product={product} />
              </Grid>
            ))}
          </Grid>
        )}
      </Container>
    );
}
```

Output:



➢ ProductDetails.jsx (Product Details Page)

Path: frontend/src/pages/ProductDetails.jsx

Code:

```
import { useEffect, useState } from "react";
import { useNavigate, useParams } from "react-router-dom";
import {
  Box,
  Button,
  Container,
  Grid,
  IconButton,
  Snackbar,
  Alert,
  Stack,
  Typography,
} from "@mui/material";
import RemoveIcon from "@mui/icons-material/Remove";
import AddIcon from "@mui/icons-material/Add";

import { productService, rentalPlanService } from "../services/productService";
import { resolveImageUrl } from "../services/api";
import RentalPlanSelector from "../components/product/RentalPlanSelector";
import LoadingSpinner from "../components/common/LoadingSpinner";
import { useAuth } from "../context/AuthContext";
import { useCart } from "../context/CartContext";

const currency = new Intl.NumberFormat("en-IN", {
  style: "currency",
  currency: "INR",
  maximumFractionDigits: 0,
});

export default function ProductDetails() {
  const { id } = useParams();
  const navigate = useNavigate();
  const { user } = useAuth();
  const { addToCart } = useCart();

  const [product, setProduct] = useState(null);
  const [plans, setPlans] = useState([]);
  const [selectedImage, setSelectedImage] = useState(0);
  const [selectedPlanId, setSelectedPlanId] = useState(null);
  const [quantity, setQuantity] = useState(1);
  const [loading, setLoading] = useState(true);
  const [adding, setAdding] = useState(false);
  const [toast, setToast] = useState(false);

  useEffect(() => {
    setLoading(true);
    Promise.all([productService.get(id), rentalPlanService.list()])
      .then(([productData, planData]) => {
        setProduct(productData);
        setPlans(planData);
        setSelectedPlanId(planData[0]?.id);
      })
      .finally(() => setLoading(false));
  }, [id]);

  if (loading) return <LoadingSpinner minHeight="70vh" />;
  if (!product) return null;
```

```
const handleAddToCart = async () => {
  if (!user) {
    navigate("/login", { state: { from: { pathname: `/products/${id}` } } });
    return;
  }
  setAdding(true);
  try {
    await addToCart(product.id, selectedPlanId, quantity);
    setToast(true);
  } finally {
    setAdding(false);
  }
};

return (
  <Container maxWidth="lg" sx={{ py: 6 }}>
    <Grid container spacing={5}>
      <Grid size={{ xs: 12, md: 6 }}>
        <Box
          component="img"
          src={resolveImageUrl(product.image_urls?.[selectedImage])}
          alt={product.name}
          sx={{ width: "100%", height: 420, objectFit: "contain", borderRadius: 5,
              bgcolor: "grey.100" }}
        />
        {product.image_urls?.length > 1 && (
          <Stack direction="row" spacing={1.5} sx={{ mt: 2 }}>
            {product.image_urls.map((url, idx) => (
              <Box
                key={url}
                component="img"
                src={resolveImageUrl(url)}
                onClick={() => setSelectedImage(idx)}
                sx={{
                  width: 72,
                  height: 72,
                  objectFit: "cover",
                  borderRadius: 2,
                  cursor: "pointer",
                  border: "2px solid",
                  borderColor: idx === selectedImage ? "primary.main" : "transparent",
                }}
              />
            ))}
          </Stack>
        )}
      </Grid>

      <Grid size={{ xs: 12, md: 6 }}>
        {product.category && (
          <Typography variant="overline" color="primary.dark" fontWeight={700}>
            {product.category.name}
          </Typography>
        )}
        <Typography variant="h4" sx={{ mb: 2 }}>
          {product.name}
        </Typography>

        <Stack direction="row" spacing={4} sx={{ mb: 3 }}>
          <Box>
            <Typography variant="caption" color="text.secondary">
```

```
            Security Deposit
          </Typography>
          <Typography variant="h6" fontWeight={700}>
            {currency.format(product.deposit)} <Typography component="span"
                variant="body2">(Refundable)</Typography>
          </Typography>
        </Box>
      </Stack>

      <Typography variant="subtitle2" sx={{ mb: 1.5 }}>
        Choose a rental plan
      </Typography>
      <RentalPlanSelector
        plans={plans}
        product={product}
        selectedPlanId={selectedPlanId}
        onChange={setSelectedPlanId}
      />

      <Typography variant="subtitle2" sx={{ mt: 4, mb: 1.5 }}>
        Quantity
      </Typography>
      <Stack direction="row" alignItems="center" spacing={2}>
        <IconButton
          onClick={() => setQuantity((q) => Math.max(1, q - 1))}
          sx={{ border: "1px solid", borderColor: "divider" }}
        >
          <RemoveIcon fontSize="small" />
        </IconButton>
        <Typography variant="h6" sx={{ minWidth: 24, textAlign: "center" }}>
          {quantity}
        </Typography>
        <IconButton
          onClick={() => setQuantity((q) => Math.min(product.stock_qty, q + 1))}
          sx={{ border: "1px solid", borderColor: "divider" }}
        >
          <AddIcon fontSize="small" />
        </IconButton>
      </Stack>

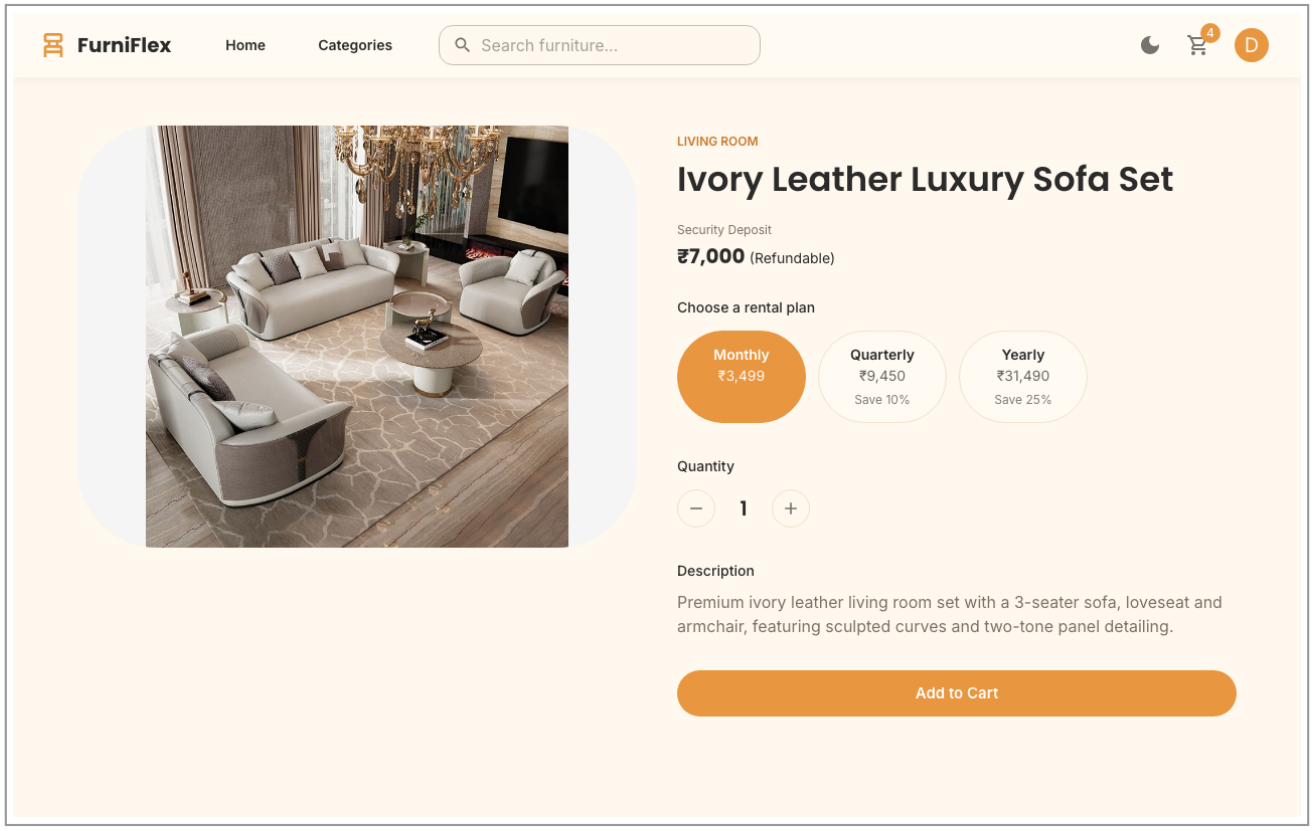
      <Typography variant="subtitle2" sx={{ mt: 4, mb: 1 }}>
        Description
      </Typography>
      <Typography variant="body1" color="text.secondary" sx={{ mb: 4 }}>
        {product.description}
      </Typography>

      <Button
        variant="contained"
        color="primary"
        size="large"
        fullWidth
        disabled={adding}
        onClick={handleAddToCart}
      >
        {adding ? "Adding..." : "Add to Cart"}
      </Button>
    </Grid>
  </Grid>

  <Snackbar open={toast} autoHideDuration={2500} onClose={() => setToast(false)}>
```

```
        <Alert severity="success" variant="filled" onClose={() => setToast(false)}>
          Added to cart!
        </Alert>
      </Snackbar>
    </Container>
  );
}
```

Output:



➢ Cart.jsx (Cart Page)

Path: frontend/src/pages/Cart.jsx

Code:

```
import { useEffect, useState } from "react";
import { Link as RouterLink, useNavigate } from "react-router-dom";
import {
  Box,
  Button,
  Container,
  Divider,
  Grid,
  IconButton,
  MenuItem,
  Paper,
  Stack,
  TextField,
  Typography,
} from "@mui/material";
import DeleteOutlineIcon from "@mui/icons-material/DeleteOutlineOutlined";
import RemoveIcon from "@mui/icons-material/Remove";
import AddIcon from "@mui/icons-material/Add";

import { useCart } from "../context/CartContext";
import { rentalPlanService } from "../services/productService";
import { resolveImageUrl } from "../services/api";
import LoadingSpinner from "../components/common/LoadingSpinner";

const currency = new Intl.NumberFormat("en-IN", {
  style: "currency",
  currency: "INR",
  maximumFractionDigits: 0,
});

export default function Cart() {
  const { items, loading, updateItem, removeItem, priceFor, total } = useCart();
  const navigate = useNavigate();
  const [plans, setPlans] = useState([]);

  useEffect(() => {
    rentalPlanService.list().then(setPlans);
  }, []);

  if (loading) return <LoadingSpinner minHeight="70vh" />;
  if (items.length === 0) {
    return (
      <Container maxWidth="sm" sx={{ py: 10, textAlign: "center" }}>
        <Typography variant="h5" sx={{ mb: 1 }}>
          Your cart is empty
        </Typography>
        <Typography color="text.secondary" sx={{ mb: 3 }}>
          Browse our categories and add furniture you'd love to rent.
        </Typography>
        <Button component={RouterLink} to="/categories" variant="contained" color="primary">
          Browse Furniture
        </Button>
      </Container>
    );
  }

  return (
```

```
<Container maxWidth="lg" sx={{ py: 6 }}>
  <Typography variant="h4" sx={{ mb: 4 }}>
    Your Cart
  </Typography>
  <Grid container spacing={4}>
    <Grid size={{ xs: 12, md: 8 }}>
      <Stack spacing={2}>
        {items.map((item) => (
          <Paper key={item.id} sx={{ p: 2 }}>
            <Stack direction={{ xs: "column", sm: "row" }} spacing={2} alignItems={{
                sm: "center" }}>
              <Box
                component="img"
                src={resolveImageUrl(item.product.image_urls?.[0])}
                alt={item.product.name}
                sx={{ width: 100, height: 100, objectFit: "contain", borderRadius: 3,
                    bgcolor: "grey.100" }}
              />
              <Box sx={{ flexGrow: 1 }}>
                <Typography variant="subtitle1" fontWeight={600}>
                  {item.product.name}
                </Typography>
                <Typography variant="body2" color="text.secondary">
                  {currency.format(priceFor(item) / item.quantity)} /
                      {item.rental_plan.name.toLowerCase()}
                </Typography>
              </Box>

              <TextField
                select
                size="small"
                label="Plan"
                value={item.rental_plan.id}
                onChange={(e) => updateItem(item.id, { rental_plan_id:
                    Number(e.target.value) })}
                sx={{ minWidth: 130 }}
              >
                {plans.map((plan) => (
                  <MenuItem key={plan.id} value={plan.id}>
                    {plan.name}
                  </MenuItem>
                ))}
              </TextField>

              <Stack direction="row" alignItems="center" spacing={1}>
                <IconButton
                  size="small"
                  sx={{ border: "1px solid", borderColor: "divider" }}
                  onClick={() => updateItem(item.id, { quantity: Math.max(1,
                      item.quantity - 1) })}
                >
                  <RemoveIcon fontSize="small" />
                </IconButton>
                <Typography sx={{ minWidth: 20, textAlign: "center"
                    }}>{item.quantity}</Typography>
                <IconButton
                  size="small"
                  sx={{ border: "1px solid", borderColor: "divider" }}
                  onClick={() => updateItem(item.id, { quantity: item.quantity + 1 })}
                >
                  <AddIcon fontSize="small" />
                </IconButton>
```

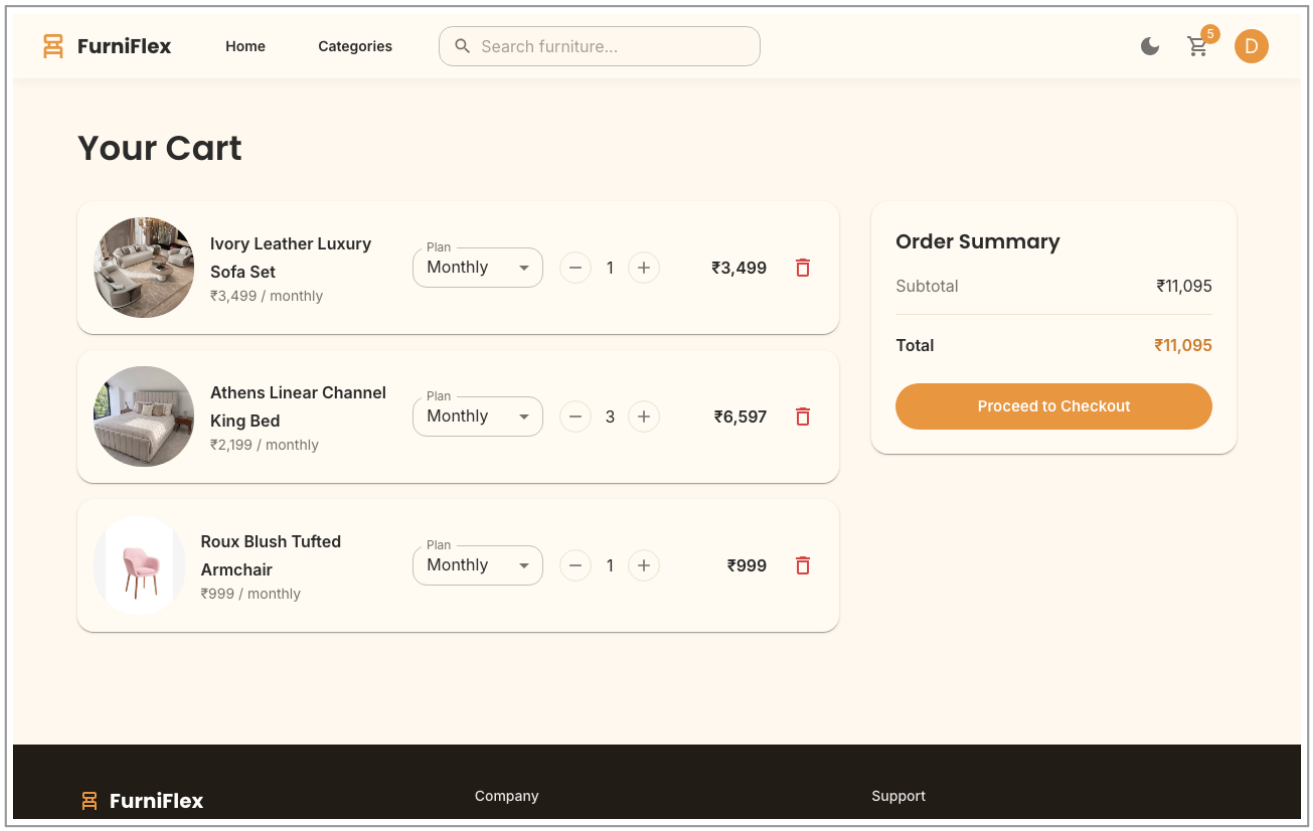
```
                </Stack>

                <Typography variant="subtitle1" fontWeight={700} sx={{ minWidth: 90,
                    textAlign: "right" }}>
                  {currency.format(priceFor(item))}
                </Typography>

                <IconButton color="error" onClick={() => removeItem(item.id)}>
                  <DeleteOutlineIcon />
                </IconButton>
              </Stack>
            </Paper>
          ))}
        </Stack>
      </Grid>

      <Grid size={{ xs: 12, md: 4 }}>
        <Paper sx={{ p: 3 }}>
          <Typography variant="h6" sx={{ mb: 2 }}>
            Order Summary
          </Typography>
          <Stack direction="row" justifyContent="space-between" sx={{ mb: 1 }}>
            <Typography color="text.secondary">Subtotal</Typography>
            <Typography>{currency.format(total)}</Typography>
          </Stack>
          <Divider sx={{ my: 2 }} />
          <Stack direction="row" justifyContent="space-between" sx={{ mb: 3 }}>
            <Typography variant="subtitle1" fontWeight={700}>
              Total
            </Typography>
            <Typography variant="subtitle1" fontWeight={700} color="primary.dark">
              {currency.format(total)}
            </Typography>
          </Stack>
          <Button fullWidth variant="contained" color="primary" size="large" onClick={()
              => navigate("/checkout")}>
            Proceed to Checkout
          </Button>
        </Paper>
      </Grid>
    </Grid>
  </Container>
 );
}
```

Output:



➢ Checkout.jsx (Checkout Page)

Path: frontend/src/pages/Checkout.jsx

Code:

```
import { useState } from "react";
import { useNavigate } from "react-router-dom";
import {
  Alert,
  Box,
  Button,
  Container,
  Divider,
  Grid,
  Paper,
  Stack,
  TextField,
  Typography,
} from "@mui/material";

import { useAuth } from "../context/AuthContext";
import { useCart } from "../context/CartContext";
import { orderService } from "../services/orderService";
import PaymentMethodSelector from "../components/checkout/PaymentMethodSelector";
const currency = new Intl.NumberFormat("en-IN", {
  style: "currency",
  currency: "INR",
  maximumFractionDigits: 0,
});
```

```
export default function Checkout() {
  const { user } = useAuth();
  const { items, priceFor, total, refresh } = useCart();
  const navigate = useNavigate();

  const [address, setAddress] = useState(user?.address || "");
  const [placing, setPlacing] = useState(false);
  const [error, setError] = useState("");

  const [paymentMethod, setPaymentMethod] = useState("CARD");
  const [card, setCard] = useState({ number: "", name: "", expiry: "", cvv: "" });
  const [upiId, setUpiId] = useState("");

  const handlePlaceOrder = async () => {
    if (!address.trim()) {
      setError("Please enter a shipping address.");
      return;
    }
    if (paymentMethod === "CARD" && (!card.number.trim() || !card.expiry.trim() ||
        !card.cvv.trim())) {
      setError("Please fill in your card details.");
      return;
    }
    if (paymentMethod === "UPI" && !upiId.trim()) {
      setError("Please enter your UPI ID.");
      return;
    }

    setError("");
    setPlacing(true);
    try {
      const paymentReference =
        paymentMethod === "CARD"
          ? `Card ending ${card.number.replace(/\s/g, "").slice(-4)}`
          : paymentMethod === "UPI"
          ? upiId
          : "GPay QR";

      const order = await orderService.checkout({
        shipping_address: address,
        payment_method: paymentMethod,
        payment_reference: paymentReference,
      });
      await refresh();
      navigate("/orders", { state: { placedOrderId: order.id } });
    } catch {
      setError("Could not place order. Please try again.");
    } finally {
      setPlacing(false);
    }
  };

  if (items.length === 0) {
    return (
      <Container maxWidth="sm" sx={{ py: 10, textAlign: "center" }}>
        <Typography variant="h5">Your cart is empty</Typography>
      </Container>
    );
  }

  return (
    <Container maxWidth="lg" sx={{ py: 6 }}>
```

```
<Typography variant="h4" sx={{ mb: 4 }}>
  Checkout
</Typography>
<Grid container spacing={4}>
  <Grid size={{ xs: 12, md: 7 }}>
    <Paper sx={{ p: 3, mb: 3 }}>
      <Typography variant="h6" sx={{ mb: 2 }}>
        Shipping Address
      </Typography>
      <TextField
        fullWidth
        multiline
        minRows={3}
        placeholder="House no., street, city, state, PIN code"
        value={address}
        onChange={(e) => setAddress(e.target.value)}
      />
    </Paper>

    <Paper sx={{ p: 3, mb: 3 }}>
      <Typography variant="h6" sx={{ mb: 2 }}>
        Payment Method
      </Typography>
      <PaymentMethodSelector
        method={paymentMethod}
        onMethodChange={setPaymentMethod}
        card={card}
        onCardChange={setCard}
        upiId={upiId}
        onUpiIdChange={setUpiId}
      />
    </Paper>
    <Paper sx={{ p: 3 }}>
      <Typography variant="h6" sx={{ mb: 2 }}>
        Order Items
      </Typography>
      <Stack spacing={2} divider={<Divider />}>
        {items.map((item) => (
          <Stack key={item.id} direction="row" justifyContent="space-between"
              alignItems="center">
            <Box>
              <Typography fontWeight={600}>{item.product.name}</Typography>
              <Typography variant="body2" color="text.secondary">
                {item.rental_plan.name} × {item.quantity}
              </Typography>
            </Box>
            <Typography fontWeight={700}>{currency.format(priceFor(item))}</Typography
                >
          </Stack>
        ))}
      </Stack>
    </Paper>
  </Grid>

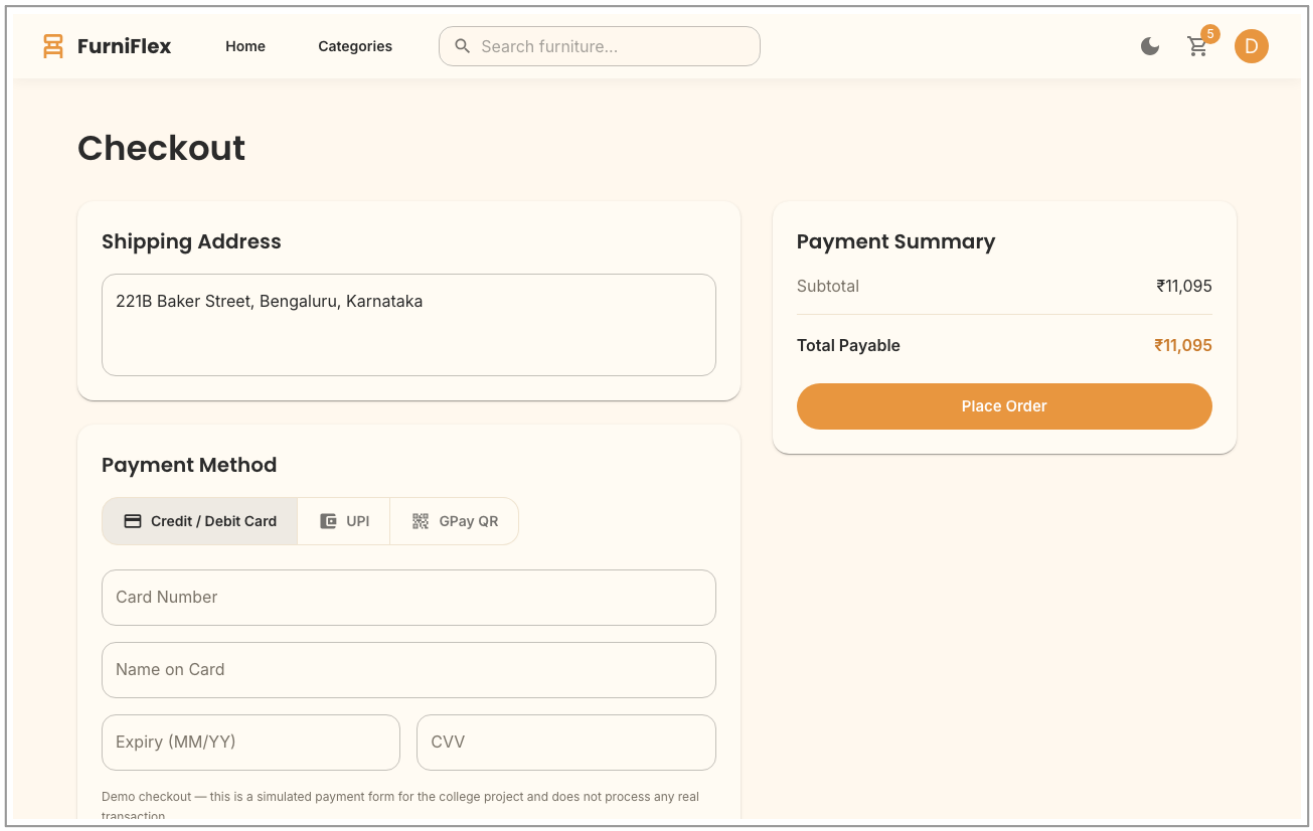
  <Grid size={{ xs: 12, md: 5 }}>
    <Paper sx={{ p: 3 }}>
      <Typography variant="h6" sx={{ mb: 2 }}>
        Payment Summary
      </Typography>
      <Stack direction="row" justifyContent="space-between" sx={{ mb: 1 }}>
        <Typography color="text.secondary">Subtotal</Typography>
        <Typography>{currency.format(total)}</Typography>
```

```
            </Stack>
            <Divider sx={{ my: 2 }} />
            <Stack direction="row" justifyContent="space-between" sx={{ mb: 3 }}>
              <Typography variant="subtitle1" fontWeight={700}>
                Total Payable
              </Typography>
              <Typography variant="subtitle1" fontWeight={700} color="primary.dark">
                {currency.format(total)}
              </Typography>
            </Stack>

            {error && (
              <Alert severity="error" sx={{ mb: 2 }}>
                {error}
              </Alert>
            )}

            <Button
              fullWidth
              variant="contained"
              color="primary"
              size="large"
              disabled={placing}
              onClick={handlePlaceOrder}
            >
              {placing ? "Placing Order..." : "Place Order"}
            </Button>
          </Paper>
        </Grid>
      </Grid>
    </Container>
  );
}
```

Output:



➢ OrderHistory.jsx (Order History Page)

Path: frontend/src/pages/OrderHistory.jsx

Code:

```
import { useEffect, useState } from "react";
import {
  Box,
  Chip,
  Container,
  Divider,
  Paper,
  Stack,
  Typography,
} from "@mui/material";

import { orderService } from "../services/orderService";
import LoadingSpinner from "../components/common/LoadingSpinner";

const currency = new Intl.NumberFormat("en-IN", {
  style: "currency",
  currency: "INR",
  maximumFractionDigits: 0,
});
const STATUS_COLORS = {
  PLACED: "info",
  CONFIRMED: "primary",
  DELIVERED: "success",
  RETURNED: "default",
  CANCELLED: "error",
```

```
};

const PAYMENT_LABELS = {
  CARD: "Card",
  UPI: "UPI",
  GPAY_QR: "GPay QR",
};

export default function OrderHistory() {
  const [orders, setOrders] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    orderService.myOrders().then(setOrders).finally(() => setLoading(false));
  }, []);

  if (loading) return <LoadingSpinner minHeight="70vh" />;

  return (
    <Container maxWidth="md" sx={{ py: 6 }}>
      <Typography variant="h4" sx={{ mb: 4 }}>
        Order History
      </Typography>

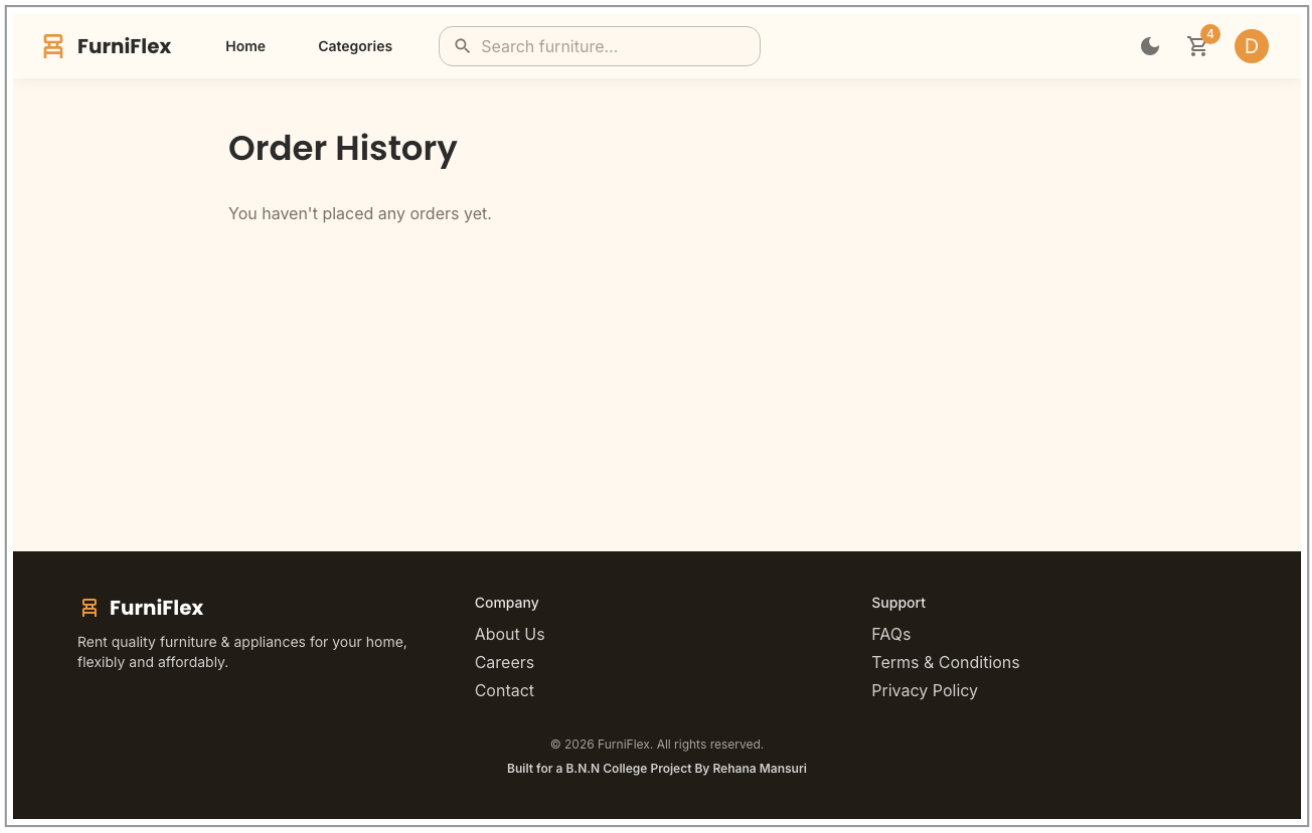
      {orders.length === 0 ? (
        <Typography color="text.secondary">You haven't placed any orders yet.</Typography>
      ) : (
        <Stack spacing={3}>
          {orders.map((order) => (
            <Paper key={order.id} sx={{ p: 3 }}>
              <Stack direction="row" justifyContent="space-between" alignItems="center"
                  sx={{ mb: 2 }}>
                <Box>
                  <Typography variant="subtitle1" fontWeight={700}>
                    Order #{order.id}
                  </Typography>
                  <Typography variant="body2" color="text.secondary">
                    {new Date(order.created_at).toLocaleDateString("en-IN", {
                      year: "numeric",
                      month: "long",
                      day: "numeric",
                    })}
                  </Typography>
                </Box>
                <Chip
                  label={order.status}
                  color={STATUS_COLORS[order.status] || "default"}
                  size="small"
                />
              </Stack>

              <Stack spacing={1.5} divider={<Divider />}>
                {order.items.map((item) => (
                  <Stack key={item.id} direction="row" justifyContent="space-between">
                    <Box>
                      <Typography fontWeight={600}>{item.product.name}</Typography>
                      <Typography variant="body2" color="text.secondary">
                        {item.rental_plan.name} × {item.quantity}
                      </Typography>
                    </Box>
                    <Typography fontWeight={600}>
                      {currency.format(item.price_at_order * item.quantity)}
```

```
                    </Typography>
                  </Stack>
                ))}
              </Stack>

              <Divider sx={{ my: 2 }} />
              <Stack direction="row" justifyContent="space-between">
                <Typography fontWeight={700}>Total</Typography>
                <Typography fontWeight={700} color="primary.dark">
                  {currency.format(order.total_amount)}
                </Typography>
              </Stack>
              <Typography variant="body2" color="text.secondary" sx={{ mt: 1 }}>
                Shipping to: {order.shipping_address}
              </Typography>
              <Typography variant="body2" color="text.secondary">
                Paid via {PAYMENT_LABELS[order.payment_method] || order.payment_method}
                {order.payment_reference ? ` · ${order.payment_reference}` : ""}
              </Typography>
            </Paper>
          ))}
        </Stack>
      )}
    </Container>
  );
}
```

Output:



➢ Profile.jsx (Profile Page)

Path: frontend/src/pages/Profile.jsx

Code:

```
import { useState } from "react";
import {
  Alert,
  Avatar,
  Button,
  Container,
  Grid,
  Paper,
  Stack,
  TextField,
  Typography,
} from "@mui/material";

import { useAuth } from "../context/AuthContext";

export default function Profile() {
  const { user, updateProfile, logout } = useAuth();
  const [form, setForm] = useState({
    name: user?.name || "",
    phone: user?.phone || "",
    address: user?.address || "",
  });
  const [saving, setSaving] = useState(false);
  const [saved, setSaved] = useState(false);

  const handleChange = (field) => (e) => setForm({ ...form, [field]: e.target.value });

  const handleSave = async () => {
    setSaving(true);
    setSaved(false);
    try {
      await updateProfile(form);
      setSaved(true);
    } finally {
      setSaving(false);
    }
  };

  return (
    <Container maxWidth="sm" sx={{ py: 6 }}>
      <Stack alignItems="center" spacing={1} sx={{ mb: 4 }}>
        <Avatar sx={{ width: 72, height: 72, bgcolor: "primary.main", fontSize: 28 }}>
          {user?.name?.[0]?.toUpperCase() || "U"}
        </Avatar>
        <Typography variant="h5">{user?.name}</Typography>
        <Typography variant="body2" color="text.secondary">
          {user?.email}
        </Typography>
      </Stack>

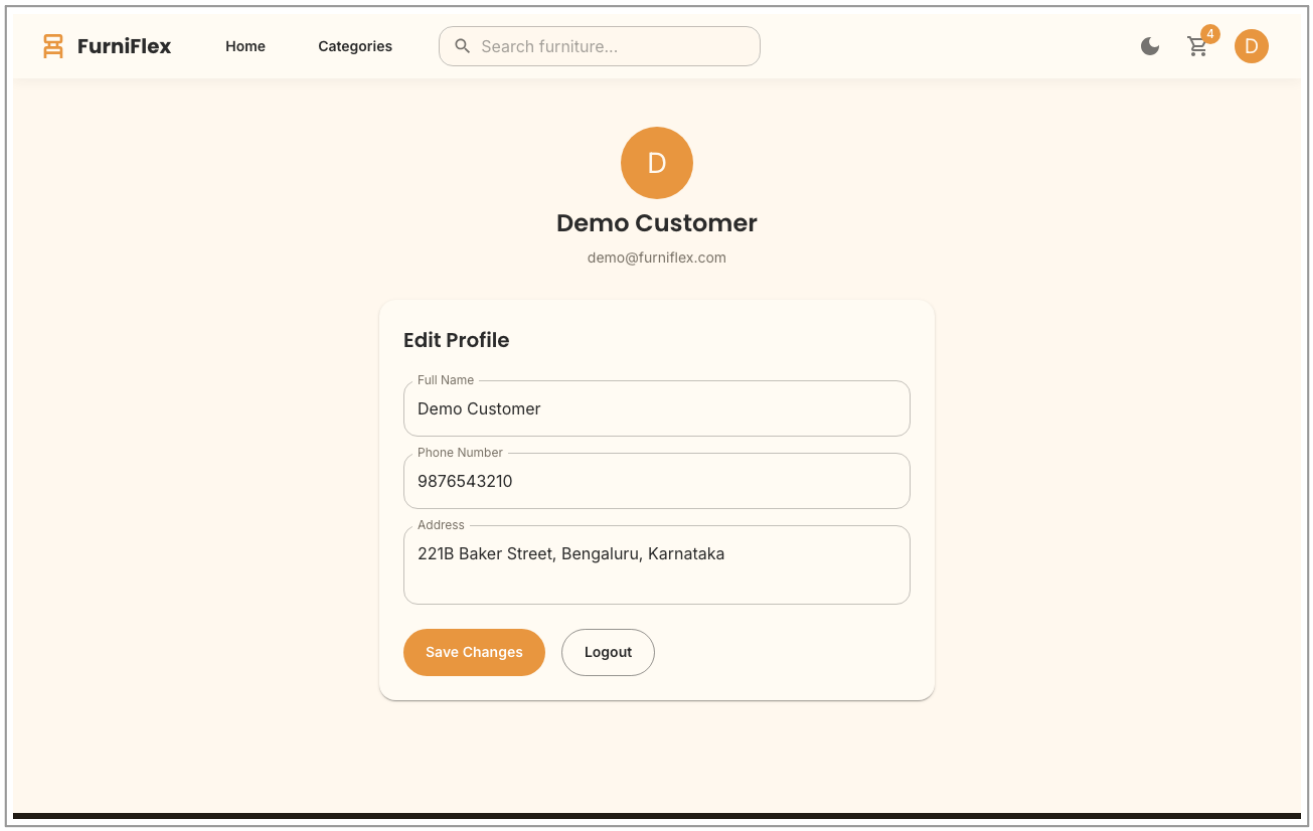
      <Paper sx={{ p: 3 }}>
        <Typography variant="h6" sx={{ mb: 3 }}>
          Edit Profile
        </Typography>

        {saved && (
          <Alert severity="success" sx={{ mb: 2 }}>
```

```
            Profile updated successfully.
          </Alert>
        )}
        <Grid container spacing={2}>
          <Grid size={12}>
            <TextField fullWidth label="Full Name" value={form.name}
                onChange={handleChange("name")} />
          </Grid>
          <Grid size={12}>
            <TextField fullWidth label="Phone Number" value={form.phone}
                onChange={handleChange("phone")} />
          </Grid>
          <Grid size={12}>
            <TextField
              fullWidth
              multiline
              minRows={2}
              label="Address"
              value={form.address}
              onChange={handleChange("address")}
            />
          </Grid>
        </Grid>

        <Stack direction="row" spacing={2} sx={{ mt: 3 }}>
          <Button variant="contained" color="primary" disabled={saving}
              onClick={handleSave}>
            {saving ? "Saving..." : "Save Changes"}
          </Button>
          <Button variant="outlined" color="secondary" onClick={logout}>
            Logout
          </Button>
        </Stack>
      </Paper>
    </Container>
  );
}
```

Output:



➢ NotFound.jsx (404 Page)

Path: frontend/src/pages/NotFound.jsx

Code:

```
import { Link as RouterLink } from "react-router-dom";
import { Button, Container, Stack, Typography } from "@mui/material";

export default function NotFound() {
  return (
    <Container maxWidth="sm" sx={{ py: 12, textAlign: "center" }}>
      <Stack spacing={2} alignItems="center">
        <Typography variant="h2" color="primary.main" fontWeight={700}>
          404
        </Typography>
        <Typography variant="h6">Page not found</Typography>
        <Button component={RouterLink} to="/" variant="contained" color="primary">
          Back to Home
        </Button>
      </Stack>
    </Container>
  );
}
```

➢ AdminLayout.jsx (Admin panel layout)

Path: frontend/src/components/admin/AdminLayout.jsx

Code:

```
import { Link as RouterLink, useLocation } from "react-router-dom";
import { Box, Container, Paper, Tab, Tabs, Typography } from "@mui/material";

const TABS = [
  { label: "Dashboard", to: "/admin" },
  { label: "Products", to: "/admin/products" },
  { label: "Categories", to: "/admin/categories" },
  { label: "Orders", to: "/admin/orders" },
  { label: "Users", to: "/admin/users" },
];

export default function AdminLayout({ children }) {
  const location = useLocation();
  const currentTab = TABS.some((tab) => tab.to === location.pathname) ? location.pathname :
      "/admin";

  return (
    <Container maxWidth="lg" sx={{ py: 5 }}>
      <Typography variant="h4" sx={{ mb: 3 }}>
        Admin Panel
      </Typography>

      <Paper sx={{ mb: 4 }}>
        <Tabs
          value={currentTab}
          variant="scrollable"
          scrollButtons="auto"
          textColor="primary"
          indicatorColor="primary"
        >
          {TABS.map((tab) => (
            <Tab key={tab.to} label={tab.label} value={tab.to} component={RouterLink}
                to={tab.to} />
          ))}
        </Tabs>
      </Paper>

      <Box>{children}</Box>
    </Container>
  );
}
```

➢ ImageUploader.jsx (Admin image uploader)

Path: frontend/src/components/admin/ImageUploader.jsx

Code:

```
import { useRef, useState } from "react";
import { Alert, Box, CircularProgress, IconButton, Stack, Typography } from "@mui/material";
import AddPhotoAlternateIcon from "@mui/icons-material/AddPhotoAlternate";
import CloseIcon from "@mui/icons-material/Close";

import { uploadService } from "../../services/uploadService";
import { resolveImageUrl } from "../../services/api";

/**
 * Uploads land in backend/static/images/<categorySlug>/ so images stay
 * organized by category on disk. `multiple=false` behaves as a single
 * cover-image picker (used for category covers); `multiple=true` builds a
 * gallery array (used for product images).
 */
export default function ImageUploader({ images, onChange, categorySlug, multiple = true,
    label = "Images" }) {
  const fileInputRef = useRef(null);
  const [uploading, setUploading] = useState(false);
  const [error, setError] = useState("");
  const canUpload = Boolean(categorySlug) && !uploading;

  const handleFileChange = async (e) => {
    const file = e.target.files?.[0];
    e.target.value = "";
    if (!file) return;

    setError("");
    setUploading(true);
    try {
      const url = await uploadService.uploadImage(file, categorySlug);
      onChange(multiple ? [...images, url] : [url]);
    } catch {
      setError("Upload failed. Make sure it's an image (jpg/png/webp/gif) under 5MB.");
    } finally {
      setUploading(false);
    }
  };

  const handleRemove = (idx) => onChange(images.filter((_, i) => i !== idx));
  return (
    <Box>
      <Typography variant="subtitle2" sx={{ mb: 1 }}>
        {label}
      </Typography>
      {error && (
        <Alert severity="error" sx={{ mb: 1 }}>
          {error}
        </Alert>
      )}
      <Stack direction="row" spacing={1.5} flexWrap="wrap" useFlexGap>
        {images.map((url, idx) => (
          <Box key={url} sx={{ position: "relative" }}>
            <Box
              component="img"
              src={resolveImageUrl(url)}
              alt=""
              sx={{
```

```
            width: 84,
            height: 84,
            objectFit: "cover",
            borderRadius: 2,
            border: "1px solid",
            borderColor: "divider",
          }}
        />
        <IconButton
          size="small"
          onClick={() => handleRemove(idx)}
          sx={{
            position: "absolute",
            top: -8,
            right: -8,
            width: 22,
            height: 22,
            bgcolor: "background.paper",
            border: "1px solid",
            borderColor: "divider",
            "&:hover": { bgcolor: "error.main", color: "#fff" },
          }}
        >
          <CloseIcon sx={{ fontSize: 14 }} />
        </IconButton>
      </Box>
    ))}

    {(multiple || images.length === 0) && (
      <Box
        onClick={() => canUpload && fileInputRef.current?.click()}
        sx={{
          width: 84,
          height: 84,
          borderRadius: 2,
          border: "2px dashed",
          borderColor: "divider",
          display: "flex",
          alignItems: "center",
          justifyContent: "center",
          cursor: canUpload ? "pointer" : "not-allowed",
          opacity: categorySlug ? 1 : 0.5,
          color: "text.secondary",
        }}
      >
        {uploading ? <CircularProgress size={22} /> : <AddPhotoAlternateIcon />}
      </Box>
    )}
  </Stack>

  <input
    ref={fileInputRef}
    type="file"
    accept="image/png,image/jpeg,image/webp,image/gif"
    hidden
    onChange={handleFileChange}
  />

  {!categorySlug && (
    <Typography variant="caption" color="text.secondary" sx={{ mt: 0.5, display:
        "block" }}>
```

```
          {multiple ? "Select a category first to upload images." : "Enter a slug first to
              upload an image."}
        </Typography>
      )}
    </Box>
  );
}
```

➢ AdminDashboard.jsx (Admin Dashboard)

Path: frontend/src/pages/admin/AdminDashboard.jsx

Code:

```
import { useEffect, useState } from "react";
import { Grid, Paper, Typography } from "@mui/material";

import AdminLayout from "../../components/admin/AdminLayout";
import LoadingSpinner from "../../components/common/LoadingSpinner";
import { adminService } from "../../services/orderService";

const currency = new Intl.NumberFormat("en-IN", {
  style: "currency",
  currency: "INR",
  maximumFractionDigits: 0,
});

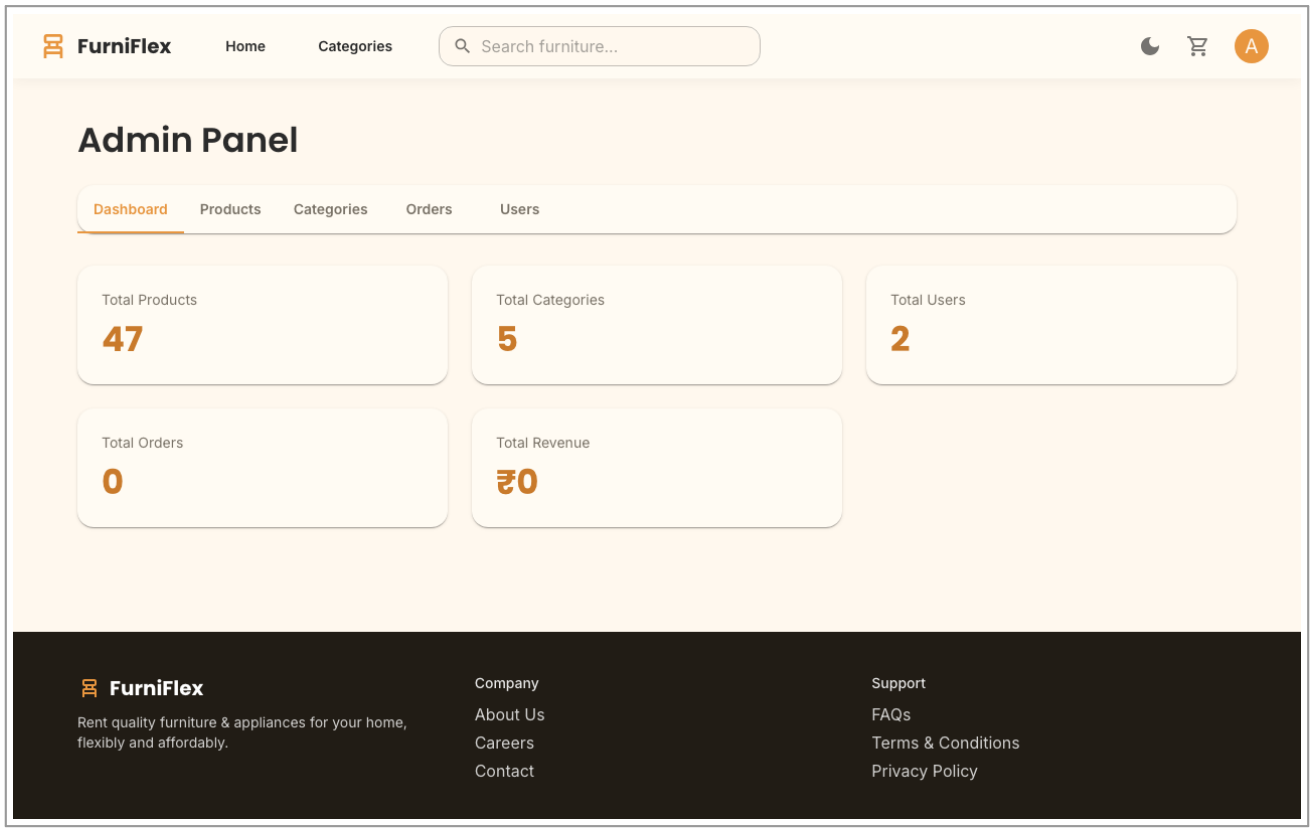
export default function AdminDashboard() {
  const [stats, setStats] = useState(null);

  useEffect(() => {
    adminService.dashboard().then(setStats);
  }, []);
  if (!stats) return <LoadingSpinner minHeight="40vh" />;

  const cards = [
    { label: "Total Products", value: stats.total_products },
    { label: "Total Categories", value: stats.total_categories },
    { label: "Total Users", value: stats.total_users },
    { label: "Total Orders", value: stats.total_orders },
    { label: "Total Revenue", value: currency.format(stats.revenue) },
  ];

  return (
    <AdminLayout>
      <Grid container spacing={3}>
        {cards.map((card) => (
          <Grid key={card.label} size={{ xs: 12, sm: 6, md: 4 }}>
            <Paper sx={{ p: 3 }}>
              <Typography variant="body2" color="text.secondary">
                {card.label}
              </Typography>
              <Typography variant="h4" fontWeight={700} color="primary.dark" sx={{ mt: 1 }}>
                {card.value}
              </Typography>
            </Paper>
          </Grid>
        ))}
      </Grid>
    </AdminLayout>
  );
}
```

Output:



➢ AdminProducts.jsx (Admin – Manage Products)

Path: frontend/src/pages/admin/AdminProducts.jsx

Code:

```
import { useEffect, useState } from "react";
import {
  Alert,
  Avatar,
  Box,
  Button,
  Dialog,
  DialogActions,
  DialogContent,
  DialogTitle,
  Grid,
  IconButton,
  MenuItem,
  Paper,
  Stack,
  Table,
  TableBody,
  TableCell,
  TableHead,
  TableRow,
  TextField,
  Typography,
} from "@mui/material";
import AddIcon from "@mui/icons-material/Add";
import EditIcon from "@mui/icons-material/Edit";
```

```
import DeleteOutlineIcon from "@mui/icons-material/DeleteOutlineOutlined";

import AdminLayout from "../../components/admin/AdminLayout";
import ImageUploader from "../../components/admin/ImageUploader";
import LoadingSpinner from "../../components/common/LoadingSpinner";
import { productService } from "../../services/productService";
import { categoryService } from "../../services/categoryService";
import { resolveImageUrl } from "../../services/api";

const currency = new Intl.NumberFormat("en-IN", {
  style: "currency",
  currency: "INR",
  maximumFractionDigits: 0,
});
const EMPTY_FORM = {
  name: "",
  category_id: "",
  description: "",
  image_urls: [],
  monthly_price: "",
  quarterly_price: "",
  yearly_price: "",
  deposit: "",
  stock_qty: 10,
  is_featured: false,
};

export default function AdminProducts() {
  const [products, setProducts] = useState([]);
  const [categories, setCategories] = useState([]);
  const [loading, setLoading] = useState(true);
  const [dialogOpen, setDialogOpen] = useState(false);
  const [editingId, setEditingId] = useState(null);
  const [form, setForm] = useState(EMPTY_FORM);
  const [error, setError] = useState("");

  const loadData = () => {
    setLoading(true);
    Promise.all([productService.list({}), categoryService.list()])
      .then(([productData, categoryData]) => {
        setProducts(productData);
        setCategories(categoryData);
      })
      .finally(() => setLoading(false));
  };

  useEffect(() => {
    loadData();
  }, []);

  const openCreateDialog = () => {
    setEditingId(null);
    setForm(EMPTY_FORM);
    setError("");
    setDialogOpen(true);
  };

  const openEditDialog = async (product) => {
    const full = await productService.get(product.id);
    setEditingId(full.id);
    setForm({
      name: full.name,
```

```
    category_id: full.category_id,
    description: full.description,
    image_urls: full.image_urls,
    monthly_price: full.monthly_price,
    quarterly_price: full.quarterly_price,
    yearly_price: full.yearly_price,
    deposit: full.deposit,
    stock_qty: full.stock_qty,
    is_featured: full.is_featured,
  });
  setError("");
  setDialogOpen(true);
};

const handleChange = (field) => (e) => {
  const value = field === "is_featured" ? e.target.checked : e.target.value;
  setForm({ ...form, [field]: value });
};

const selectedCategorySlug = categories.find((c) => c.id ===
    Number(form.category_id))?.slug || "";

const handleSubmit = async () => {
  setError("");
  const payload = {
    name: form.name,
    category_id: Number(form.category_id),
    description: form.description,
    image_urls: form.image_urls,
    monthly_price: Number(form.monthly_price),
    quarterly_price: Number(form.quarterly_price),
    yearly_price: Number(form.yearly_price),
    deposit: Number(form.deposit || 0),
    stock_qty: Number(form.stock_qty || 0),
    is_featured: Boolean(form.is_featured),
  };

  try {
    if (editingId) {
      await productService.update(editingId, payload);
    } else {
      await productService.create(payload);
    }
    setDialogOpen(false);
    loadData();
  } catch {
    setError("Could not save product. Check all fields are filled correctly.");
  }
};

const handleDelete = async (id) => {
  if (!window.confirm("Delete this product?")) return;
  await productService.remove(id);
  loadData();
};

if (loading) return <LoadingSpinner minHeight="40vh" />;

return (
  <AdminLayout>
    <Stack direction="row" justifyContent="space-between" alignItems="center" sx={{ mb: 3
        }}>
```

```
  <Typography variant="h6">Products ({products.length})</Typography>
  <Button variant="contained" color="primary" startIcon={<AddIcon />}
      onClick={openCreateDialog}>
    Add Product
  </Button>
</Stack>

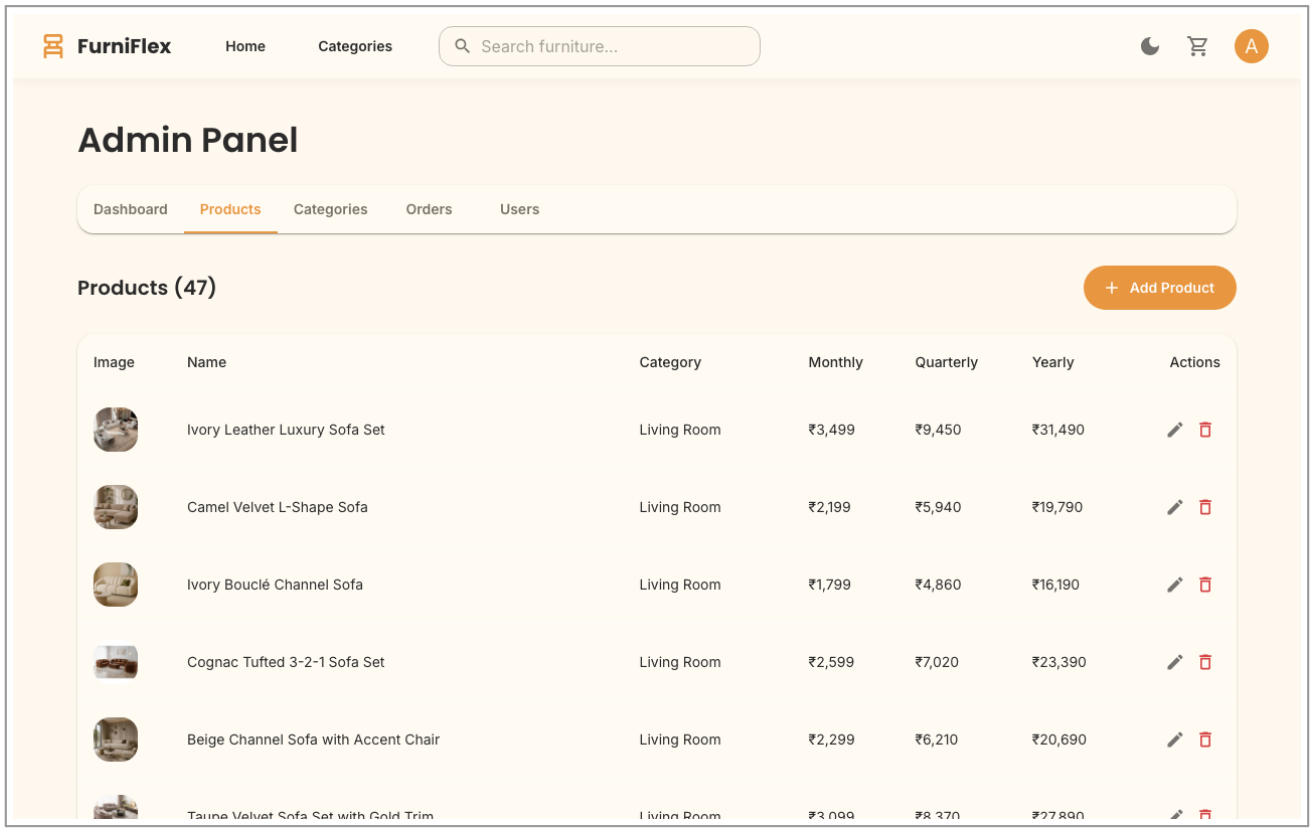
<Paper sx={{ overflowX: "auto" }}>
  <Table>
    <TableHead>
      <TableRow>
        <TableCell>Image</TableCell>
        <TableCell>Name</TableCell>
        <TableCell>Category</TableCell>
        <TableCell>Monthly</TableCell>
        <TableCell>Quarterly</TableCell>
        <TableCell>Yearly</TableCell>
        <TableCell align="right">Actions</TableCell>
      </TableRow>
    </TableHead>
    <TableBody>
      {products.map((product) => (
        <TableRow key={product.id}>
          <TableCell>
            <Avatar
              variant="rounded"
              src={resolveImageUrl(product.image_urls?.[0])}
              alt={product.name}
              sx={{ width: 44, height: 44 }}
            />
          </TableCell>
          <TableCell>{product.name}</TableCell>
          <TableCell>
            {categories.find((c) => c.id === product.category_id)?.name || "—"}
          </TableCell>
          <TableCell>{currency.format(product.monthly_price)}</TableCell>
          <TableCell>{currency.format(product.quarterly_price)}</TableCell>
          <TableCell>{currency.format(product.yearly_price)}</TableCell>
          <TableCell align="right">
            <IconButton size="small" onClick={() => openEditDialog(product)}>
              <EditIcon fontSize="small" />
            </IconButton>
            <IconButton size="small" color="error" onClick={() =>
                handleDelete(product.id)}>
              <DeleteOutlineIcon fontSize="small" />
            </IconButton>
          </TableCell>
        </TableRow>
      ))}
    </TableBody>
  </Table>
</Paper>

<Dialog open={dialogOpen} onClose={() => setDialogOpen(false)} maxWidth="sm"
    fullWidth>
  <DialogTitle>{editingId ? "Edit Product" : "Add Product"}</DialogTitle>
  <DialogContent>
    {error && (
      <Alert severity="error" sx={{ mb: 2 }}>
        {error}
      </Alert>
    )}
```

```
<Grid container spacing={2} sx={{ mt: 0.5 }}>
  <Grid size={12}>
    <TextField fullWidth label="Name" value={form.name}
        onChange={handleChange("name")} />
  </Grid>
  <Grid size={12}>
    <TextField
      select
      fullWidth
      label="Category"
      value={form.category_id}
      onChange={handleChange("category_id")}
    >
      {categories.map((c) => (
        <MenuItem key={c.id} value={c.id}>
          {c.name}
        </MenuItem>
      ))}
    </TextField>
  </Grid>
  <Grid size={12}>
    <TextField
      fullWidth
      multiline
      minRows={2}
      label="Description"
      value={form.description}
      onChange={handleChange("description")}
    />
  </Grid>
  <Grid size={12}>
    <ImageUploader
      label="Product Images"
      multiple
      categorySlug={selectedCategorySlug}
      images={form.image_urls}
      onChange={(images) => setForm({ ...form, image_urls: images })}
    />
  </Grid>
  <Grid size={4}>
    <TextField
      fullWidth
      type="number"
      label="Monthly Price"
      value={form.monthly_price}
      onChange={handleChange("monthly_price")}
    />
  </Grid>
  <Grid size={4}>
    <TextField
      fullWidth
      type="number"
      label="Quarterly Price"
      value={form.quarterly_price}
      onChange={handleChange("quarterly_price")}
    />
  </Grid>
  <Grid size={4}>
    <TextField
      fullWidth
      type="number"
      label="Yearly Price"
```

```
                value={form.yearly_price}
                onChange={handleChange("yearly_price")}
              />
            </Grid>
            <Grid size={6}>
              <TextField
                fullWidth
                type="number"
                label="Deposit"
                value={form.deposit}
                onChange={handleChange("deposit")}
              />
            </Grid>
            <Grid size={6}>
              <TextField
                fullWidth
                type="number"
                label="Stock Quantity"
                value={form.stock_qty}
                onChange={handleChange("stock_qty")}
              />
            </Grid>
            <Grid size={12}>
              <Box sx={{ display: "flex", alignItems: "center", gap: 1 }}>
                <input
                  type="checkbox"
                  checked={form.is_featured}
                  onChange={handleChange("is_featured")}
                  id="is_featured"
                />
                <label htmlFor="is_featured">Featured product</label>
              </Box>
            </Grid>
          </Grid>
        </DialogContent>
        <DialogActions>
          <Button onClick={() => setDialogOpen(false)}>Cancel</Button>
          <Button variant="contained" color="primary" onClick={handleSubmit}>
            {editingId ? "Save Changes" : "Create Product"}
          </Button>
        </DialogActions>
      </Dialog>
    </AdminLayout>
  );
}
```

Output:



➢ AdminCategories.jsx (Admin – Manage Categories)

Path: frontend/src/pages/admin/AdminCategories.jsx

Code:

```
import { useEffect, useState } from "react";
import {
  Alert,
  Avatar,
  Button,
  Dialog,
  DialogActions,
  DialogContent,
  DialogTitle,
  Grid,
  IconButton,
  Paper,
  Stack,
  Table,
  TableBody,
  TableCell,
  TableHead,
  TableRow,
  TextField,
  Typography,
} from "@mui/material";
import AddIcon from "@mui/icons-material/Add";
import EditIcon from "@mui/icons-material/Edit";
import DeleteOutlineIcon from "@mui/icons-material/DeleteOutlineOutlined";
```

```
import AdminLayout from "../../components/admin/AdminLayout";
import ImageUploader from "../../components/admin/ImageUploader";
import LoadingSpinner from "../../components/common/LoadingSpinner";
import { categoryService } from "../../services/categoryService";
import { resolveImageUrl } from "../../services/api";

const EMPTY_FORM = { name: "", slug: "", description: "", image_url: "" };

export default function AdminCategories() {
  const [categories, setCategories] = useState([]);
  const [loading, setLoading] = useState(true);
  const [dialogOpen, setDialogOpen] = useState(false);
  const [editingId, setEditingId] = useState(null);
  const [form, setForm] = useState(EMPTY_FORM);
  const [error, setError] = useState("");
  const loadData = () => {
    setLoading(true);
    categoryService.list().then(setCategories).finally(() => setLoading(false));
  };

  useEffect(() => {
    loadData();
  }, []);

  const openCreateDialog = () => {
    setEditingId(null);
    setForm(EMPTY_FORM);
    setError("");
    setDialogOpen(true);
  };

  const openEditDialog = (category) => {
    setEditingId(category.id);
    setForm({
      name: category.name,
      slug: category.slug,
      description: category.description || "",
      image_url: category.image_url || "",
    });
    setError("");
    setDialogOpen(true);
  };

  const handleChange = (field) => (e) => setForm({ ...form, [field]: e.target.value });

  const handleSubmit = async () => {
    setError("");
    try {
      if (editingId) {
        await categoryService.update(editingId, form);
      } else {
        await categoryService.create(form);
      }
      setDialogOpen(false);
      loadData();
    } catch {
      setError("Could not save category. Slug might already be in use.");
    }
  };

  const handleDelete = async (id) => {
    if (!window.confirm("Delete this category? Its products will also be removed.")) return;
```

```
  await categoryService.remove(id);
  loadData();
};

if (loading) return <LoadingSpinner minHeight="40vh" />;

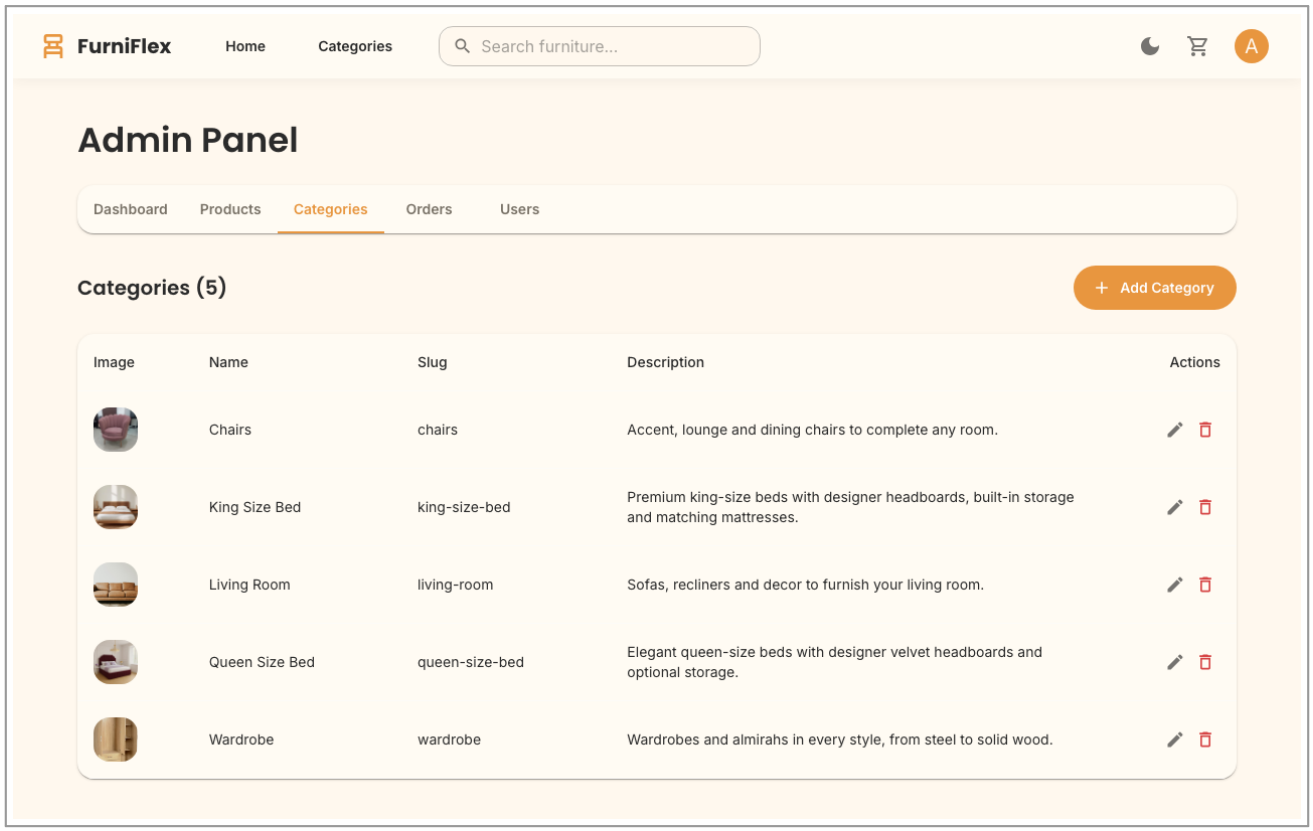
return (
  <AdminLayout>
    <Stack direction="row" justifyContent="space-between" alignItems="center" sx={{ mb: 3
        }}>
      <Typography variant="h6">Categories ({categories.length})</Typography>
      <Button variant="contained" color="primary" startIcon={<AddIcon />}
          onClick={openCreateDialog}>
        Add Category
      </Button>
    </Stack>

    <Paper sx={{ overflowX: "auto" }}>
      <Table>
        <TableHead>
          <TableRow>
            <TableCell>Image</TableCell>
            <TableCell>Name</TableCell>
            <TableCell>Slug</TableCell>
            <TableCell>Description</TableCell>
            <TableCell align="right">Actions</TableCell>
          </TableRow>
        </TableHead>
        <TableBody>
          {categories.map((category) => (
            <TableRow key={category.id}>
              <TableCell>
                <Avatar
                  variant="rounded"
                  src={resolveImageUrl(category.image_url)}
                  alt={category.name}
                  sx={{ width: 44, height: 44 }}
                />
              </TableCell>
              <TableCell>{category.name}</TableCell>
              <TableCell>{category.slug}</TableCell>
              <TableCell sx={{ maxWidth: 320 }}>{category.description}</TableCell>
              <TableCell align="right">
                <IconButton size="small" onClick={() => openEditDialog(category)}>
                  <EditIcon fontSize="small" />
                </IconButton>
                <IconButton size="small" color="error" onClick={() =>
                    handleDelete(category.id)}>
                  <DeleteOutlineIcon fontSize="small" />
                </IconButton>
              </TableCell>
            </TableRow>
          ))}
        </TableBody>
      </Table>
    </Paper>

    <Dialog open={dialogOpen} onClose={() => setDialogOpen(false)} maxWidth="sm"
        fullWidth>
      <DialogTitle>{editingId ? "Edit Category" : "Add Category"}</DialogTitle>
      <DialogContent>
        {error && (
```

```
            <Alert severity="error" sx={{ mb: 2 }}>
              {error}
            </Alert>
          )}
          <Grid container spacing={2} sx={{ mt: 0.5 }}>
            <Grid size={12}>
              <TextField fullWidth label="Name" value={form.name}
                  onChange={handleChange("name")} />
            </Grid>
            <Grid size={12}>
              <TextField
                fullWidth
                label="Slug"
                helperText="URL-friendly identifier, e.g. living-room — also the image
                    folder name"
                value={form.slug}
                onChange={handleChange("slug")}
              />
            </Grid>
            <Grid size={12}>
              <TextField
                fullWidth
                multiline
                minRows={2}
                label="Description"
                value={form.description}
                onChange={handleChange("description")}
              />
            </Grid>
            <Grid size={12}>
              <ImageUploader
                label="Category Image"
                multiple={false}
                categorySlug={form.slug.trim()}
                images={form.image_url ? [form.image_url] : []}
                onChange={(images) => setForm({ ...form, image_url: images[0] || "" })}
              />
            </Grid>
          </Grid>
        </DialogContent>
        <DialogActions>
          <Button onClick={() => setDialogOpen(false)}>Cancel</Button>
          <Button variant="contained" color="primary" onClick={handleSubmit}>
            {editingId ? "Save Changes" : "Create Category"}
          </Button>
        </DialogActions>
      </Dialog>
    </AdminLayout>
  );
}
```

Output:



➢ AdminOrders.jsx (Admin – Manage Orders)

Path: frontend/src/pages/admin/AdminOrders.jsx

Code:

```
import { useEffect, useState } from "react";
import {
  Chip,
  MenuItem,
  Paper,
  Select,
  Table,
  TableBody,
  TableCell,
  TableHead,
  TableRow,
  Typography,
} from "@mui/material";

import AdminLayout from "../../components/admin/AdminLayout";
import LoadingSpinner from "../../components/common/LoadingSpinner";
import { adminService } from "../../services/orderService";

const currency = new Intl.NumberFormat("en-IN", {
  style: "currency",
  currency: "INR",
  maximumFractionDigits: 0,
});

const STATUSES = ["PLACED", "CONFIRMED", "DELIVERED", "RETURNED", "CANCELLED"];
```

```
const STATUS_COLORS = {
  PLACED: "info",
  CONFIRMED: "primary",
  DELIVERED: "success",
  RETURNED: "default",
  CANCELLED: "error",
};

export default function AdminOrders() {
  const [orders, setOrders] = useState([]);
  const [loading, setLoading] = useState(true);

  const loadData = () => {
    setLoading(true);
    adminService.orders().then(setOrders).finally(() => setLoading(false));
  };

  useEffect(() => {
    loadData();
  }, []);

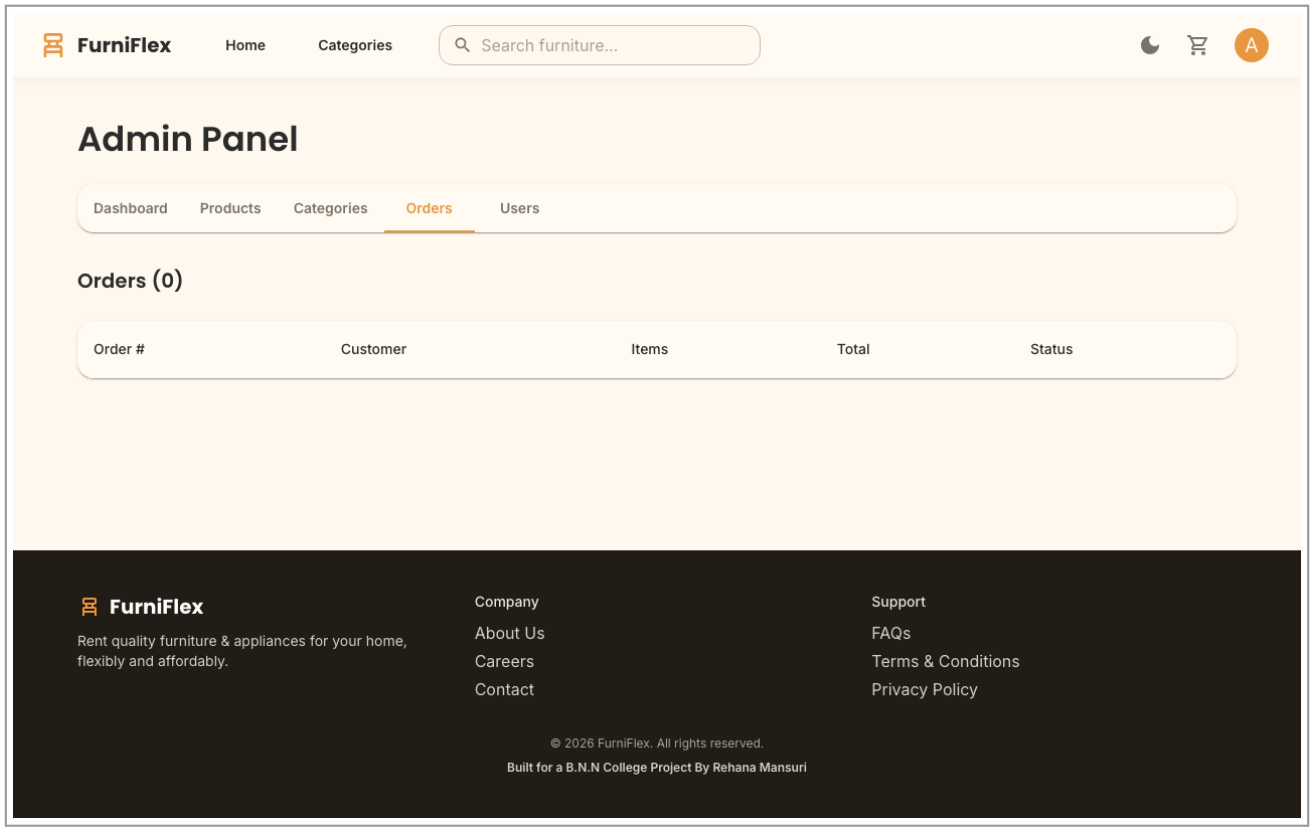
  const handleStatusChange = async (orderId, status) => {
    await adminService.updateOrderStatus(orderId, status);
    loadData();
  };

  if (loading) return <LoadingSpinner minHeight="40vh" />;

  return (
    <AdminLayout>
      <Typography variant="h6" sx={{ mb: 3 }}>
        Orders ({orders.length})
      </Typography>
      <Paper sx={{ overflowX: "auto" }}>
        <Table>
          <TableHead>
            <TableRow>
              <TableCell>Order #</TableCell>
              <TableCell>Customer</TableCell>
              <TableCell>Items</TableCell>
              <TableCell>Total</TableCell>
              <TableCell>Status</TableCell>
            </TableRow>
          </TableHead>
          <TableBody>
            {orders.map((order) => (
              <TableRow key={order.id}>
                <TableCell>#{order.id}</TableCell>
                <TableCell>
                  {order.user?.name}
                  <br />
                  <Typography variant="caption" color="text.secondary">
                    {order.user?.email}
                  </Typography>
                </TableCell>
                <TableCell>{order.items.length} item(s)</TableCell>
                <TableCell>{currency.format(order.total_amount)}</TableCell>
                <TableCell>
                  <Select
                    size="small"
                    value={order.status}
```

```
                    onChange={(e) => handleStatusChange(order.id, e.target.value)}
                    renderValue={(value) => (
                      <Chip label={value} size="small" color={STATUS_COLORS[value] ||
                          "default"} />
                    )}
                  >
                    {STATUSES.map((status) => (
                      <MenuItem key={status} value={status}>
                        {status}
                      </MenuItem>
                    ))}
                  </Select>
                </TableCell>
              </TableRow>
            ))}
          </TableBody>
        </Table>
      </Paper>
    </AdminLayout>
  );
}
```

Output:



➢ AdminUsers.jsx (Admin – Users)

Path: frontend/src/pages/admin/AdminUsers.jsx

Code:

```
import { useEffect, useState } from "react";
import { Chip, Paper, Table, TableBody, TableCell, TableHead, TableRow, Typography } from
    "@mui/material";

import AdminLayout from "../../components/admin/AdminLayout";
import LoadingSpinner from "../../components/common/LoadingSpinner";
import { adminService } from "../../services/orderService";

export default function AdminUsers() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    adminService.users().then(setUsers).finally(() => setLoading(false));
  }, []);

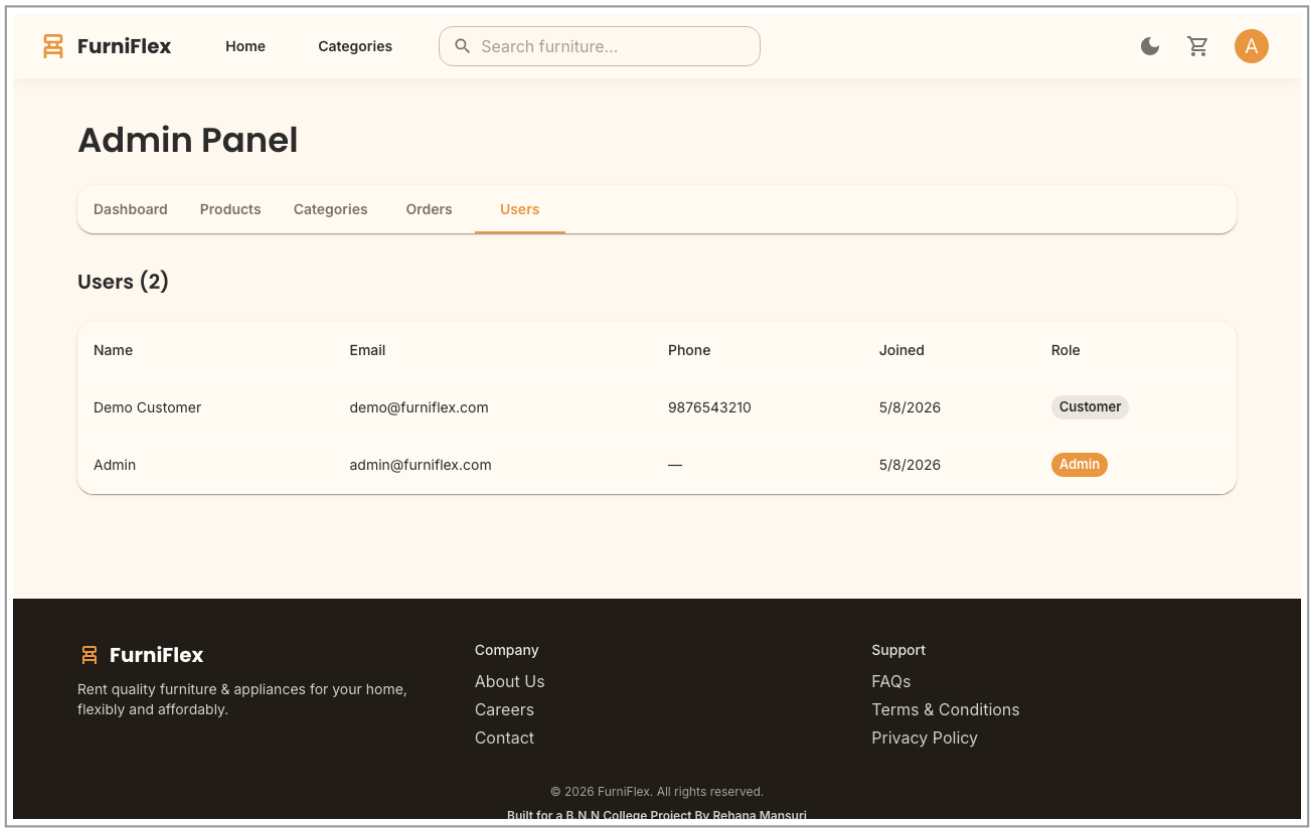
  if (loading) return <LoadingSpinner minHeight="40vh" />;

  return (
    <AdminLayout>
      <Typography variant="h6" sx={{ mb: 3 }}>
        Users ({users.length})
      </Typography>

      <Paper sx={{ overflowX: "auto" }}>
        <Table>
          <TableHead>
            <TableRow>
              <TableCell>Name</TableCell>
              <TableCell>Email</TableCell>
              <TableCell>Phone</TableCell>
              <TableCell>Joined</TableCell>
              <TableCell>Role</TableCell>
            </TableRow>
          </TableHead>
          <TableBody>
            {users.map((user) => (
              <TableRow key={user.id}>
                <TableCell>{user.name}</TableCell>
                <TableCell>{user.email}</TableCell>
                <TableCell>{user.phone || "—"}</TableCell>
                <TableCell>{new Date(user.created_at).toLocaleDateString("en-IN")}</TableCel
                    l>
                <TableCell>
                  <Chip
                    label={user.is_admin ? "Admin" : "Customer"}
                    color={user.is_admin ? "primary" : "default"}
                    size="small"
                  />
                </TableCell>
              </TableRow>
            ))}
          </TableBody>
        </Table>
      </Paper>
    </AdminLayout>
  );
```

}

Output:


FurniFlex
Home
Categories
Search furniture...
A
Admin Panel
Dashboard
Products
Categories
Orders
Users
Users (2)
Name
Email
Phone
Joined
Role
Demo Customer
demo@furniflex.com
9876543210
5/8/2026
Customer
Admin
admin@furniflex.com
—
5/8/2026
Admin
FurniFlex
Rent quality furniture & appliances for your home, flexibly and affordably.
Company
About Us
Careers
Contact
Support
FAQs
Terms & Conditions
Privacy Policy
© 2026 FurniFlex. All rights reserved.
Built for a B.N.N College Project By Rehana Mansuri

TESTING

5.1 Testing Approach:-

1. Introduction to Testing

Testing is a crucial phase in the software development life cycle that ensures the web application functions as intended. For this project both manual and tool-assisted testing techniques were applied to validate the application's functionality, security and user experience. The primary focus was to ensure that the React frontend, the FastAPI backend and the PostgreSQL database work together seamlessly and that the business rules (rental plan pricing, cart handling, checkout and admin-only access) execute correctly.

2. Testing Environment

- **Development Platform:** VS Code, macOS
- **Technology Stack:** React (Vite) + Material UI, Python FastAPI, SQLAlchemy
- **Server:** Uvicorn (inside Docker), Vite development server
- **Database:** PostgreSQL 16 (Docker container)
- **Tools:** Swagger UI (/docs), browser developer tools, psql

3. Types of Testing Conducted

a. API (Unit) Testing

- **Objective:** To test each REST endpoint independently.
- **Approach:** Every endpoint was called through the auto-generated Swagger UI with valid and invalid inputs, and the HTTP status codes and JSON responses were checked – for example 201 on successful registration, 401 for a wrong password, 403 when a customer calls an admin endpoint and 422 for an invalid e-mail.

b. Integration Testing

- **Objective:** To ensure proper interaction between the React pages, the API and the database.
- **Approach:** The complete flow from the React pages (Axios calls with the JWT token) to the FastAPI routers and then to PostgreSQL was tested, e.g. adding an item to the cart on the product page and verifying that it appears in the cart page and in the *cart_items* table.

c. Functional Testing

- **Objective:** To verify that each feature works as per the requirements.
- **Approach:** Manual testing was performed on each functionality, such as:
  - User registration, login and logout.
  - Profile management.
  - Browsing categories, global search, filters and sorting.

- Rental plan selection, cart add / update / remove.
- Checkout with Card, UPI and GPay QR and order history.
- Admin product / category CRUD, image upload and order status update.

d. Database Testing

- **Objective:** To verify the integrity of data stored in and retrieved from PostgreSQL.
- **Approach:** Queries were executed to check the correctness of insertion, updates and deletions in tables such as users, products, cart_items, orders and order_items, and to confirm that the cart is cleared and *price_at_order* is stored after checkout.

e. User Interface Testing

- **Objective:** To ensure that the user interface provides a seamless experience across browsers and screen sizes.
- **Approach:** The UI was tested in Chrome, Firefox and Edge, in both light and dark modes, and at mobile, tablet and desktop widths using the browser's responsive design mode.

f. Security Testing

- **Objective:** To confirm that protected data and admin functions cannot be accessed without authorisation.
- **Approach:** Requests were sent without a token, with a tampered token and with a customer token to admin endpoints; all were correctly rejected with 401 or 403.

5.2 Test Cases:-

The following test cases were executed against the running application and API:

ID	Test Scenario	Input	Expected Result	Actual Result	Status
TC01	Login with valid credentials	demo@furniflex .com / correct password	200 OK with JWT token	Token returned, user logged in	Pass
TC02	Login with wrong password	Valid e-mail, wrong password	401 Unauthorized	401 "Incorrect email or password"	Pass
TC03	Register with existing e-mail	Already registered e-mail	400 Bad Request	400 "Email already registered"	Pass
TC04	Register with invalid e-mail	"notanemail"	422 Validation error	422 "value is not a valid email address"	Pass
TC05	Access profile without login	No token	401 Unauthorized	401 "Not authenticated"	Pass
TC06	Access with tampered token	Modified JWT	401 Unauthorized	401 "Could not validate credentials"	Pass

ID	Test Scenario	Input	Expected Result	Actual Result	Status
TC07	Customer opens admin dashboard API	Customer token	403 Forbidden	403 “Admin privileges required”	Pass
TC08	Customer creates a product	Customer token	403 Forbidden	403 “Admin privileges required”	Pass
TC09	Add invalid product to cart	product_id = 999999	400 Bad Request	400 “Invalid product_id”	Pass
TC10	Checkout with invalid payment method	payment_method = BITCOIN	400 Bad Request	400 “payment_method must be one of [CARD, GPAY_QR, UPI]”	Pass
TC11	Search with filters	search=bed, min_price=1000	Only matching products	Filtered product list returned	Pass
TC12	Open non-existing product	/api/products/999999	404 Not Found	404 “Product not found”	Pass
TC13	View rental plans	GET /api/rental-plans	Monthly, Quarterly, Yearly	3 plans with discounts returned	Pass

DEPLOYMENT

6.1 Deployment Overview:-

After development and testing, FurniFlex was deployed to the cloud so that it can be accessed by anyone on the internet through a proper domain name over a secure HTTPS connection. The application is hosted on an **Amazon Web Services (AWS) EC2** virtual server running Ubuntu Linux. The following technologies are used in the deployment:

- **AWS EC2 Instance:** A virtual server in the AWS cloud on which the complete application runs. A **Security Group** acts as a firewall and only allows SSH (22), HTTP (80) and HTTPS (443) traffic.
- **Docker & Docker Compose:** The FastAPI backend and the PostgreSQL database run as two containers managed by Docker Compose, exactly as in development. Data is kept in a Docker volume so it survives restarts.
- **Nginx (Reverse Proxy):** Nginx is the single entry point for all traffic. It serves the built React files directly and forwards (*reverse proxies*) every */api* and */static* request to the backend container on 127.0.0.1:8000. It also terminates SSL, so the backend itself never handles certificates.
- **DuckDNS (Free Domain):** DuckDNS is a free dynamic DNS service. It provides the sub-domain **furniflex.duckdns.org** and points it to the public IP address of the EC2 instance. A small cron job keeps the IP up to date.
- **Certbot (SSL Certificate):** Certbot is the official client for **Let's Encrypt**, a free certificate authority. It proves ownership of the domain, obtains a trusted SSL certificate, configures Nginx for HTTPS and renews the certificate automatically before it expires (every 90 days).

6.2 Deployment Architecture:-

The figure below shows how the different parts of the deployment are connected. A user's browser first resolves the DuckDNS domain to the EC2 public IP, then connects to Nginx over HTTPS. Nginx serves the React application and forwards API and image requests to the FastAPI container, which reads and writes data in the PostgreSQL container.

Components of the deployment:

Component	Technology	Role in Deployment	Port
Cloud Server	AWS EC2 (Ubuntu 24.04 LTS)	Hosts all components of the application	–
Firewall	AWS Security Group	Allows only SSH, HTTP and HTTPS from the internet	22, 80, 443
Domain Name	DuckDNS	Maps furniflex.duckdns.org to the EC2 public IP	–
Reverse Proxy	Nginx	Serves React build, proxies /api and /static, terminates SSL	80, 443
SSL Certificate	Certbot + Let's Encrypt	Free HTTPS certificate with automatic renewal	–
Frontend	React (Vite build)	Static files in /var/www/furniflex served by Nginx	–
Backend	FastAPI container (Docker)	REST API, authentication, image files	8000 (localhost only)
Database	PostgreSQL 16 container (Docker)	Stores users, products, carts and orders	5432 (Docker network only)

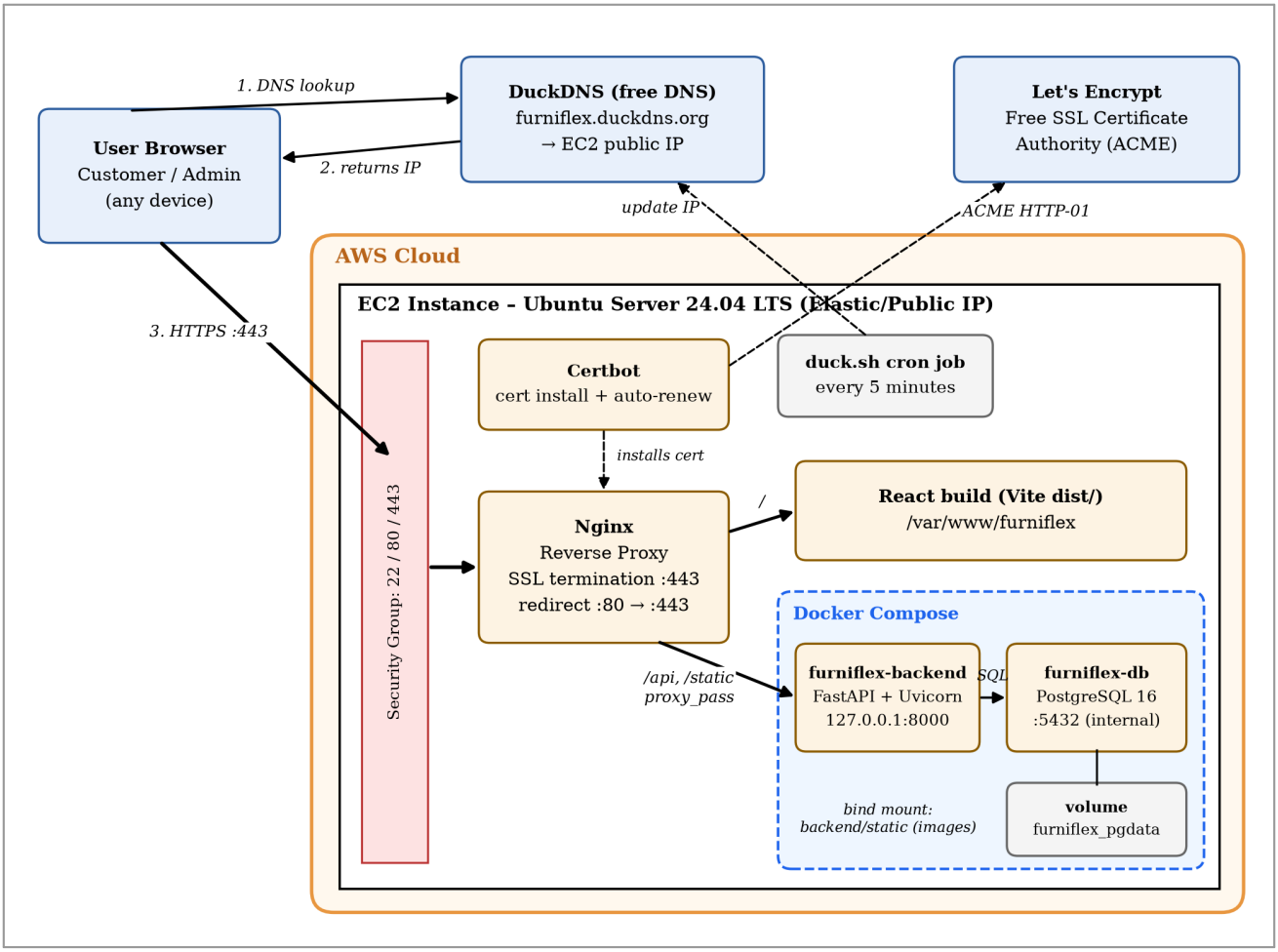


Fig 6.1: Deployment architecture of FurniFlex on AWS EC2

6.3 Deployment Use Case Diagram:-

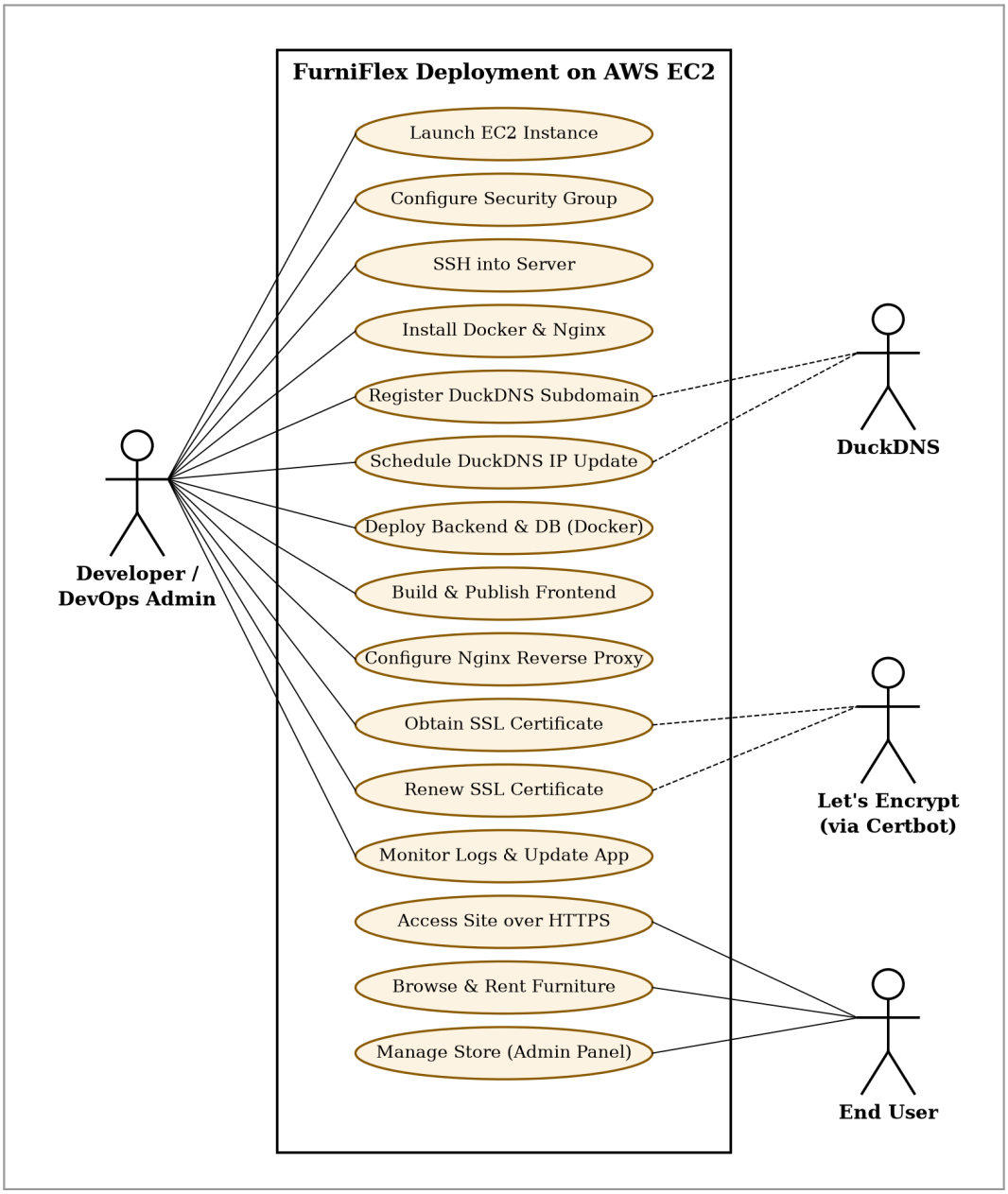


Fig 6.2: Use case diagram for deploying and accessing FurniFlex

The **Developer / DevOps Admin** performs all deployment tasks: launching and securing the EC2 instance, installing Docker and Nginx, setting up the DuckDNS domain, deploying the containers and the React build, configuring Nginx and obtaining the SSL certificate. **DuckDNS** and **Let's Encrypt** are external systems that take part in the domain and certificate use cases. The **End User** (customer or store admin) simply opens the site over HTTPS and uses the application.

6.4 Deployment Flow:-

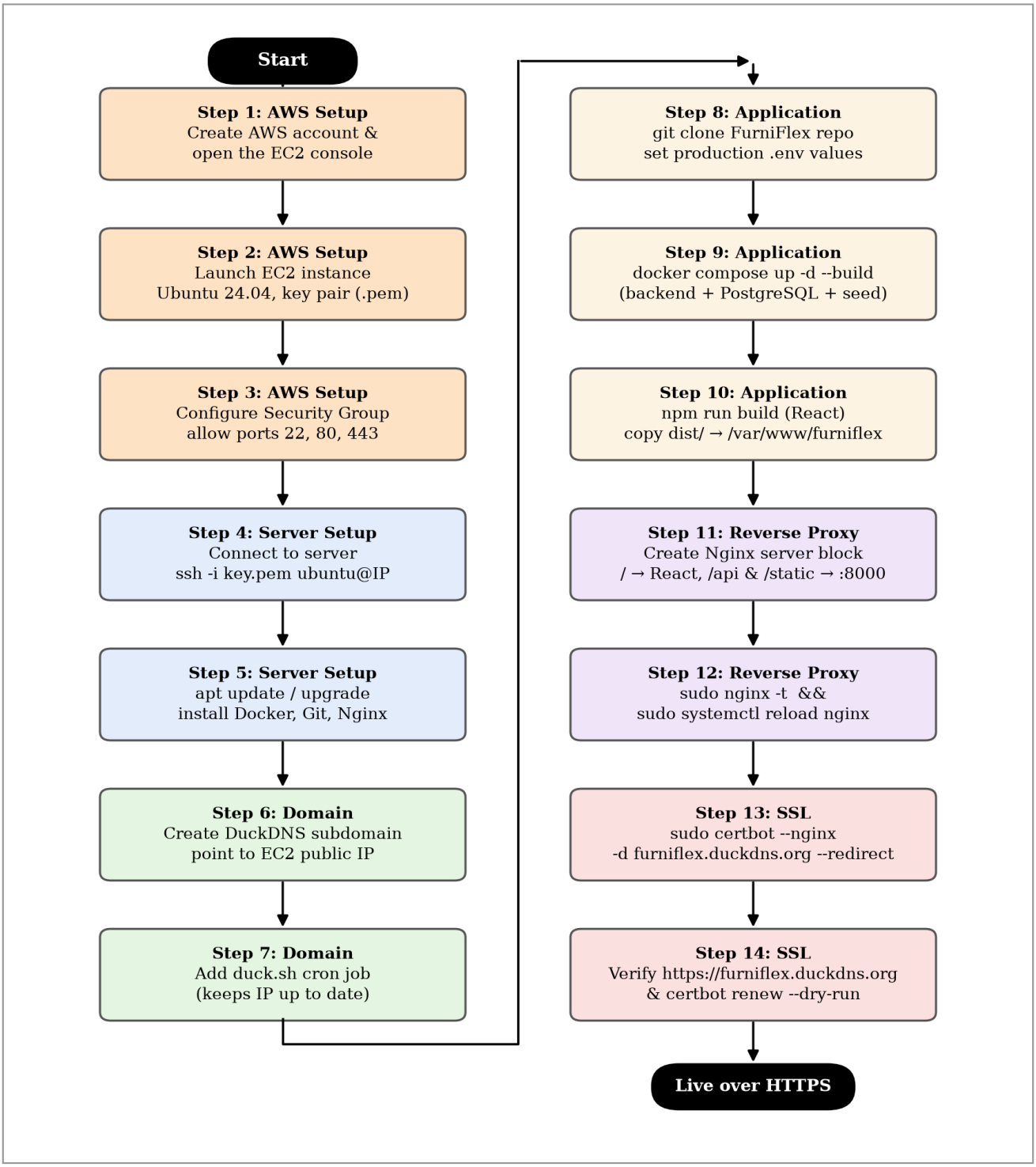


Fig 6.3: Step-by-step deployment flow on AWS EC2

6.5 Step-by-Step Deployment on AWS EC2:-

The following steps were used to deploy FurniFlex. The domain used in this report is **furniflex.duckdns.org**; values shown in angle brackets (for example <EC2-PUBLIC-IP>) are replaced with the real values of the server.

Step 1: Launch the EC2 Instance

Log in to the AWS Management Console, open **EC2 → Instances → Launch instance** and choose the following settings:

Setting	Value
Name	furniflex-server
AMI (Operating System)	Ubuntu Server 24.04 LTS (64-bit x86)
Instance type	t2.micro / t3.micro (free tier) or t3.small
Key pair	Create new key pair → furniflex-key.pem (downloaded and kept safe)
Storage	20 GB gp3 EBS volume
Elastic IP (optional)	Allocate and associate, so the public IP does not change after a stop / start

Step 2: Configure the Security Group (Firewall)

While launching the instance, a security group was created with the following inbound rules. All outbound traffic is allowed.

Type	Protocol	Port	Source	Purpose
SSH	TCP	22	My IP	Remote login for the developer only
HTTP	TCP	80	0.0.0.0/0, ::/0	Let's Encrypt challenge and redirect to HTTPS
HTTPS	TCP	443	0.0.0.0/0, ::/0	Secure access to the website

Ports 8000 (backend) and 5432 (database) are **not** opened, so they cannot be reached directly from the internet – all traffic must go through Nginx.

Step 3: Connect to the Server using SSH

From the developer's computer, the downloaded key file is protected and used to log in:

```
chmod 400 furniflex-key.pem
ssh -i furniflex-key.pem ubuntu@<EC2-PUBLIC-IP>
```

Step 4: Install Docker, Git and Nginx

The system packages are updated and the required software is installed. Docker is installed with the official convenience script, which also installs the Docker Compose plugin.

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y git nginx curl

curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh
sudo usermod -aG docker ubuntu      # run docker without sudo
newgrp docker

docker --version
docker compose version
sudo systemctl enable --now docker nginx
```

Because a micro instance has only 1 GB of RAM, a 2 GB swap file is added so that the Docker image build and the React build do not run out of memory:

```
sudo fallocate -l 2G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile && sudo swapon /swapfile
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
```

Step 5: Create a Free Domain with DuckDNS

On **https://www.duckdns.org** the developer signs in (with GitHub / Google), creates the sub-domain **furniflex** and enters the EC2 public IP in the *current ip* field. The domain **furniflex.duckdns.org** now points to the server. DuckDNS also shows a personal *token*, which is used by a small script to keep the IP address up to date:

```
mkdir -p ~/duckdns && cd ~/duckdns
cat > duck.sh <<'EOF'
echo url="https://www.duckdns.org/update?domains=furniflex&token=<TOKEN>\
&ip=" | curl -k -o ~/duckdns/duck.log -K -
EOF
chmod 700 duck.sh
./duck.sh && cat duck.log          # prints OK on success

crontab -e
# add this line -> runs every 5 minutes
*/5 * * * * ~/duckdns/duck.sh >/dev/null 2>&1
```

The DNS record can be checked with *nslookup furniflex.duckdns.org*, which should return the EC2 public IP.

Step 6: Clone the Project and Set Production Values

The source code is cloned from GitHub. Before starting the containers, the development defaults in *docker-compose.yml* are changed for production: a strong database password and JWT secret, the HTTPS domain as the allowed CORS origin, the backend bound to localhost only, and the public PostgreSQL port removed.

```
cd ~
git clone https://github.com/RehanaPinjari22/furniflex.git
cd furniflex
nano docker-compose.yml
```

Changed parts of *docker-compose.yml*:

```
  db:
    environment:
      POSTGRES_USER: furniflex
      POSTGRES_PASSWORD: <db-password>
      POSTGRES_DB: furniflex
    # no "ports:" -> PostgreSQL reachable only inside Docker

  backend:
    environment:
      DATABASE_URL: postgresql+psycopg2://furniflex:<db-password>@db:5432/furniflex
      JWT_SECRET_KEY: <long-random-secret>   # openssl rand -hex 32
      CORS_ORIGINS: https://furniflex.duckdns.org
    ports:
      - "127.0.0.1:8000:8000"   # reachable only from the server (Nginx)
```

Step 7: Start the Backend and Database with Docker

Docker Compose builds the backend image, starts PostgreSQL, waits for its health check, seeds the database (categories, rental plans, products and demo accounts) and starts the FastAPI server.

```
docker compose up -d --build
docker compose ps                    # db healthy, backend running
docker compose logs -f backend       # check the startup logs
curl http://127.0.0.1:8000/api/rental-plans
```

Step 8: Build the React Frontend

Node.js 22 is installed and the React application is built with the production API address. The generated *dist/* folder contains static HTML, CSS and JavaScript files, which are copied to the Nginx web root.

```
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -
sudo apt install -y nodejs

cd ~/furniflex/frontend
echo "VITE_API_BASE_URL=https://furniflex.duckdns.org/api" > .env.production
npm ci
npm run build

sudo mkdir -p /var/www/furniflex
sudo cp -r dist/* /var/www/furniflex/
```

Step 9: Configure Nginx as a Reverse Proxy

A new Nginx site is created. The / location serves the React files (falling back to index.html so that React Router URLs such as /products/5 work on refresh), while */api/* and */static/* are forwarded to the FastAPI container.

```
sudo nano /etc/nginx/sites-available/furniflex
```

/etc/nginx/sites-available/furniflex:

```
server {
    listen 80;
    server_name furniflex.duckdns.org;

    # React production build
    root /var/www/furniflex;
    index index.html;

    # admin image uploads are limited to 5 MB by the backend
    client_max_body_size 6M;

    # React Router (single-page app) fallback
    location / {
        try_files $uri $uri/ /index.html;
    }

    # REST API -> FastAPI container
    location /api/ {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    # product / category images served by FastAPI
    location /static/ {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
    }
}
```

The site is enabled, the default site is removed, the configuration is tested and Nginx is reloaded:

```
sudo ln -s /etc/nginx/sites-available/furniflex /etc/nginx/sites-enabled/
sudo rm /etc/nginx/sites-enabled/default
sudo nginx -t                      # syntax is ok / test is successful
sudo systemctl reload nginx
```

At this point the site is already reachable at **http://furniflex.duckdns.org** (without HTTPS).

Step 10: Obtain a Free SSL Certificate with Certbot

Certbot and its Nginx plugin are installed. Certbot contacts Let's Encrypt, completes the HTTP-01 challenge on port 80, installs the certificate in the Nginx configuration and adds an automatic redirect from HTTP to HTTPS.

```
sudo apt install -y certbot python3-certbot-nginx
sudo certbot --nginx -d furniflex.duckdns.org --redirect --agree-tos -m <your-email>
```

Certbot adds the following lines to the Nginx server block automatically:

```
    # managed by Certbot
    listen 443 ssl;
    ssl_certificate     /etc/letsencrypt/live/furniflex.duckdns.org/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/furniflex.duckdns.org/privkey.pem;
    include /etc/letsencrypt/options-ssl-nginx.conf;
    ssl_dhparam /etc/letsencrypt/ssl-dhparams.pem;
```

Let's Encrypt certificates are valid for 90 days. The Certbot package installs a systemd timer that renews them automatically; renewal is tested with a dry run:

```
sudo certbot renew --dry-run
systemctl list-timers | grep certbot
```

Step 11: Verify the Live Deployment

Finally the deployment is verified from a browser and from the terminal:

Check	How	Expected Result
Website opens over HTTPS	Open https://furniflex.duckdns.org	FurniFlex home page with padlock icon
HTTP redirects to HTTPS	curl -I http://furniflex.duckdns.org	301 Moved Permanently → https://
API works through Nginx	curl https://furniflex.duckdns.org/api/rental-plans	JSON list of 3 rental plans
Images load	Open any category page	Product images visible (/static via Nginx)
Page refresh on inner route	Refresh https://furniflex.duckdns.org/categories	Page loads (no 404)
Backend not public	Open http://<EC2-PUBLIC-IP>:8000	Connection fails (port closed)
Login & admin panel	Log in as admin	Admin dashboard and CRUD work

6.6 Request Flow (HTTPS):-

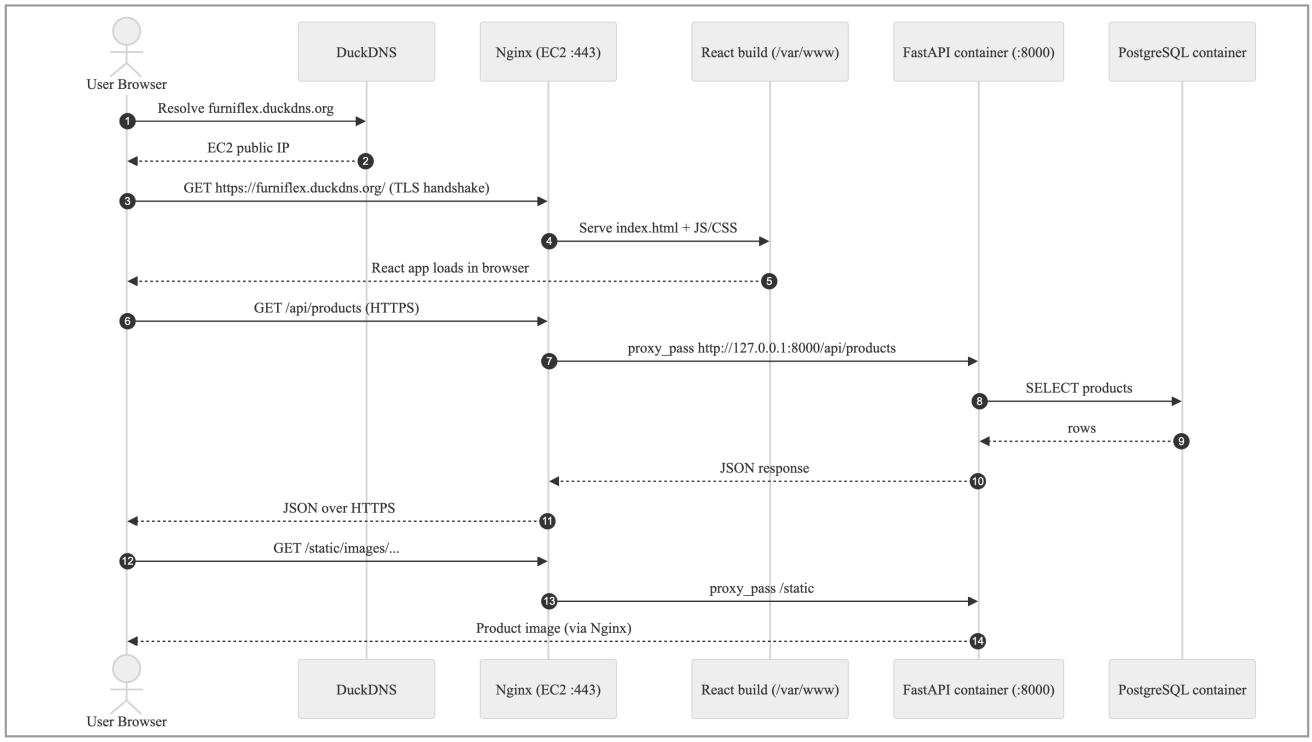


Fig 6.4: How a browser request travels through DuckDNS, Nginx, FastAPI and PostgreSQL

When a user opens the website, the browser first asks DuckDNS for the IP address of the domain. It then opens an encrypted HTTPS connection to Nginx on the EC2 instance. Requests for pages are answered directly by Nginx from the React build, while every request beginning with */api* or */static* is forwarded to the FastAPI container on 127.0.0.1:8000. FastAPI queries PostgreSQL and returns JSON (or an image), which Nginx sends back to the browser over the same secure connection.

6.7 SSL Certificate Flow (Certbot + Let's Encrypt):-

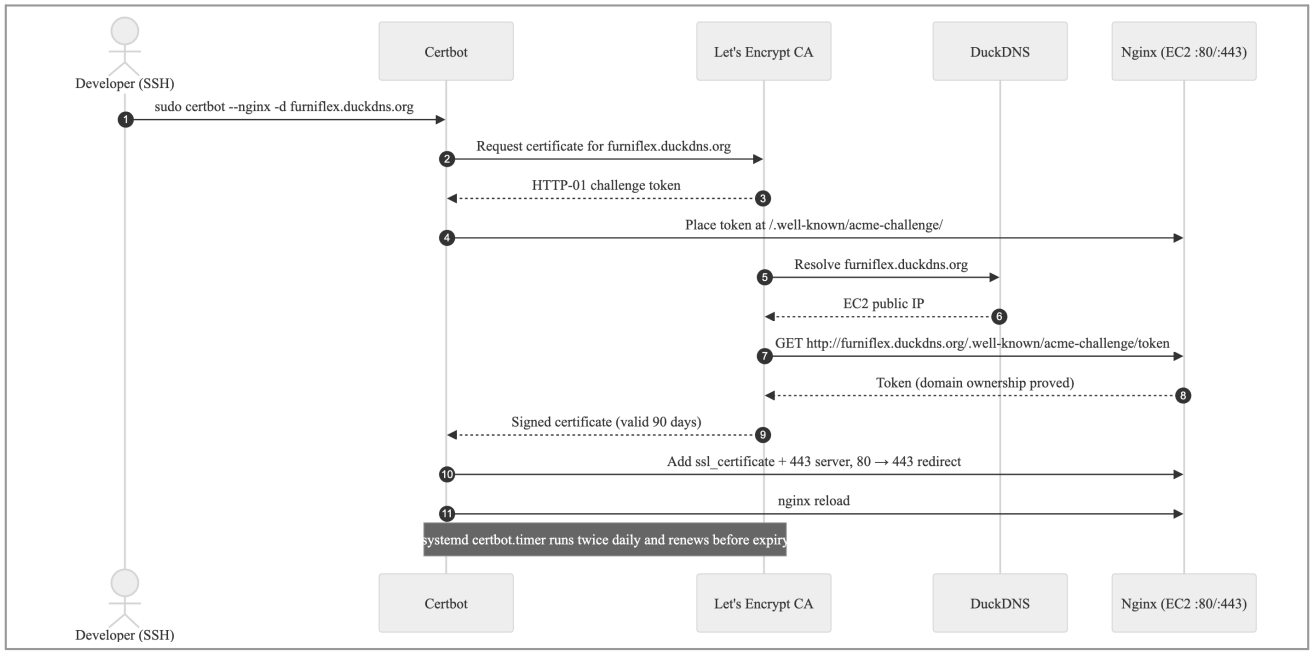


Fig 6.5: Obtaining and renewing the SSL certificate with Certbot

Let's Encrypt issues a certificate only after verifying that the requester controls the domain. Certbot places a temporary token file on the Nginx server; Let's Encrypt resolves the DuckDNS domain and downloads the token over port 80. Once the token matches, the signed certificate is returned, Certbot updates the Nginx configuration for HTTPS and reloads Nginx. The certbot systemd timer repeats this process automatically before the certificate expires.

6.8 Maintenance & Updates:-

When new code is pushed to GitHub, the live server is updated with the following commands:

```
cd ~/furniflex
git pull

# backend changes
docker compose up -d --build

# frontend changes
cd frontend && npm ci && npm run build
sudo cp -r dist/* /var/www/furniflex/
```

Useful commands for monitoring and troubleshooting:

Task	Command
Container status	docker compose ps
Backend logs	docker compose logs -f backend
Restart application	docker compose restart

Task	Command
Nginx error log	sudo tail -f /var/log/nginx/error.log
Test / reload Nginx	sudo nginx -t && sudo systemctl reload nginx
Certificate status	sudo certbot certificates
Database backup	docker exec furniflex-db pg_dump -U furniflex furniflex > backup.sql
DuckDNS update log	cat ~/duckdns/duck.log

ADVANTAGES

7.1 Advantages:-

The FurniFlex web application developed using React, FastAPI and PostgreSQL provides several key advantages that enhance the functionality, reliability and usability of the furniture rental platform:

1. Affordable and Flexible Renting:

Customers can rent furniture on Monthly, Quarterly or Yearly plans with automatic discounts for longer durations, avoiding the large up-front cost of buying furniture.

2. Server-side Cart:

The cart is stored in the database against the logged-in user, so items remain available even after the browser is closed or when the customer logs in from another device.

3. Secure Authentication and Role-based Access:

Passwords are hashed with bcrypt and every request is authenticated with a signed JWT token. Admin APIs are protected on the server side, so a customer cannot access them even by calling the API directly.

4. Device Flexibility and Modern UI:

The responsive Material UI design works on laptops, tablets and smartphones, and supports both light and dark modes, which are remembered for the next visit.

5. Simplified Catalogue Management:

Administrators can add, edit and delete products and categories and upload images directly from the admin panel without touching the code or the database.

6. Accurate Order Records:

Each order item stores the price at the time of ordering and the selected rental plan, so historical orders remain correct even if product prices change later.

7. Fast and Self-documenting API:

FastAPI automatically generates interactive Swagger documentation for every endpoint, which makes testing and future integration (for example a mobile app) much easier.

8. Live, Secure and Low-cost Deployment:

The application runs on AWS EC2 using Docker, so the same containers work in development and production. Nginx, a free DuckDNS domain and a free Let’s Encrypt certificate make the site publicly available over HTTPS at almost no cost, and the backend and database ports are never exposed to the internet.

LIMITATIONS

7.2 Limitations:-

While the FurniFlex application provides a functional platform for renting furniture online, several limitations have been identified during its development:

1. No Real Payment Gateway:

The Card / UPI / GPay QR payment step is only a simulation for demonstration. No real payment is processed, so the system cannot yet be used for actual business transactions.

2. No Automatic Stock Update:

Products have a stock quantity, but it is not automatically reduced when an order is placed, and rental start, end and return dates of each rented item are not tracked.

3. No Notifications:

Customers do not receive e-mail or SMS confirmations for registration, orders or status changes, and there is no “forgot password” feature.

4. No Reviews or Wishlist:

Customers cannot rate or review products or save items to a wishlist for later.

5. Single-server Deployment:

The whole application runs on one EC2 instance with product images on its local disk. If the instance fails, the site is down until it is restored, and the setup cannot scale horizontally without moving images to shared storage (such as S3) and the database to a managed service.

6. Manual Deployment and No Automated Tests:

Updates are deployed manually over SSH, tables are created with SQLAlchemy’s create_all() instead of migrations, and testing was done manually rather than with an automated test suite.

PLAGIARISM CHART

7.3 Plagiarism Chart:-

Attach the plagiarism scan report for this project here.

FUTURE SCOPE

8.1 Future Scope:-

The FurniFlex web application has laid a solid foundation for renting furniture online. However, several enhancements and new features can be integrated in the future to improve its functionality, security and scalability. Based on the analysis of the current advantages and limitations, the following areas offer potential for future development:

1. Real Payment Gateway Integration:

Integrating a payment gateway such as Razorpay or Stripe with support for UPI AutoPay would allow real payments and automatic monthly rent collection.

2. Inventory and Rental Period Tracking:

Stock could be reduced automatically on each order, and the system could track delivery date, rental end date, renewals and returns of each rented item.

3. E-mail / SMS Notifications and OTP:

Order confirmations, status updates, rent-due reminders, OTP based login and a “forgot password” flow could be added.

4. Reviews, Ratings and Wishlist:

Customers could rate and review products and save favourites to a wishlist, helping other users make better decisions.

5. Scalable Cloud Architecture:

Images could be moved to Amazon S3 with CloudFront CDN, the database to Amazon RDS, and multiple EC2 instances could run behind an Application Load Balancer with a custom paid domain managed in Route 53.

6. Mobile Application:

Because the backend is a documented REST API, a native Android / iOS app could reuse the same API.

7. Maintenance and Damage Requests:

Customers could raise service, relocation or damage requests for rented furniture and track them from their account.

8. Advanced Analytics for Admin:

Charts for monthly revenue, most rented products, category-wise demand and customer retention could be added to the admin dashboard.

9. Automated Testing and CI/CD:

Automated tests with pytest and React Testing Library, database migrations with Alembic and a GitHub Actions pipeline that builds and deploys to EC2 automatically on every push would make future development safer and faster.

CONCLUSION

8.2 Conclusion:-

The development of FurniFlex using React, FastAPI and PostgreSQL provided a comprehensive understanding of modern full-stack web application architecture. Separating the system into a React single-page frontend, a REST API backend and a relational database made each layer easier to design, test and maintain.

Through this project we successfully implemented user registration and JWT based authentication, profile management, category-wise browsing with global search, filters and sorting, rental plans with automatic discounts, a server-side cart, checkout with a simulated payment step, order history and a complete admin panel with product, category, image, order and user management.

FastAPI with Pydantic made it possible to build a fast, validated and self-documenting API, while SQLAlchemy mapped the seven database tables to clean Python classes. On the frontend, React with Material UI enabled a responsive and attractive interface with light and dark themes.

The project was taken beyond local development by deploying it on an AWS EC2 instance. Docker Compose runs the backend and database, Nginx acts as a reverse proxy and serves the React build, DuckDNS provides a free domain name and Certbot secures the site with a free Let's Encrypt SSL certificate. This gave practical experience of Linux servers, cloud networking, DNS, firewalls and HTTPS.

Overall, the project demonstrated how modern open-source technologies and cloud services can be combined to build and host a secure, efficient and user-friendly e-commerce style application. With features such as real payment integration, notifications, inventory tracking and a scalable cloud architecture, FurniFlex can grow into a production-ready furniture rental service.

REFERENCES

8.3 References:-

- **React:** https://react.dev/
- **Vite:** https://vite.dev/guide/
- **Material UI:** https://mui.com/material-ui/getting-started/
- **React Router:** https://reactrouter.com/
- **Axios:** https://axios-http.com/docs/intro
- **FastAPI:** https://fastapi.tiangolo.com/
- **SQLAlchemy:** https://docs.sqlalchemy.org/en/20/
- **Pydantic:** https://docs.pydantic.dev/
- **PostgreSQL:** https://www.postgresql.org/docs/
- **Docker & Docker Compose:** https://docs.docker.com/
- **AWS EC2:** https://docs.aws.amazon.com/ec2/
- **Nginx:** https://nginx.org/en/docs/
- **Certbot:** https://certbot.eff.org/
- **Let's Encrypt:** https://letsencrypt.org/docs/
- **DuckDNS:** https://www.duckdns.org/install.jsp
- **JSON Web Tokens:** https://jwt.io/introduction
- **Python:** https://docs.python.org/3.12/
- **HTML, CSS & JavaScript:** https://developer.mozilla.org/en-US/docs/Web